

سجل المحادثات — zzz.txt

الجزء 2 من 11

الصفحات 81-167 من أصل 913 • 22 محادثة

gpt-5.5-instant — 14	81
gpt-5.5-instant — 15	95
gpt-5.5-instant — 16	100
gpt-5.5-instant — 17	101
gpt-5.5-instant — 18	112
gpt-5.5-instant — 19	119
gpt-5.5-instant — 20	124
gpt-5.5-instant — 21	133
gpt-5.5-instant — 22	138
gpt-5.5-instant — 23	141
gpt-5.5-instant — 24	143
gpt-5.5-instant — 25	145
gpt-5.5-instant — 26	147
gpt-5.5-instant — 27	148
gpt-5.5-instant — 28	151
gpt-5.5-instant — 29	153
gpt-5.5-instant — 30	154
gpt-5.5-instant — 31	155
gpt-5.5-instant — 32	156
gpt-5.5-instant — 33	159
gpt-5.5-instant — 34	163
gpt-5.5-instant — 35	165

Liquidity

↓

Order Blocks

↓

Dealing Range

↓

Hypotheses

وبعدها يصبح النظام مكتملاً بما يكفي لنفعل الامتحان الذي سيقدر إذا تعبنا له قيمة مالية

[text]

ما كان يعرفه النظام عند T

↓

ماذا حدث بعد T

↓

هل يوجد فرق عن Baseline؟

إذا ما وجدنا فرقاً بعد اكتمال المدخلات واختبارها خارج العينة، وقتها أقول لك بوضوح إن النظام الحالي لم يثبت Edge.

أما اليوم، الحكم الأدق هو:

«البنية التقنية والسببية ناجحة بشكل قوي حتى الآن. عدة detectors تعمل فعلياً على BTC الحقيقي. جزء البنية/الموجات غير مكتمل إحصائياً. أما الذكاء التنبؤي والربحية فلم يفشلا ولم ينجح بعد؛ لم ندخل امتحانهما أصلاً.»

شو يعني المشروع شغال صح وناجح ولا فاشل بالاقسام يلي بنيناها لحد الان

gpt-5.5-instant

الحالة الرسمية: أول تشغيل حقيقي للمشروع على BTC اكتمل حسابه فعلياً تقريباً، وبسرعة ممتازة: الجزء الثقيل انتهى خلال نحو دقيقتين. الذي فشل في آخر ثواني فقط هو إنشاء ملف التحقق الآلي بسبب سطر خاص بـ Linux لا يعمل على Windows:

import resource

لذلك هذا ليس فشلاً في المحركات ولا في بيانات BTC. وهو أيضاً يفسر لماذا عندك SUMMARY.json وبقية ملفات النتائج، لكن لا يوجد MACHINE_VALIDATION.json.

ماذا أنجزنا فعلياً؟

لأول مرة المشروع قرأ شهر BTC الحقيقي كاملاً:

[text]

44,640 دقيقة

من 1 مايو إلى نهاية 31 مايو 2026

وقارن مصدرين مستقلين:

[text]

AggTrades الحقيقي عندك

+

Klines الرسمية من Binance

وكانت النتيجة:

[text]

44,640 / 44,640

EXACT MATCH

ثم بنى Timeline حقيقية لـ 44,640 دقيقة وشغل محركات المشروع الموجودة.

وهذا بحد ذاته إنجاز مهم: لم نعد على Fixtures.

السرعة الجديدة ممتازة جداً

على نفس جهازك:

[text]

Dynamic Volatility

قبل: 581.4 ثانية

الآن: 2.4 ثانية

تقريباً أسرع 242 مرة.

والFVG:

[text]

قبل: 1594.3 ثانية

الآن: 5.3 ثانية

تقريباً أسرع 300 مرة.

وكل المحركات الأساسية التي شُغلت:

[text]

52.6 ثانية

ثم Evidence + Narrative:

[text]

14 ثانية

وبقية التقرير عدة ثوانٍ.

يعني انتقلنا فعلياً من:

[text]

أكثر من ساعة وغير مكتمل

إلى:

[text]

حوالي دقيقتين للحساب الحقيقي
هذا تغيير جذري، وليس تحسناً بسيطاً.

ماذا اكتشف المشروع فعلياً؟

أهم رقم جديد:

[text]

FVG = 18,560

على:

[text]

44,640 دقيقة

أي تقريباً:

[text]

416 FVG لكل 1000 شمعة

هذا رقم كبير جداً.

لكن لا نستنتج منه أن FVG جيدة أو سيئة.

بالعكس، أول سؤال بحثي ظهر أمامنا الآن هو:

هل تعريف FVG الحالي واسع جداً لدرجة أنه يكتشف عدداً هائلاً من الحالات؟

أم هذا طبيعي لتعريف المشروع على إطار الدقيقة؟

هذا بالضبط سبب وجود:

entities_fvg.csv

و:

entities_fvg_events.csv

و:

chart_overlay.csv

الآن نستطيع لأول مرة أن ننظر إلى ما يعتبره المشروع FVG على سوق حقيقي بدلاً من النقاش النظري.

وهناك نتيجة مثيرة عن عمر الFVG

عندنا:

[text]

Median lifetime:
30 bars
P25:
9 bars
P75:
160 bars

Mean:
611 bars
لاحظ الفرق الكبير جداً:

[text]

Median = 30

Mean = 611

هذا غالباً يعني أن التوزيع له ذيل طويل جداً: معظم الفجوات قصيرة نسبياً، لكن بعض الفجوات تبقى لفترات طويلة جداً وترفع المتوسط.

وفي نهاية الشهر بقي:

[text]

FVG 96

غير منتهية وفق دورة الحياة.

هذه أول نتيجة وصفية سوقية حقيقية خرجت من المشروع.

وماذا وجدنا في Executed Flow؟

ملفك الحقيقي أعطى:

[text]

إجمالي التنفيذ:

BTC 425,331 تقريباً

مقسمة إلى:

[text]

BUY initiated:

BTC 213,598 تقريباً

SELL initiated:

BTC 211,733 تقريباً

أي إجمالاً كان ال-BUY initiated أعلى بحوالي:

[text]

BTC 1,865

خلال الشهر كله.

لكن:

[text]

mean_delta_ratio = -0.00317

ليس تناقضاً بالضرورة.

الأول يجمع أحجام الشهر كلها.

الثاني يأخذ متوسط نسب الدقائق، وبالتالي دقيقة صغيرة ودقيقة ضخمة قد تدخلان بوزن متساوٍ في المتوسط.

وهذه نقطة بحثية مهمة لاحقاً: لا يجوز اختصار التدفق برقم واحد.

ماذا لم يعمل؟

هذا أهم جزء في التقرير.

المشروع رفض تشغيل:

[text]

Swing Detector

Swing Sequence

Structural Breaks

Liquidity

Order Blocks

Dealing Range

لماذا؟

لأن:

[text]

swing_quantile = null

ولم نخترع رقماً.

يعني حالياً المشروع لا يستطيع أن يقول لنا بشكل شرعي:

[text]

هذه موجة كبيرة

هذه موجة صغيرة

هذا HH

هذا HL

هذا BOS

هذه سيولة

هذا OB

وهذا يعيدنا بالضبط إلى سؤالك السابق:

«كيف يعرف إنه أصلاً فاهم الموجة صح؟»

الجواب الحالي الصادق:

ما زلنا لم نحدد له تعريفاً مُعاييراً للموجة.

وأنا سعيد أنه قال:

NOT_CONFIGURED

بدل أن يخترع موجات.

لذلك أيضاً عندنا:

text

hypotheses_created = 0

لا تفسرها هكذا:

«المشروع لم يجد فرصاً في مايو.»

هذا خطأ.

السرد اشتغل، لكن أجزاء بنيوية أساسية غير مهيأة، وبالتالي النظام الحالي لم ينتج Hypotheses كاملة بهذه التهيئة.

لسنا في وضع يسمح بتقييم العقل أو التداول بعد.

ما الذي اشتغل فعلياً؟

عندك 12 فرعاً عملوا:

[text]

Volatility

Session/calendar facts

FVG

Order Flow PROXY

Order Flow ACTUAL

Absorption/Response ACTUAL

Absorption/Response PROXY

HTF factual aggregation
Evidence ACTUAL
Evidence PROXY
Narrative ACTUAL
Narrative PROXY
و7 فروع بقيت غير مهياة.

يعني أول Field Run نجح جزئياً من ناحية الوظائف، وهذا متوقع.
ماذا عن الخطأ في النهاية؟
الخطأ:

[text]

'ModuleNotFoundError: No module named 'resource'
سبب بسيط جداً.
مكتبة:

resource

موجودة عادة في Linux/Unix، وليست موجودة في Windows.
البيلدر اختبر الRunner على Linux، ثم حاول استخدام المكتبة لقياس Peak Memory.
المشكلة في:

[text]

field_runner/reporting.py
وليست داخل:

trading_project

ولا داخل أي محرك مغلق.
يجب إصلاحه بحيث:

[text]

:Linux

يقيس peak memory

:Windows

يستخدم طريقة Windows إن كانت متاحة
أو يسجل:

PEAK_MEMORY_NOT_AVAILABLE_ON_PLATFORM

ولا يفشل التقرير كله بسبب metric تشخيصية غير أساسية.

هل نحتاج إعادة كل هذا؟

بعد Patch صغير نعم، لكن لا تقلق: التشغيل صار سريعاً.

إعادته ليست ساعة.

على جهازك الآن حوالي دقائق قليلة.

ولكن قبل ذلك أريد إصلاح خطأ Windows فقط، لا أي شيء آخر.

أرسل للبيلدر:

[text]

FIELD RUNNER — WINDOWS REPORTING BLOCKER — PATCH ONLY

REAL OWNER RUN وصل بنجاح حتى آخر مرحلة.

المحركات والحسابات انتهت فعلياً.

الأزمة الحقيقية على Windows:

sources exact cross-witness:

27.6s

dynamic_volatility:
2.4s
fvg:
5.3s
order_flow_proxy:
1.7s
order_flow_actual:
1.8s
absorption_actual:
2.9s
absorption_proxy:
3.0s
HTF 1h:
35.3s
all engines:
52.6s
evidence+narrative:
14.0s
detection audit:
4.4s
outcome film:
0.1s
chart/entities/preview:
6.5s

ثم فشل فقط أثناء MACHINE_VALIDATION:

Traceback:

field_runner/reporting.py line 685
import resource

ModuleNotFoundError:
No module named 'resource'

المالك يعمل Windows 10.

PATCH ONLY للـ field_runner الخارجي.

ممنوع تعديل:

trading_project

أي CLOSED

أي engine

أي performance patch

أي semantics

أي output fact.

المطلوب:

1.

اجعل MACHINE_VALIDATION cross-platform.

2.

ممنوع أن يكون قياس peak memory شرط نجاح للتقرير.

على Unix:
يجوز استخدام resource.

على Windows:
استخدم stdlib/platform-safe mechanism إن كان هناك واحد موثوق.
إذا لا يوجد stdlib exact equivalent:
سجل صراحة:

peak_memory:

```
{  
  "status": "NOT_AVAILABLE_ON_THIS_PLATFORM",  
  "platform": "...",  
  "method": null  
}
```

ولا تختبر قياساً.

3.

لا تضيف dependency جديدة فقط لقياس الذاكرة.

4.

MACHINE_VALIDATION يجب أن يُنتج على Windows بدون resource.

5.

أضف regression test يحاكي غياب module resource.

6.

اختبر أن Linux path الحالي لا يتغير.

7.

أصلح نص progress القديم أيضاً فقط إن كان خارج trading_project:
حالياً يقول:

"exact history scans ... expected TENS OF MINUTES"

وهذا صار قديماً بعد Exact Performance V2.

استبدله بنص محايد لا يدعي زمناً:

".runtime depends on dataset and certified engine implementation; stage timings are reported below"

هذا تعديل رسالة فقط، لا semantics.

8.

لا تغير أي نتيجة في SUMMARY أو CSVs.

9.

ابن ZIP جديد جاهز للمالك.

أعد:

FIELD RUNNER WINDOWS REPORTING PATCHED — READY FOR OWNER RETRY

مع:

- الملفات الخارجية المعدلة.

- الاختبارات.

- ZIP + SHA256.

- trading_project unchanged.

- نفس الأمر:

```
python run_btc_may_2026.py
```

.STOP

ماذا نفعل بعد إعادة التشغيل؟
هنا يبدأ الجزء الممتع فعلاً.

لن نذهب مباشرة إلى AI.

عندي الآن ملف:

chart_overlay.csv

وملف:

entities_fvg.csv

أريد أن نستخدمهما لعمل أول Detection Reality Audit بصري.

ثم نحل أكبر فراغ واضح في النظام:

الموجات.

لكن لا أريد أن يكون الحل:

[text]

swing_quantile = 0.25

لأن هذا يعيدنا للذوق.

أريد أن نفكر في Swing كـ Hierarchy:

[text]

Micro

Local

Major

أو تمثيل متعدد المقاييس مشابه، ثم نختبره على TRAIN بدلاً من تقديس Quantile واحد.

بعدها فقط تشتغل:

[text]

HH/HL/LH/LL

BOS/CHoCH

Liquidity

OB

Dealing Range

Hypotheses

وبعدها يصبح عندنا أول نظام مكتمل بما يكفي لاختبار السؤال الحقيقي:

هل هذه المعلومات تغيّر توزيع المستقبل أم لا؟

أما الآن، فقد حققنا شيئين مهمين جداً:

المشروع اشتغل فعلياً على BTC الحقيقي.

والأداء أصبح عملياً.

المشكلة الحالية مجرد آخر ملف تحقق خاص بـ Windows، وليست فشلاً في التحليل.

Microsoft Windows [Version 10.0.19045.6466]

(c) Microsoft Corporation. All rights reserved.

C:\Users\pc\Desktop\usdt\34_BTC_MAY2026_FIELD_RUNNER_PACKAGE_EXACT_PERF_V2_COMPLETE\BTC_MAY2026_FIELD_PACKAGE>python run_btc_may_2026.py

[07:49:58] BTC MAY 2026 FIELD RUNNER starting

[07:49:58] NOTE: the certified CLOSED engines use expanding-history empirical percentiles

[07:49:58] (exact history scans by contract). At month scale (~44,640 bars) the engine

[07:49:58] stage is expected to take TENS OF MINUTES on a desktop CPU. Each stage below

[07:49:58] prints its own timing so you can see progress; nothing is written until the end.

[07:49:58] STEP inputs: minute_facts=minute_facts_BTCUSDT-aggTrades-2026-05.csv

sidecar=minute_facts_BTCUSDT-aggTrades-2026-05.sidecar.json raw=ABSENT (FAST mode, provenance via sidecar)

[07:49:58] STEP identity gates passed (artifact-role hashes + sidecar bindings)

downloading official artifact:

zzz.txt — سجل المحادثات

```

https://data.binance.vision/data/spot/monthly/klines/BTCUSDT/1m/BTCUSDT-1m-2026-05.zip
downloaded BTCUSDT-1m-2026-05.zip: 2120776 bytes in 5.2s
downloading official artifact:
https://data.binance.vision/data/spot/monthly/klines/BTCUSDT/1m/BTCUSDT-1m-2026-05.zip.CHECKSUM
M
downloaded BTCUSDT-1m-2026-05.zip.CHECKSUM: 88 bytes in 1.1s
[07:50:04] STEP klines ready: BTCUSDT-1m-2026-05.csv
(acquisition=DOWNLOADED_FROM_DATA_BINANCE_VISION_CHECKSUM_VERIFIED)
[07:50:04] STEP loading sources through CLOSED adapters (validate + cross-witness + canonical) ...
[07:50:32] STEP sources loaded + exact cross-witness OK (44640 minutes) in 27.6s
[07:50:34] STEP sealing MarketObservationTimeline (consumer-side public seal) ...
[07:50:34] STEP timeline sealed: 44640 bars in 0.0s (hash a8578565bd4a3962...)
[07:50:34] STEP deterministic sample selected: 100 timestamps
[07:50:34] STEP running CLOSED engines (this is the long stage) ...
[07:50:35] ENGINE dynamic_volatility_1_1 ...
[07:50:37] ENGINE dynamic_volatility_1_1 done in 2.4s
[07:50:37] ENGINE session_context_1_2 ...
[07:50:37] ENGINE session_context_1_2 done in 0.0s
[07:50:37] ENGINE fvg_4_2a ...
[07:50:42] ENGINE fvg_4_2a done in 5.3s
[07:50:42] ENGINE order_flow_proxy_3_1 ...
[07:50:44] ENGINE order_flow_proxy_3_1 done in 1.7s
[07:50:44] ENGINE order_flow_actual_3_1 ...
[07:50:46] ENGINE order_flow_actual_3_1 done in 1.8s
[07:50:46] ENGINE absorption_actual_3_2 ...
[07:50:49] ENGINE absorption_actual_3_2 done in 2.9s
[07:50:49] ENGINE absorption_proxy_3_2 ...
[07:50:52] ENGINE absorption_proxy_3_2 done in 3.0s
[07:50:52] ENGINE htf_5_1_1h ...
[07:51:27] ENGINE htf_5_1_1h done in 35.3s
[07:51:27] STEP engines finished in 52.6s
[07:51:27] STEP evidence vectors + narrative (ACTUAL and PROXY, kept separate) ...
[07:51:27] EVIDENCE 6.1A actual ...
[07:51:27] EVIDENCE 6.1A actual done in 0.1s
[07:51:27] NARRATIVE 6.1B actual ...
[07:51:34] NARRATIVE 6.1B actual done in 6.9s
[07:51:34] EVIDENCE 6.1A proxy ...
[07:51:34] EVIDENCE 6.1A proxy done in 0.1s
[07:51:34] NARRATIVE 6.1B proxy ...
[07:51:41] NARRATIVE 6.1B proxy done in 6.8s
[07:51:41] STEP evidence+narrative finished in 14.0s
[07:51:41] STEP building DETECTION_AUDIT.csv (as-of sample) ...
[07:51:45] STEP audit done in 4.4s
[07:51:50] STEP building OUTCOME_FILM_SAMPLE.csv (separate outcome-side view) ...
[07:51:50] STEP outcome film done in 0.1s
[07:51:50] STEP building chart_overlay.csv + entity exports + reality preview ...
[07:51:57] STEP chart/entities/preview done in 6.5s
[07:51:57] STEP writing outputs (SUMMARY / MACHINE_VALIDATION / README) ...
Traceback (most recent call last):
File
"C:\Users\pc\Desktop\usdt\34_BTC_MAY2026_FIELD_RUNNER_PACKAGE_EXACT_PERF_V2_COMPLETE\BT
C_MAY2026_FIELD_PACKAGE\run_btc_may_2026.py", line 12, in <module>

```

```
raise SystemExit(main())
```

```
File
```

```
"C:\Users\pc\Desktop\usdt\34_BTC_MAY2026_FIELD_RUNNER_PACKAGE_EXACT_PERF_V2_COMPLETE\BTC_MAY2026_FIELD_PACKAGE\field_runner\runner_btc_may_2026.py", line 527, in main
```

```
machine = R.build_machine_validation(
```

```
File
```

```
"C:\Users\pc\Desktop\usdt\34_BTC_MAY2026_FIELD_RUNNER_PACKAGE_EXACT_PERF_V2_COMPLETE\BTC_MAY2026_FIELD_PACKAGE\field_runner\reporting.py", line 685, in build_machine_validation
```

```
import resource
```

```
ModuleNotFoundError: No module named 'resource'
```

```
C:\Users\pc\Desktop\usdt\34_BTC_MAY2026_FIELD_RUNNER_PACKAGE_EXACT_PERF_V2_COMPLETE\BTC_MAY2026_FIELD_PACKAGE>
```

ولملفات :

سوماري دوت جيسون :

}

actual_means": "EXECUTED_INITIATED_FLOW_FROM_BINANCE_SPOT per source semantics only; NOT"
order-book truth, NOT buying pressure, NOT institutional activity, NOT whale activity, NOT market-wide
flow",

"branch_statuses": {

"absorption_actual_3_2": {

"detail": "ACTUAL executed/initiated basis",

"status": "RAN"

},

"absorption_proxy_3_2": {

"detail": "PROXY basis (stays PROXY)",

"status": "RAN"

},

"dealing_range_4_2b": {

"detail": "blocked by unconfigured swing confirmation policy (no invented quantile)",

"status": "NOT_CONFIGURED"

},

"dynamic_volatility_1_1": {

"detail": "",

"status": "RAN"

},

"evidence_vector_6_1a_actual": {

"detail": "flow_mode=ACTUAL; multiscale=False (5.2 outside owner run list)",

"status": "RAN"

},

"evidence_vector_6_1a_proxy": {

"detail": "flow_mode=PROXY; multiscale=False (5.2 outside owner run list)",

"status": "RAN"

},

```
"fvg_4_2a": {
```

```
"detail": "",
```

```
"status": "RAN"
```

```
},
```

```
"htf_factual_aggregation_5_1": {
```

```
"detail": "scales=['1h']",
```

```
"status": "RAN"
```

```
},
```

```
"liquidity_2_2": {
```

```
"detail": "blocked by unconfigured swing confirmation policy (no invented quantile)",
```

```
"status": "NOT_CONFIGURED"
```

```
},
```

```

"narrative_hypotheses_6_1b_actual": {
  "detail": "",
  "status": "RAN"
},
"narrative_hypotheses_6_1b_proxy": {
  "detail": "",
  "status": "RAN"
},
"order_blocks_4_1": {
  "detail": "blocked by unconfigured swing confirmation policy (no invented quantile)",
  "status": "NOT_CONFIGURED"
},
"order_flow_actual_3_1": {
  "detail": "EXECUTED_INITIATED_FLOW_FROM_BINANCE_SPOT \u2014 EXECUTED_INITIATED_FLOW_FROM_BINANCE_SPOT
(source semantics only); NOT order-book truth, NOT buying pressure, NOT institutional activity, NOT
whale activity, NOT market-wide flow",
  "status": "RAN"
},
"order_flow_proxy_3_1": {
  "detail": "OHLCV_PROXY (stays PROXY)",
  "status": "RAN"
},
"session_context_1_2": {
  "detail": "session schedules NOT_CONFIGURED (calendar features still RAN)",
  "status": "RAN"
},
"structural_breaks_2_1c": {
  "detail": "blocked by unconfigured swing confirmation policy (no invented quantile)",
  "status": "NOT_CONFIGURED"
},
"swing_detector_2_1a": {
  "detail": "blocked by unconfigured swing confirmation policy (no invented quantile)",
  "status": "NOT_CONFIGURED"
},
"swing_sequence_2_1b": {
  "detail": "blocked by unconfigured swing confirmation policy (no invented quantile)",
  "status": "NOT_CONFIGURED"
},
"swings_branch_summary": {
  "detail": "swing confirmation policy requires an explicit quantile; none configured (owner parameter
swing_quantile)",
  "status": "NOT_CONFIGURED"
}
},
"counts": {
  "confirmed_swings_high": null,
  "confirmed_swings_low": null,
  "confirmed_swings_total": null,
  "dealing_ranges": null,
  "evidence_vectors_actual_rows": 44640,
  "evidence_vectors_proxy_rows": 44640,
  "fvg_entities": 18560,
  "hypotheses_created": 0,
  "hypotheses_state_counts_at_data_end": {},
  "liquidity_events_by_type": {},
  "liquidity_levels": null,
  "order_block_candidates": null,
  "sequence_events_HH_HL_LH_LL": null,
  "session_rows": 44640,
  "session_schedules_configured": "session schedules NOT_CONFIGURED (calendar features still RAN)",
  "structure_events_BOS_CHOCH": null,
  "volatility_rows": 44640,
  "volatility_states": null
},
"detector_validity_notice": "Detector output = operational project definition (module version bound per
entity). It does NOT mean 'the only market truth'. Entities are reported as 'detector version X under
this run', never as 'detector works/fails'.",
"flow_summaries_SEPARATE_NEVER_MERGED": {
  "ACTUAL_executed_initiated": {
    "label": "EXECUTED_INITIATED_FLOW_FROM_BINANCE_SPOT",
    "mean_delta_ratio": -0.003169341593186486,
    "rows": 44640,
    "semantic_scope": "EXECUTED_INITIATED_FLOW_FROM_BINANCE_SPOT per source semantics only; NOT order-book
truth, NOT buying pressure, NOT institutional activity, NOT whale activity, NOT market-wide flow",

```

```

"total_buy_volume": 213598.04666000005,
"total_sell_volume": 211732.98373000004,
"total_volume": 425331.03038999997
},
"PROXY_ohlcv_proxy": {
"label": "OHLCV_PROXY (stays PROXY)",
"mean_volume_pressure_proxy": -0.0016970790059499486,
"rows": 44640
},
},
"kline_identity": {
"acquisition": "DOWNLOADED_FROM_DATA_BINANCE_VISION_CHECKSUM_VERIFIED",
"checksum_path":
"C:\Users\pc\Desktop\usdt\34_BTC_MAY2026_FIELD_RUNNER_PACKAGE_EXACT_PERF_V2_COMPLETE\BTC_MAY2026_FIELD_P
ACKAGE\data\klines_cache\BTCUSDT-1m-2026-05.zip.CHECKSUM",
"contract": "BINANCE_SPOT_PUBLIC_DATA_KLINES_MONTHLY_1M_ZIP",
"csv_path":
"C:\Users\pc\Desktop\usdt\34_BTC_MAY2026_FIELD_RUNNER_PACKAGE_EXACT_PERF_V2_COMPLETE\BTC_MAY2026_FIELD_P
ACKAGE\data\klines_cache\BTCUSDT-1m-2026-05.csv",
"klines_csv_sha256": "6d804daafd396cf7714830b369ba5a9dacf264979b353d988b2fc6685fbeb1b3d",
"ohlcv_source_label": "PUBLISHED_KLINE_OHLC",
"source_url": "https://data.binance.vision/data/spot/monthly/klines/BTCUSDT/1m/BTCUSDT-1m-2026-05.zip",
"symbol": "BTCUSDT",
"year_month": "2026-05",
"zip_path":
"C:\Users\pc\Desktop\usdt\34_BTC_MAY2026_FIELD_RUNNER_PACKAGE_EXACT_PERF_V2_COMPLETE\BTC_MAY2026_FIELD_P
ACKAGE\data\klines_cache\BTCUSDT-1m-2026-05.zip",
"zip_sha256": "62983d93c35e26ec6c29116c629e443b7ddfe3d301aca83331ff58112199d8d0"
},
"minute_facts_count": 44640,
"not_configured_items": [
"swing_detector_2_1a",
"swing_sequence_2_1b",
"structural_breaks_2_1c",
"liquidity_2_2",
"order_blocks_4_1",
"dealing_range_4_2b",
"swings_branch_summary"
],
"reality_check_preview": {
"coverage": {
"timelineBars": 44640,
"timeline_end": "2026-06-01 00:00:00+00:00",
"timeline_start": "2026-05-01 00:01:00+00:00"
},
"entity_frequencies_per_1000_bars": {
"confirmed_swings": null,
"dealing_ranges": null,
"fvg_candidates": 415.7706,
"liquidity_levels": null,
"ob_candidates": null
},
},
"excursion_distributions": {
"status": "NOT_CONFIGURED"
},
},
"forbidden_absent": [
"no_horizon_selection",
"no_threshold_selection",
"no_combination_selection",
"no_strategy_optimization",
"no_profitability_claim"
],
"hypothesis_outcome_factual_distributions": {
"censoring_rate_note": "MONITORING at data end = still open under the closed lifecycle; not a loss and
not a win",
"state_counts_at_data_end": {},
"terms": "CLOSED 6.1B literal states only (MONITORING / OBSERVED_DIRECTION_ESTABLISHED / CONTRADICTED /
SUPERSEDED); no WIN/LOSS/PnL"
},
"lifecycle_durations_bars": {
"dealing_ranges": {
"status": "NOT_CONFIGURED"
},
},
},
"fvg": {

```

```

"durationBarsMean": 611.5802262931035,
"durationBarsMedian": 30.0,
"durationBarsP25": 9.0,
"durationBarsP75": 160.0,
"durationNote": "bars from creation to last terminal event, else to data end (censored)",
"entityCount": 18560,
"openAtDataEndCount": 96,
"status": "RAN"
},
"orderBlocks": {
"status": "NOT_CONFIGURED"
}
},
"scope": "DESCRIPTIVE_ONLY_NO_INFORMATION_EDGE_CLAIM"
},
"report": "BTC_MAY_2026_FIELD_REPORT",
"sampleSelection": {
"algorithm": "SHA256_TIMELINE_HASH_SLASH_K_MOD_BAR_COUNT_DISTINCT_ASCENDING",
"barCount": 44640,
"parameterStatus": "EXPLICIT_OWNER_PARAMETER_NOT_SACRED",
"positions": [
439,
480,
1971,
2075,
2328,
2330,
2431,
2585,
4090,
4237,
4370,
4578,
4626,
4774,
5202,
5552,
5593,
5673,
6158,
6579,
6618,
7272,
7953,
8081,
8162,
8203,
8516,
8565,
10627,
10657,
10746,
10862,
10876,
11015,
11094,
11852,
12891,
13178,
13447,
13498,
13590,
14193,
14726,
15683,
15822,
16828,
17135,
17212,
17718,
18051,
18415,
18880,
19841,

```

```

20051,
20470,
20642,
21324,
21596,
21696,
22313,
24737,
25214,
25293,
27014,
28366,
30009,
30188,
30840,
32495,
32964,
33046,
33544,
33907,
34389,
34917,
35034,
35237,
35424,
35508,
36013,
36191,
36421,
36733,
38275,
38502,
38655,
39071,
39169,
39183,
39391,
39478,
39750,
39874,
40855,
41486,
42085,
42402,
42435,
42704,
43869
],
"sample_size_parameter": 100,
"sample_size_realized": 100,
"timeline_hash": "a8578565bd4a3962734725b66cf5a961857327999e403ece54c8942fe53e6275"
},
"source_identity": {
"cross_witness": {
"compared_minutes": 44640,
"result": "EXACT_MATCH_FAIL_CLOSED_ENFORCED"
},
"facts_provenance": {
"converter_name": "aggtrades_to_minute_facts",
"minute_facts_sha256": "a6a06c4c583e64e7613ddca275533a15062ab13864482cacf30b2f125094601f",
"raw_source_sha256": "86d4f3d335ae244dcc143569bd1cc382320027c3fe4702e5d3ddf0757e8f1c04"
},
"minute_facts_csv":
"C:\Users\pc\Desktop\usdt\34_BTC_MAY2026_FIELD_RUNNER_PACKAGE_EXACT_PERF_V2_COMPLETE\BTC_MAY2026_FIELD_P
ACKAGE\data\minute_facts_BTCUSD-aggTrades-2026-05.csv",
"minute_facts_sha256": "a6a06c4c583e64e7613ddca275533a15062ab13864482cacf30b2f125094601f",
"minute_facts_sidecar":
"C:\Users\pc\Desktop\usdt\34_BTC_MAY2026_FIELD_RUNNER_PACKAGE_EXACT_PERF_V2_COMPLETE\BTC_MAY2026_FIELD_P
ACKAGE\data\minute_facts_BTCUSD-aggTrades-2026-05.sidecar.json",
"minute_facts_sidecar_sha256": "bdcf5d4d1cca85eb0c16lead2eb36ea080546c56b1481d5a20f6ed3b040689f8",
"raw_aggtrades_csv": null,
"raw_aggtrades_sha256": null
},
"status": "FIELD_RUN_OUTPUT",

```

```

"status_counts": {
"NOT_CONFIGURED": 7,
"RAN": 12
},
"tie_order_contract": "NOT_PROVEN",
"timeline": {
"bar_count": 44640,
"timeline_hash": "a8578565bd4a3962734725b66cf5a961857327999e403ece54c8942fe53e6275",
"timeline_id": "BINANCE_SPOT_BTCUSDT_1M_2026-05-01T00:00:00Z"
}
}

```

باقي لملفات هيك اسمائهن :
 chart_overlay.csv
 DETECTION_AUDIT.csv
 entities_fvg.csv
 entities_fvg_events.csv
 OUTCOME_FILM_SAMPLE.csv

gpt-5.5-instant

الحالة الرسمية: هذا التشغيل ممتاز حتى الآن. أنت فعلياً تشغّل النسخة السريعة على بيانات BTC الحقيقية، وقد عبرت البيانات 44,640/44,640 والمصادر تطابقت. الرقع ما زالت غير مغلقة رسمياً، لذلك نعامل النتائج كتجربة مرشحة، لكن من ناحية الأداء حصل تحول ضخم.

أول شيء: السرعة الجديدة ليست تحسناً بسيطاً. قارن جهازك نفسه:
 قبل:

[text]

Dynamic Volatility

581.4 ثانية

≈ 9 دقائق و41 ثانية

الآن:

[text]

2.4 ثانية

يعني تقريباً 242 مرة أسرع عندك.

وFVG:

قبل:

[text]

1594.3 ثانية

≈ 26 دقيقة و34 ثانية

الآن:

[text]

5.3 ثانية

يعني تقريباً 301 مرة أسرع.

هذا أفضل حتى من تقدير البيلدز.

ومن بداية التشغيل:

[text]

07:49:58

إلى وصوله لبناء المخرجات:

[text]

07:51:50

حوالي دقيقتين فقط.

هذا بالضبط النوع من السرعة الذي كنت تطلبه، ومن دون أننا قللنا عدد شموع مايو أو استخدمنا window أقصر أو approximation.

الترتيب الحالي منطقي جداً:

[text]

1. تحقق ملفاتك وبصماتها

↓

2. تنزيل Klines الرسمية والتحقق منها

↓

3. مقارنة Klines مع Minute Facts

↓

4. بناء Timeline السببي

↓

5. تشغيل محركات الحقائق

↓

6. Evidence

↓

7. Narrative

↓

8. Detection Audit

↓

9. Outcome Film

↓

10. ملفات الرسم والكيانات والتقارير
وهذا فصل ممتاز بين:

[text]

ما كان معروفاً وقتها
وبين:

[text]

ماذا حدث بعد ذلك
الآن هو في آخر مرحلة تقريباً:

[text]

building chart_overlay.csv
+ entity exports
+ reality preview

هذه غالباً كتابة وتجميع ملفات، وليست إعادة تشغيل الحساب كله.

لكن هناك شيء مهم جداً ظهر من السجل.

لاحظ المحركات التي لم تظهر:

[text]

Swing

Swing Sequence

Structural Breaks

Liquidity

Order Blocks

Dealing Range

وهذا متوقع؛ لأننا تركنا:

[text]

swing_quantile = null
بمعنى أن المشروع رفض اختراع تعريف للموجة من عنده.
لذلك أول تقرير لن يكون «كل ICT اشتغل».
سيكون أقرب إلى:

[text]

Volatility	✓
Session	✓
FVG	✓
Executed Flow	✓
Proxy Flow	✓
Absorption	✓
HTF factual	✓

Swings	NOT_CONFIGURED
Structure	NOT_CONFIGURED
Liquidity	NOT_CONFIGURED
Order Blocks	NOT_CONFIGURED
Dealing Range	NOT_CONFIGURED

وهذا مهم جداً بخصوص سؤالك القديم:

«بركي هو أصلاً مو عارف الموجة الكبيرة من الصغيرة؟»

صحيح. نحن لم نسمح له يتظاهر أنه يعرفها قبل أن نحل تعريف Scale/Policy بشكل علمي.

وفيه عنق سرعة جديد ظهر، لكنه صغير حالياً:

[text]

HTF 1h = 35.3 ثانية

بينما كل المحركات الأخرى أصبحت ثواني.

يعني إذا أردنا التحسين لاحقاً، HTF صار المرشح الجديد. لكن لا ألمسه الآن؛ 35 ثانية مقبولة لأول Reality Run، ولا أريد نرجع لهوس تحسين كل ثانية قبل رؤية النتائج.

الـ Narrative أيضاً:

[text]

ثانية 6.9 + 6.8 = 13.7

مقبول جداً.

الأهم: ACTUAL و PROXY بقيا منفصلين:

[text]

Evidence actual
Narrative actual

Evidence proxy

Narrative proxy

وهذا صحيح. ما عم نخلط تدفق التداولات الحقيقي حسب Binance مع الاستدلال التقريبي من الشموع.

هلق لا توقفه.

أنت على آخر مرحلة على الأغلب.

لما يخلص، المفروض تشوف رسالة نجاح وبرجع:

[text]

C:\...>

وقتها نحن لا نحتاج البيلدر ولا أوديتور مباشرة.

أرسل لي ثلاثة ملفات/محتويات:

[text]

SUMMARY.json

text

MACHINE_VALIDATION.json

text

README_RESULT_AR.txt

ومن SUMMARY.json تحديداً أريد أعرف لأول مرة أرقام السوق الحقيقية، مثل:

[text]

كم FVG وجد؟

كم Hypothesis؟

ما حالات Narrative؟

كيف توزع Volatility؟

ماذا قال ACTUAL Flow؟

كم UNKNOWN / NOT_CONFIGURED؟

وبعدها نفتح:

[text]

chart_overlay.csv

ونبدأ نجاب سؤالك الحقيقي، وليس سؤال الكود:

هل ما يراه المشروع على BTC منطقي أصلاً؟

ثم عندنا قرار استراتيجي مهم جداً: كيف نبني مفهوم الموجات بطريقة متعددة المقاييس بدلاً من اختيار quantile عشوائي. لكن لا نقرر أي شيء قبل أن نرى محصول التشغيل الحالي.

ومن ناحية الأداء فقط: النتيجة حتى الآن قوية جداً. الانتقال في FVG من 26 دقيقة إلى 5 ثوانٍ مع نفس حجم البيانات هو بالضبط الاتجاه الصحيح لجعل سنوات من البحث قابلة للتنفيذ.

تمام هي لنسخه يلي عم استخدمها

BTC_MAY2026_FIELD_RUNNER_PACKAGE_EXACT_PERF_V2_COMPLETE.zip_34

833KB, 235) sha256: c7b8bb8ae56d2479c53ff37e7b28b0183782140c857bf31447fd61c2cdd611ad
(ملفاً)

المحتوى (بنية حزمة الـ 30 نفسها + آخر كل شيء):

trading_project / كامل — بكل patches الخمسة: الـ 3 محركات (causal_percentile / fvg / narrative) + الـ 2

تسريع التيسر (hashing / manifest_identity) + tests/_patch_reference_src / الخمسة المثبتة

field_runner/ +

run_btc_may_2026.bat

(الأمر الواحد كما هو — ما تغيّر)

UPDATE_NOTES_2026-09-29.txt

— ملخص كل تعديل + جدول الـ 10 ملفات بـ sha256 بعد patches + أزمدة التيسر المتوقعة عندك (~20-35 دقيقة بدل +3 ساعات)

/reports/perf_lab + reports/32+33 / (سكرينات القياس ونتائجها لإعادة إنتاج كل رقم)

235/235 — FILE_INVENTORY.sha256 مطابقة (تحققت من الاستخراج التنظيف كما ستفتحه أنت)

نقطة صراحة مهمة: أثناء التغليف اكتشفت أن حارس الـ manifest في runner_tests كان يفترض «183/183 مطابقة» — أي أحمر منذ أول patch مشروع ولا يقبل أي حالة مُرقّعة مؤثّقة. حوّلت له لقفل أقوى مو أضعف: بصمة MANIFEST.sha256 نفسها مثبتة (الملف لم يُعدّل — 183 سطرًا) + جدول الـ 10 ملفات المُرقّعة بـ sha256 المؤثّق حرفياً — أي انحراف غير هالعهشرة = أحمر فوراً. النتيجة: 31/31 بوابات الـ runner خضراء، والتفصيل مسجّل في ملحق القسم 8 من تقرير 33.

التحقق من داخل الحزمة بعد استخراج تنظيف: MANIFEST pinned ✓ | الـ 10 بـ sha256 المؤثّق ✓ | 5 مراجع ✓ |

31/31 runner ✓ | inventory 235/235 ✓ | اختبارات patches الخمسة خضراء ✓.

كل شيء يبقى PENDING_OWNER_RE_AUDIT — الإغلاق قرارك في شات التدقيق. أمرك للتشغيل على الجهاز ما تغيّر:

run_btc_may_2026.bat

.

وهي الـ cmd هلق

Microsoft Windows [Version 10.0.19045.6466]

سجل المحادثات — zzz.txt

```
C:\Users\pc\Desktop\usdt\34_BTC_MAY2026_FIELD_RUNNER_PACKAGE_EXACT_PERF_V2_COMPLETE\BTC
_MAY2026_FIELD_PACKAGE>python run_btc_may_2026.py
[07:49:58] BTC MAY 2026 FIELD RUNNER starting
[07:49:58] NOTE: the certified CLOSED engines use expanding-history empirical percentiles
[07:49:58] (exact history scans by contract). At month scale (~44,640 bars) the engine
[07:49:58] stage is expected to take TENS OF MINUTES on a desktop CPU. Each stage below
[07:49:58] prints its own timing so you can see progress; nothing is written until the end.
[07:49:58] STEP inputs: minute_facts=minute_facts_BTCUSDT-aggTrades-2026-05.csv
sidecar=minute_facts_BTCUSDT-aggTrades-2026-05.sidecar.json raw=ABSENT (FAST mode, provenance
via sidecar)
[07:49:58] STEP identity gates passed (artifact-role hashes + sidecar bindings)
downloading official artifact:
https://data.binance.vision/data/spot/monthly/klines/BTCUSDT/1m/BTCUSDT-1m-2026-05.zip
downloaded BTCUSDT-1m-2026-05.zip: 2120776 bytes in 5.2s
downloading official artifact:
https://data.binance.vision/data/spot/monthly/klines/BTCUSDT/1m/BTCUSDT-1m-2026-05.zip.CHECKSU
M
downloaded BTCUSDT-1m-2026-05.zip.CHECKSUM: 88 bytes in 1.1s
[07:50:04] STEP klines ready: BTCUSDT-1m-2026-05.csv
(acquisition=DOWNLOADED_FROM_DATA_BINANCE_VISION_CHECKSUM_VERIFIED)
[07:50:04] STEP loading sources through CLOSED adapters (validate + cross-witness + canonical) ...
[07:50:32] STEP sources loaded + exact cross-witness OK (44640 minutes) in 27.6s
[07:50:34] STEP sealing MarketObservationTimeline (consumer-side public seal) ...
[07:50:34] STEP timeline sealed: 44640 bars in 0.0s (hash a8578565bd4a3962...)
[07:50:34] STEP deterministic sample selected: 100 timestamps
[07:50:34] STEP running CLOSED engines (this is the long stage) ...
[07:50:35] ENGINE dynamic_volatility_1_1 ...
[07:50:37] ENGINE dynamic_volatility_1_1 done in 2.4s
[07:50:37] ENGINE session_context_1_2 ...
[07:50:37] ENGINE session_context_1_2 done in 0.0s
[07:50:37] ENGINE fvg_4_2a ...
[07:50:42] ENGINE fvg_4_2a done in 5.3s
[07:50:42] ENGINE order_flow_proxy_3_1 ...
[07:50:44] ENGINE order_flow_proxy_3_1 done in 1.7s
[07:50:44] ENGINE order_flow_actual_3_1 ...
[07:50:46] ENGINE order_flow_actual_3_1 done in 1.8s
[07:50:46] ENGINE absorption_actual_3_2 ...
[07:50:49] ENGINE absorption_actual_3_2 done in 2.9s
[07:50:49] ENGINE absorption_proxy_3_2 ...
[07:50:52] ENGINE absorption_proxy_3_2 done in 3.0s
[07:50:52] ENGINE htf_5_1_1h ...
[07:51:27] ENGINE htf_5_1_1h done in 35.3s
[07:51:27] STEP engines finished in 52.6s
[07:51:27] STEP evidence vectors + narrative (ACTUAL and PROXY, kept separate) ...
[07:51:27] EVIDENCE 6.1A actual ...
[07:51:27] EVIDENCE 6.1A actual done in 0.1s
[07:51:27] NARRATIVE 6.1B actual ...
[07:51:34] NARRATIVE 6.1B actual done in 6.9s
[07:51:34] EVIDENCE 6.1A proxy ...
[07:51:34] EVIDENCE 6.1A proxy done in 0.1s
[07:51:34] NARRATIVE 6.1B proxy ...
```

[07:51:41] NARRATIVE 6.1B proxy done in 6.8s
[07:51:41] STEP evidence+narrative finished in 14.0s
[07:51:41] STEP building DETECTION_AUDIT.csv (as-of sample) ...
[07:51:45] STEP audit done in 4.4s
[07:51:50] STEP building OUTCOME_FILM_SAMPLE.csv (separate outcome-side view) ...
[07:51:50] STEP outcome film done in 0.1s
[07:51:50] STEP building chart_overlay.csv + entity exports + reality preview ...

لسا عم يشتغل ما خلص
شو رايك بلترتيب ؟

gpt-5.5-instant

الحالة الرسمية: معك نسخة الأداء السريعة على جهازك، والملفان الحقيقيان موجودان عندك. بما أنك تريد تجربة السرعة الآن وترك التدقيق لوقت لاحق، لا نحتاج أي إعداد إضافي إذا الحزمة التي نزلتها هي الحزمة الجديدة التي تحتوي رقع الأداء فعلاً. لكن انتبه لنقطة واحدة: التقرير رقم 33 هو حالة PENDING RE-AUDIT، لذلك هذا التشغيل تجريبي لقياس السرعة ورؤية الناتج، وليس اعتماداً نهائياً للرقعة بعد.

اعمل التالي بالضبط.

افتح CMD داخل جذر الحزمة، أي المجلد الذي يحتوي:

[text]

run_btc_may_2026.py

field_runner

trading_project

data

وتأكد أن داخل data موجودان:

[text]

minute_facts_BTCUSDT-aggTrades-2026-05.csv

minute_facts_BTCUSDT-aggTrades-2026-05.sidecar.json

بعدها شغل:

[cmd]

python run_btc_may_2026.py

فقط.

لا تصف أي خيارات.

لا تستخدم:

[text]

allow-hash-mismatch--

ولا تغير swing_quantile

المفروض يبدأ تقريباً:

[text]

BTC MAY 2026 FIELD RUNNER starting

STEP inputs ...

... STEP identity gates passed

ثم ينزل ملف Klines الصغير من Binance أو يستخدم الموجود، وبعدها:

[text]

STEP sources loaded ...

STEP timeline sealed ...

... STEP running CLOSED engines

الشيء الذي نريد مراقبته هذه المرة هو الوقت.

في التشغيل القديم كان:

[text]

Dynamic Volatility = 581 ثانية

FVG = 1594 ثانية

أما النسخة السريعة، بحسب قياسات البيلدر، نتوقع انخفاضاً هائلاً. لا أريد أن أعدك بالرقم قبل جهازك، لكن إذا رأيت مثلاً:

[text]

Dynamic Volatility done in عشرات الثواني

و:

[text]

FVG done in عشرات الثواني

فهذا دليل عملي أن الرقعة وصلت للحزمة الصحيحة وتعمل.

إذا بقي:

[text]

Dynamic Volatility

حوالي 9-10 دقائق مرة ثانية، توقف عندها وأخبرني؛ غالباً وقتها الحزمة التي عندك لا تحتوي نسخة الإنتاج السريعة.

وعندما ينتهي، المفروض يظهر:

[text]

outputs\btc_may_2026_reality

لا تعيد تشغيله.

أرسل لي أولاً:

[text]

SUMMARY.json

MACHINE_VALIDATION.json

README_RESULT_AR.txt

وأرسل أيضاً أسطر CMD التي تحتوي أزمنة المحركات، خصوصاً:

[text]

dynamic_volatility

fvg

order_flow

narrative

وقتها سنعرف شيئين معاً: هل التسريع فعلاً وصل لجهازك، وماذا قال المشروع لأول مرة عن BTC الحقيقي.

تركلي الاوديتور هلق هات شلون شغلو نزلت الزيففايل وفكيت ضغطو علىكمبيوتر ونقلت الملفين المطلوبين

gpt-5.5-instant

الحالة الرسمية: صار عندنا الآن مجموعتان من تحسينات غير مغلقة تنتظر التدقيق، وليس واحدة:

الأولى Performance V2 للمحركات نفسها:

.causal_percentile + FVG + narrative

والثانية Test-Suite Acceleration:

.research/hashing + manifest_identity

لا ترسل عقد التدقيق السابق كما هو، لأنه صار ناقصاً بعد إضافة المجموعة الثانية. والأهم: البيلدر توسع إلى ملفين إضافيين مغلقين بعد تقرير Performance V2. من كلامه يبدو أنه كان عنده «تفويض المالك الأخير» لتسريع الاختبارات، لكن بالنسبة لخريبتنا يجب أن نتعامل معها كرقعة منفصلة داخل نفس إعادة التدقيق، لا نخلط نجاح السرعة مع صحة الشهادة.

الخبر الجيد أن النتيجة التقنية واعدة جداً. صار عندنا ثلاث طبقات تسريع:

[text]

المحركات نفسها:

Tracker أسرع ~184x

FVG أسرع ~160x

التحقق:/Hashing
Test suite أسرع ~2.1-2.4x

:Narrative

تحسين وصول، بلا تغيير معلن
وهذا أقرب لما نريده: لا نخدع الحساب، بل نزيل العمل المكرر.

في موضوع الخيارين اللذين تركهما البيلدر:

لا أوافق الآن على تحويل fixtures إلى module-scoped. ليس لأن الفكرة سيئة، بل لأننا لا نحتاجها الآن، وقد تقلل إعادة البناء المستقلة التي كانت تكشف state leakage.
أوافق لاحقاً على تجربة pytest-xdist كطريقة تشغيل اختيارية فقط، وليس كتغيير للشهادة أو للتستات. تشغيل نفس الاختبارات بالتوازي لا يخفف asserts، وقد يكشف أصلاً مشاكل isolation. لكن لا أريد إدخاله قبل قبول الرقع الحالية.
يعني Freeze الآن. لا تحسين إضافي قبل التدقيق.

وأريد تدقيقاً واحداً موحداً على المجموعتين، لا أوديتين منفصلين يتوهان عن بعضهما. أرسل للأوديتور هذا العقد بدل السابق:

[text]

INDEPENDENT READ-ONLY RE-AUDIT
EXACT PERFORMANCE V2 + TEST-SUITE ACCELERATION

:الحالة

PATCHED — PENDING RE-AUDIT

:FREEZE

لا تغييرات إضافية أثناء التدقيق.

يوجد نطاقان منفصلان يجب تدقيقهما داخل نفس الأثر:

A) ENGINE PERFORMANCE PATCH

Production:

src/trading_system/core/causal_percentile.py
src/trading_system/zones/fvg.py
src/trading_system/decision/narrative.py

B) VERIFICATION PERFORMANCE PATCH

Production/research verification:

src/trading_system/research/hashing.py
src/trading_system/research/manifest_identity.py

واختبارات هذه الرقع + نسخ pre-patch المرجعية فقط.

الهدف الوحيد:

إثبات أن جميع التغييرات Performance-only.

ZERO SEMANTIC CHANGE

ZERO CAUSAL CHANGE

ZERO HASH/IDENTITY CHANGE

ZERO VERIFICATION WEAKENING

لا تعدّل أي ملف.

لا تصلح.

لا تتق بتقارير البيلدر.

Baseline MANIFEST قبل الرقع:

entries 183

SHA256:

7796a73fc30fc311902d7ba8e033eb1699a73c5db10e824d02653fbdd1681587

MANIFEST نفسه يجب ألا يكون قد عُدّل أثناء PATCH.

PART A — ARTIFACT / SCOPE GATE

احصر كل الملفات المعدلة/الجديدة فعلياً.
أي production change خارج الملفات الخمسة أعلاه = BLOCKER ما لم يكن اختباراً مصرحاً به.
تحقق أن كل CLOSED غير المستهدفة مطابقة.
حدد stale MANIFEST entries بدقة:
يجب أن تقابل الملفات المغلقة المعدلة المصرح بها واختباراتها الحالية فقط.
أي stale غير مفسر = BLOCKER.
لا تعتبر stale المتوقع أثناء PATCH إغلاقاً أو فساداً بحد ذاته.
تحقق من: pre-patch references
/tests/_patch_reference_src
لكن لا تثق بها لمجرد الاسم.
طابق SHA256 المرجع مع baseline artifact التاريخي/الأختام/الأثر السابق قدر الممكن.

PART B — CAUSAL PERCENTILE ORACLE

ابن oracle مستقل بسيط من دلالة النسخة القديمة:
historical prefix scan فقط.

rank:

read-before-push.

mid-rank:

$(\text{less} + 0.5 * \text{equal}) / N$.

percentile_value:

`np.quantile(history, q, method="linear").`

اختبر old/reference/new على:

empty

one row

duplicates

all equal

heavy ties

increasing

decreasing

alternating

random seeded

signed +0/-0 permutations

subnormals

extreme finite values

nextafter neighborhoods

NaN skip

NaN raise

bool rejection

+inf/-inf rejection

reset

history_snapshot

repeated reset.

```
=====
PART C — MAX_HISTORY
```

```
=====
CRITICAL PUBLIC CONTRACT.
```

اختبر:
max_history = 1,2,3,7
.random small sizes
هاجم:
evicted minimum
evicted maximum
duplicate extrema
signed zero extrema
alternating extremes
.ties after eviction
كل:
rank
sample_count
min/max
snapshot
percentile_value
reset
يجب أن يطابق القديم.
اختلاف واحد قانوني = PATCH REQUIRED.

```
=====
PART D — PERCENTILE_VALUE
=====
```

اختبر آلاف q × histories مستقلة مباشرةً مقابل:

```
np.quantile(
    np.array(history, dtype=float64),
    q,
    method="linear"
)
q:
0
1
0.5
random
nextafter
fractional h cases.
```

ابحث عن اختلاف:
rounding order
overflow
underflow
signed zero
subnormal
.adjacent floats

=====

PART E — FVG DIFFERENTIAL

=====

هذه أكثر رقعة engine خطراً لأن التنفيذ تغير من scan per-bar/per-entity إلى .vectorized first-occurrence
ولد آلاف OHLC sequences مستقلة تشمل:

flat
trend
alternating
large gaps
dense FVG
wick interactions
close interactions
same-price boundaries
creation then immediate interaction
multiple active entities
long-lived entities
late reclaim
end-of-data entities
signed-zero إن كان قانونياً.

OLD vs NEW exact:

bar frame
event frame
entities
IDs
event ordering
availability
origins
positions
lifecycle
all counters
dtype/index/columns.

=====

PART F — FVG EVENT ORDER / TIME

=====

أثبت أن final sort keys تشكل total deterministic order لكل الأحداث القانونية.

هاجم وجود أكثر من event:
bar نفس
entity نفس
مختلف event type.

اختبر .same-bar creation/touch/close/reclaim boundaries

لا vectorized search يسمح بحدث أبكر ببار واحد.

Full vs prefix vs adversarial future suffix:

≤T exact.

=====

PART G — NARRATIVE DIFFERENTIAL

=====

OLD vs NEW على evidence frames مستقلة:

mixed legal dtypes
pd.NA
np.nan
bool
strings/enums القانونية
DatetimeIndex
integer labels not equal positions
sliced indexes
non-zero-start indexes
.UNAVAILABLE/UNKNOWN evidence
قارن exact:

hypothesis IDs
creation time
states
contradiction
supersession
provenance
ordering
all outputs.

هاجم positional off-by-one عمداً.

=====

PART H — HASHING SECURITY ORACLE

=====

هذا CRITICAL لأنه جزء من جهاز الإثبات.
لا يكفي أن tests الحالية خضراء.
استخدم hashing_pre.py كمرجع فقط بعد التحقق من أصالته.
ولد DataFrames adversarial تشمل:

float64
float32 إن كان قانونياً
signed zero
NaN variants
subnormals
huge finite
nullable Int64
nullable Float64
boolean / nullable boolean
strings
empty strings
pd.NA
object mixed legal values
Enum
tz-aware Timestamp
Timedelta
empty frames
empty columns
non-default indexes إذا العقد يتعامل معها
column order permutations

.row order permutations

قارن OLD vs NEW:

_dataframe_payload
canonical JSON bytes
canonical_sha256
BYTE IDENTICAL.

أي اختلاف hash على input قانوني = BLOCKER.

PART I — HASH ERROR SEMANTICS

قارن class + message لأول خطأ على:

naive timestamp
nonfinite حيث ممنوع
complex
unsupported object
bad enum/object
invalid index/schema إن ينطبق.

تحسين loop لا يجوز أن يغير first-failure order.

PART J — MANIFEST IDENTITY SECURITY

هذه أيضاً CRITICAL.

ولد manifests صحيحة ثم شوّه:

كل خلية منفردة
صف مفقود
صف زائد
صفوف reordered
column renamed
column reordered
duplicate rows
missing values
wrong scalar type
signed-zero/NaN إذا قانوني
index variations القانونية.

:OLD vs NEW

نفس accept/reject.

وعند الرفض:

نفس exception class + message حرفياً إذا observable.

خصوصاً تحقق أن التحويل من:

iloc[row] لكل خلية

إلى:

(column.tolist)

لم يغير row-major first-error semantics.

PART K — VERIFICATION WEAKENING

لا يكفي old==new على fixtures.

حاول تزوير artifacts بحيث:

- خلية واحدة تتغير.

- row swap.

- column swap.

dtype change - بنفس displayed values.

- index change.

.- DataFrame mutable after hash

تأكد أن النظام الجديد لا يقبل شيئاً كان القديم سيرفضه.

PART L — CAUSALITY / REENTRANCY

لـ3 engines:

full

truncated T

future-mutated suffix

≤T exact.

fresh run twice.

reset/reuse حيث العقد يسمح.

لا state leakage

Hashing/manifest:

repeated calls deterministic.

PART M — MUTATION TESTS

على نسخة خارج الأثر:

1 midrank broken.

2 push-before-rank.

3 max_history eviction broken.

4 signed-zero extrema broken.

5 FVG counter increment broken.

6 FVG first occurrence shifted.

7 FVG same-bar reclaim allowed incorrectly.

8 narrative row position +1.

9 hashing row iteration changed column-major.

10 hashing float canonicalization changed.

11 manifest row/column scan order changed.

12 manifest missing normalization weakened.

حدد أي tests تمسك كل طفرة.

إذا طفرة تؤدي خرقاً ولا يمسكها الاختبار الحالي:

قد تكون فجوة اختبار، وأضفها كتوصية PATCH إذا كانت تمنع شهادة التكافؤ.

PART N — FULL SUITE

شغل الاختبارات المستهدفة.
ثم full suite كاملة من نفس الأثر.

أعط:
collected
passed
failed
.runtime

تحقق لا:
test deleted
skip added
xfail added
assert weakened
.fixture disabled

PART O — PERFORMANCE SANITY ONLY

بعد إثبات الصحة فقط:
Benchmark مستقل:

percentile n=44,640
FVG n=44,640
narrative representative
hashing representative frames
manifest validation representative.

المطلوب فقط إثبات:
speedup حقيقي
ولا memory explosion.

لا تجعل failure في target speed وحده semantic blocker إذا الصحة كاملة؛ سجله performance finding.

PART P — SCOPE

ممنوع اقتراح الآن:
module-scoped fixtures
pytest-xdist
native C/Rust
cache
parallel engine execution
أو optimization جديد.
نريد FREEZE للأثر الحالي.
لا Reality
لا Model
لا Strategy
لا PnL

FINAL VERDICT

ACCEPTED FOR CLOSURE

أو

PATCH REQUIRED

أو

REJECTED — DESIGN REQUIRED

ACCEPTED فقط إذا:

ZERO reproducible semantic differences.

ZERO causal differences.

ZERO hash/identity differences.

ZERO verification weakening.

ZERO unauthorized production changes.

أي اختلاف input قانوني:

PATCH REQUIRED مع:

fixture

steps

expected

actual

.minimal fix surface

إذا ACCEPTED:

حد الشهادة:

PERFORMANCE EQUIVALENCE ONLY.

لا يشهد:

better market definitions

edge

profitability

model

.strategy

READ ONLY.

STOP.

وهلق لا تعطي البيلدر أي تفويض إضافي. لا xdist، ولا fixtures مشتركة، ولا cache. عندنا أثر سريع جداً يدعي تطابقاً كاملاً؛ الأفضل الآن تجميده وضربه من كل الجهات.

إذا هذا التدقيق ينجح، عندها يصبح عندنا إنجاز مهم فعلاً؛ لم نسرّع التقرير فقط؛ سرعنا المحركات نفسها وجهاز التحقق بدون تغيير النتائج. بعدها الإغلاق وإعادة حزم Field Runner وتشغيل مايو عندك مرة واحدة، ونقيس الزمن الحقيقي بدل التقديرات.

قبل ما ابعت شي شوف وين وصلنا

TEST-SUITE ACCELERATION (PATCHED — PENDING INDEPENDENT RE-AUDIT) — 33

الحالة: PATCHED — PENDING_OWNER_RE_AUDIT

البند: EXACT PERFORMANCE V2 — تسريع زمن التيسر نفسه (تفويض المالك الأخير) — «أسرع ما يُمكن بلا فقد دقة واحدة من التيسر».

القاعدة المُتبَّنة: كل تعديل هنا إمّا (أ) test-harness/قراءة بيانات بمخرجات byte-identical (payload/قرار/رسالة خطأ متطابقة حرفياً)، أو (ب) مرفوض. لا اختبار حُذف، لا تحقق قُلص، لا fixture أطفئ، لا assert أضعف.

الخلاصة العددية (42، sandbox ملفاً بعمليات منفصلة، نفس الأداة قبل/بعد)

المقياس قبل بعد التحسين

الإجمالي (42 ملفاً) 119.3-136.2s 282.5s (تفاوت تشغيل) $\approx 2.1-2.4x$

test_trajectory_stage4b2 62.1s 11.8-13.0s $\sim 5x$

test_adaptive_confluence_calibration 48.7s 23.1-26.5s $\sim 2x$

test_research_dataset_builder 34.5s 14.5-15.9s $\sim 2.3x$

test_trajectory_stage3 33.1s 14.7-14.9s $\sim 2.2x$

سجل المحادثات — zzz.txt

test_research_outcome_observer 11.7s 4.7-5.2s ~2.3x

test_research_eligibility 6.8s 3.3-4.0s ~2x

test_evidence_family_reasoning 12.4s 5.2-5.9s ~2.2x

باقي الملفات — بلا تراجع مُلاحظ —

42/42 ملفاً أخضر بعد كل تعديل (إجمالي الاختبارات لم يُحذف منه اختبار واحد).

تقدير جهاز المالك (كان متجهاً إلى +3 ساعات للتيسر الكامل بنصفه 1.5h): ~20-35 دقيقة — ويدخل في هذا أيضاً ربح

الـ patches الإنتاجية (FVG 160x إلخ) إن التست شغل مساراتها.

التشغيل الحقيقي على جهاز المالك فقط — الرقم أعلاه sandbox؛ جهازه يُنبت القياس.

سبب البطء (تشخيص cProfile مُوثق)

البطء ليس في منطق الـ engines — بل في طبقة الـ verification نفسها، ثلاث عقد دقيقة:

research/hashing.py::_dataframe_payload كان يبني الـ canonical payload خلية خلية عبر DataFrame.iat (445k)

استدعاء 350k + scalar استخراج عمود لكل خلية في ملفي اختبار فقط) — تكلفة pandas لكل خلية × خلايا × مرات

إعادة التحقق.

research/manifest_identity.py::_validate_narrative_manifest_identity كان يقرأ الصف كاملاً [row] iloc لكل

خلية (2×870 استخراج صف لكل نداء؛ يُودي به 591 مرة في ملفي اختبار).

تكرار التحقق ذاته آلاف المرات (10-22k عملية canonical_sha256 لكل ملفي اختبار ثقيل) — وهذا التصميم التحقيقي

(كل سجل يُربط ببصمة)؛ لم يُمس.

(3 الإصلاحات (5 قطع — كلها بمخرجات مُطابقة بالبتة)

الملف الإصلاح إثبات الهوية

fast paths + src/trading_system/research/hashing.py _dataframe_payload P4a: استخراج كل عمود مرة واحدة +

numpy float/int/bool (نفس float.hex()/str(int)/bool) بطارية تفاضلية json bytes + payload dicts old==new:

NaN/subnormal/Int64+NA/string+NA/Float64 nullable/object/0.0±) sha256 + متطابقة على إطارات عدائية

مختلط/Enum/Timestamp/Timedelta/فارغة) + رسائل الأخطاء متطابقة (naive timestamp / nonfinite / complex)

research/hashing.py::_scalar P4b ترتيب الفحص (float أولاً — np.float64 فرع من float وhex () مطابق) نفس

البطارية التفاضلية

research/hashing.py::_column_cells fallback series.tolist () بدل. iloc[r] (الـ boxing نفسه؛ NaN/NA ←

نفس missing sentinel) نفس البطارية التفاضلية

research/manifest_identity.py P5a مسح المانيفست: tolist () لكل عمود مرة واحدة (row-major محفوظ = نفس

رسالة الخطأ الأولى) differential old==new: قرار القبول/الرفض ورسالة الخطأ حرفياً على مانيفستات مُطبَّعة + مُشوَّهة

(خلايا مبدلة/صف مفقود/عمود مُعاد تسميته)

manifest_identity.py::_normalized fast paths P5b لنوع str/float/bool/int بالضبط (MISSING←NaN محفوظ)

نفس دفتر القبول/الرفض

المرجعات المُثَبِّتة (بـ sha256 داخل الاختبارات):

tests/_patch_reference_src/hashing_pre.py =

5426f1030fa84242cec0f6fd92f7a1fe03dc7917b6ffcf27db8ba2b613bd534e

tests/_patch_reference_src/manifest_identity_pre.py =

b6ac5a0c3af24f9f0cb9960109228f07771a156415f81c70003b4d31e1f64771

بعد: sha256

research/hashing.py = f2a64c8e3e098f6149c5d01039122b10a8929e44b430cbd7edf40a10317d5c7d

research/manifest_identity.py =

4c88fc44638af9ddd64a6259c86dc3b80fea927097363fe3e87141d35d92414b

tests/test_research_hashing.py =

d9eaf0513c0f58f40cb190a352962ba30058c98818b1a0d88c3df7d53a761e67

بوابات جديدة)

tests/test_research_manifest_identity.py =

15d1a7271c325f38e230bc67de40b0edf3dc3df5bb8e0edf039c97a455485d49

(4 البوابات (كلها خضراء)

tests/test_research_hashing.py — 18/18: منها البطارية التفاضلية (payload/json/sha256 متطابقة على 21 حالة) +

تطابق الأخطاء + mutation proof (قلب ترتيب الصفوف في المسح المُعدَّل → البطارية تكشفه).

tests/test_research_manifest_identity.py — كلها خضراء: differential قرار+رسالة + snapshot مُثَبِّت + رفض

«ليس إطاراً».

tests/test_research_visibility.py + test_research_dataset_integration.py + ملفات الـ engines الثلاثة المُرفَّعة

سجل المحادثات — zzz.txt

(causal_percentile/fvg/narrative) — خضراء (bitwise battery 30,888 كما هي).
المسح الكامل 42 ملفاً بعد كل تعديل: 42/42 — لا اختبار واحد سقط في أي مرحلة (كل تشغيل مُتسلسل أخضر قبل الانتقال للخطوة التالية).

(5) ما لم يُطبَّق عمداً (يحتاج موافقتك الصريحة)
هذه رافعات أسرع لكنها تغيّر طريقة التنفيذ لا الكود — رفضت تطبيقها وحدي تحت شرط «لا ذرّة دقّة»:

تجميع fixtures المشتركة (module-scoped): بعض fixtures تبني نفس المدخلات المحكمة لكل اختبار (~160ms × عشرات المرات). تقليص إعادة البناء يسرّع أكثر لكنه يقلّل عدد مرات إثبات «البناء حتمي عبر عمليات مستقلة». كل asserts تبقى — لكن «التكرار المستقل» نفسه جزء من التحقق لديك.
pytest-xdist (تشغيل متوازي ب-n): نفس الاختبارات بنفس asserts بجدولة متوازية — جدارياً ÷ عدد الأنوية. يتطلب تثبيت pytest-xdist على جهازك + قد يكشف تلوئاً عالمياً بين الاختبارات كأخطاء متقطعة (وهو كشف دقّة لا فقدانها).
تحسين أعمق داخل freeze_creation_feature_snapshot/visibility.project/evidence_families.analyze (منطق تحقيقي صرف) — المتبقي الآن موزّع على pandas internals + حجم التحقق المتكرر؛ كل خصم إضافي سيعني مساساً بالتحقق.

إن وافقت على 1 أو 2: أمر واحد فقط يبقى كما هو — run_btc_may_2026.bat (31 اختباراً، FAST mode).

الممنوعات (ثابتة)

لا تنزيل بديل، لا حجب hash guards، لا allow-hash-mismatch--، لا تعطيل تحقق بصمة، لا اختبار محذوف/مُخَفَّف/مُعْطَل، لا «كريبتو 7×24»، لا Reality/Model/Strategy/PnL/Signals، PROXY≠ACTUAL، لا TIE_ORDER_CONTRACT=NOT_PROVEN دائماً. REAL OWNER RUN على جهازك فقط.

المطالبة (PENDING_OWNER_RE_AUDIT_)

عندك: الملفات الخمسة المُعدّلة + مرجعان مُنَبَّان + اختباراتهما — أعد إنتاج أي رقم في هذا التقرير بالأدوات أعلاه. «تم الإغلاق» قرارك أنت في شات التدقيق، وحده.

gpt-5.5-instant

الحالة الرسمية: تحسين الأداء V2 تم تنفيذه، لكنه PENDING RE-AUDIT وليس معتمداً بعد. الأرقام ممتازة مبدئياً: Tracker أسرع نحو 184× عند حجم الشهر، وFVG نحو 160×، والتشغيل الصناعي الكامل هبط إلى 22 ثانية على جهاز البيلدر، مع ادعاء تطابق النتائج حرفياً. لكن هناك نقطة إجرائية مهمة: MANIFEST يظهر الآن OK 177 و6 stale وهذا طبيعي مؤقتاً لأننا عدّلنا ستة ملفات مغلقة بتفويض PATCH؛ لا نغلق أو نحدّث المانيفست قبل التدقيق.

أنا متفائل جداً بالنتيجة، لكن يوجد شيء أريد الأوديتور يهاجمه بقوة: البيلدر لم يكتفِ بالتحسينات الثلاثة المصرح بها حرفياً؛ داخل FVG عمل أيضاً تحويل أكبر من loop لكل كيان إلى مساح NumPy لكل كيان. قد يكون ممتازاً وصحيحاً، والدifferential tests تقول إنه مطابق، لكن هذه أكثر نقطة عرضة لتغيير ترتيب أحداث دقيق أو حالة lifecycle نادرة. كذلك إعادة تنفيذ تفاصيل np.quantile يدوياً تحتاج هجوماً قوياً جداً لأن اختلاف bit واحد قد يظهر في قيمة غريبة لم تغطيها fixtures.

لا تشغل مايو عندك الآن. إذا الاختبارات والتدقيق قبلت الرقعة، وقتها نجهز ZIP محدثاً وتعيد التشغيل مرة واحدة فقط، والمفروض فرق السرعة يكون هائلاً.

أرسل للأوديتور المستقل هذا النص:

[text]

INDEPENDENT RE-AUDIT — EXACT PERFORMANCE V2

READ ONLY — PATCH EQUIVALENCE + CAUSALITY + PERFORMANCE

الحالة:

PATCHED — PENDING RE-AUDIT

الهدف الوحيد:

إثبات أن Performance V2 أسرع فقط، ولا يغير أي observable semantics أو causality.

لا تعدّل ملفات.

لا تصلح شيئاً.

لا تثق بتقرير البيلدر أو reference tests دون تحقق مستقل.

Production patches المسموحة فقط:

src/trading_system/core/causal_percentile.py

src/trading_system/zones/fvg.py

src/trading_system/decision/narrative.py

سجل المحادثات — zzz.txt

واختباراتها المصريح بها.

PATCH: MANIFEST baseline قبل

183 entries

SHA256:

7796a73fc30fc311902d7ba8e033eb1699a73c5db10e824d02653fbbdd1681587

PATCH: أثناء

177 OK / 6 stale

لأن 3 existing tests + 3 production تغييرت.

لا تعتبر ذلك blocker بحد ذاته.

أي stale آخر = blocker.

المطلوب قرار:

ACCEPTED FOR CLOSURE

أو

PATCH REQUIRED

أو

.REJECTED — DESIGN REQUIRED

1. ARTIFACT/SCOPE GATE

أثبت أن production changes محصورة حرفياً بالملفات الثلاثة.

تحقق أن كل CLOSED أخرى مطابقة.

تحقق أن MANIFEST نفسه لم يتغير.

تحقق أن stale entries هي الستة المصرحة فقط.

تحقق من نسخ pre-patch داخل:

/tests/_patch_reference_src

ولا تثق أنها pre-patch فعلاً:

طابقها مع baseline hashes/history/seals المتاح حيث يمكن.

2. CAUSAL PERCENTILE — INDEPENDENT ORACLE

لا تستخدم implementation الجديد كـ oracle.

أعد بناء oracle بسيط مستقل مطابق للعقد القديم:

scan historical prefix فقط لكل rank.

np.quantile(method="linear") على history القديم لـ percentile_value.

هاجم:

empty

one item

all equal

duplicates

heavy ties

monotonic up/down

alternating

random adversarial

signed +0.0/-0.0 permutations

subnormals

very large/small finite floats

nextafter neighborhoods

q=0

q=1

q=0.5

random q

fractional interpolation cases

NaN skip

NaN raise

bool

+inf/-inf

reset

history_snapshot

.multiple resets

قارن bitwise/float.hex حيث العقد observable.

3. MAX_HISTORY — CRITICAL

التقرير يقول المحركات الحالية لا تستخدمه، لكنه public contract.

هاجم max_history:

1

2

3

7

وrandom small windows.

اختبر:

eviction of min

eviction of max

eviction of duplicate extrema

signed-zero extrema

alternating extremes

.duplicate multiplicities

rank/read-before-push/history_min/history_max/history_snapshot/percentile_value يجب أن يطابق القديم حرفياً.

أي اختلاف = BLOCKER.

4. PERCENTILE_VALUE IMPLEMENTATION

هذه نقطة حرجية.

البيلدر أعاد صياغة numpy linear interpolation.

اختبر عشوائياً آلاف:

histories × q

مقارنة مباشرة مع:

np.quantile(np.array(history, dtype=float64), q, method="linear")

اطلب bitwise where applicable.

ابحث خصوصاً عن:

rounding-order difference

overflow/intermediate behavior

underflow

signed zero

.adjacent representable floats

إذا يوجد input قانوني واحد يعطي observable مختلفاً عن العقد القديم: PATCH REQUIRED

5. FVG — FULL DIFFERENTIAL ATTACK

لا تكتفِ بfixtures البيلدر.

شغل FVG pre-patch OLD وNEW على آلاف OHLC sequences مولدة independent seeded.

تتضمن:

flat bars

gaps
alternating spikes
long trends
dense FVG creation
same-price boundaries
reclaims on same/next/later bars
wick-only interactions
close interactions
multiple simultaneously active entities
very long-lived entities
.end-of-data active entities

قارن:
DataFrame كل
index
column order
dtype
values
NaNs
entity IDs
event order
event positions
availability
lifecycle
all counters
prefix projections إن وجدت.

Exact.

6. FVG EVENT ORDER

التغيير من per-bar/per-entity loop إلى per-entity vector scans قد يغيّر insertion chronology قبل final sort.
أثبت أن final sort key فعلاً total/unique لكل event الممكن.

ابحث عن ties في:
event_position
fvg_id
_o

وحالات event types متعددة لنفس bar/same entity.

إذا ordering observable يمكن أن يختلف:
.BLOCKER

7. FVG SAME-BAR SEMANTICS

هاجم خصوصاً:
creation + interaction في نفس bar
close + reclaim boundary
first occurrence rules
.strict vs non-strict comparisons

تأكد أن vectorized search لا يسمح لحدث أن يُرى قبل availability أو في bar كان القديم يمنعه.

8. FVG FUTURE LEAKAGE

:Full vs truncated
كل $\text{output legally available} \leq T$ يجب أن يطابق.

جُزِب suffixes فيها extreme spikes مصممة لتغيير first occurrence لو scan يقرأ المستقبل بشكل خاطئ.
أي تغيير في prefix = BLOCKER.

9. NARRATIVE DIFFERENTIAL

ولد evidence frames متنوعة ومستقلة:
mixed dtypes
NA
pd.NA
np.nan
booleans
strings/categories القانونية
indexes غير trivial لكن قانونية
حالات feature unavailable.

OLD vs NEW:
hypothesis IDs
creation timing
state
contradiction
supersession
provenance
ordering
all frames exact.

10. NARRATIVE ROW ALIGNMENT

هاجم index labels التي لا تساوي positions:

100,200,300...

DatetimeIndex
sliced frames
non-zero-start legal indexes.

تأكد positional binding لا يخلط label مع iloc position.

11. CAUSALITY

لثلاثة:
full run
truncated at T
full with adversarial suffix
كل $\text{output} \leq T$ exact.

12. REENTRANCY/STATE

كرر نفس: engine/new tracker
fresh run twice
reset
partial run + continuation حيث العقد يسمح
وتأكد لا state leakage.

13. EXISTING TESTS

شغل tests المخصصة ثم full suite.
لا تقبل فقط pass count؛ تحقق من collected count الجديد.

14. MUTATION SENSITIVITY

على نسخة خارج الأثر:
- change midrank tie handling.

- push before rank.
 - signed-zero extrema behavior.
 - break max_history eviction.
 - shift FVG first occurrence by one.
 - allow same-bar reclaim incorrectly.
 - shift narrative positional array by one row.
- تحقق أن differential tests تمسكها.

15. PERFORMANCE SANITY

بعد إثبات التكافؤ فقط:
 benchmark مستقل 44,640 synthetic bars.
 لا يلزم مطابقة رقم البيلدر.
 المطلوب إثبات speedup حقيقي substantial وعدم memory explosion.
 سجل:
 tracker
 FVG
 narrative
 peak memory تقريباً.

16. VERDICT

ACCEPTED فقط إذا:
 ZERO reproducible semantic differences
 ZERO causal differences
 ZERO unauthorized production changes
 أي اختلاف ولو rare input قانوني:
 PATCH REQUIRED مع reproduction كامل.
 إذا ACCEPTED:
 اذكر أن الشهادة هي:
 PERFORMANCE EQUIVALENCE ONLY
 ولا تثبت:
 edge
 profitability
 better detector definitions
 أو أي تغيير اقتصادي.
 READ ONLY.
 STOP.

إذا الأوديتور يقبل، الخطوة بعدها قصيرة: إغلاق رفعة الأداء وتحديث المانيفست، ثم البيلدر يعيد حزم Field Runner مع النسخة السريعة من trading_project. بعدها تشغل نفس الأمر على جهازك.

إذا أرقام البيلدر تتحقق ميدانياً، بدل ما تنتظر 26 دقيقة على FVG وحدها، المفروض تمر مراحل ثقيلة كثيرة في ثوانٍ أو دقائق قليلة. وهذا بالضبط نوع التحسين الذي نريده: نفس العقل، نفس الحقائق، لكن بدون إضاعة وقت في إعادة الحساب.

EXACT PERFORMANCE V2 — PATCHED — PENDING RE-AUDIT — 32

الحالة: STOP — PENDING RE-AUDIT (تسليم مرشح، لا يُعتمد قبل تدقيق مستقل على جهاز المالك).

القاعدة المفروضة: أسرع تسريع ممكن — بصفر فقدان أي ذرة دقة. كل تغيير مُثبت old==new على مستوى البتة.

النطاق (3 ملفات مُصحَّح + اختبارات فقط)

الملف قبل (sha256) بعد (sha256)

src/trading_system/core/causal_percentile.py d0db2281...d88696 1543794f...1f0f8654

src/trading_system/zones/fvg.py c6f46286...e58cd2 1df18e55...31dcd4b

src/trading_system/decision/narrative.py c9f8c751...1dc118e0 407c4d2f...2b0dd24f9

55f2be1b...3e11f228 (قديم) tests/test_causal_percentile.py

سجل المحادثات — zzz.txt

tests/test_fvg.py (قديم) 471702f7...22672b05a
tests/test_narrative.py (قديم) c9c30000...fbb0740

جديد (دعم اختياري فقط): tests/_patch_reference_src/{causal_percentile,fvg,narrative}_pre.py — نسخ حرفية pre-patch، مثبتة بـ sha256 داخل الاختبارات نفسها. لا ملف إنتاجي آخر لُمس.
MANIFEST.sha256: لم يُعدّل (183 سطراً). الفحص الآن: OK / 6 stale 177 — الستة = الملفات الستة أعلاه (مصرّح بها).
لا ملف CLOSED آخر تغيّر.

ماذا تغيّر بالضبط

PATCH-1 – causal_percentile.py (exact online order-statistics)
rank(): بدل المسح 0(n) لكل قيمة → bisect_left/right على multiset مرتّب مُحاظ عليه
حرفياً less/equal/greater = (less + 0.5*equal)/n (حرفياً).
history_min/max: قاعدة strict-compare first-wins (± 0.0 منها) incremental، إعادة حساب دقيقة،
percentile_value(): numpy linear بحرفٍ virtual=(n-1)*q, bounds حسب _get_indexes, _lerp
حرفي للمسار القديم fallback → معاً والطرف المختار صفر 0.0 وعند وجود - t ≥ 0.5. بفرع
np.quantile) np.fromiter + tolerance غير مستقر ولا يمكن تقليده تدريجياً. لا 0.0 لتعادل ± لأن ترتيب (np.quantile).
read-before-push (observe = rank ثم push) max_history FIFO eviction + history_snapshot + reset
رسائل الاستثناءات – كما هي حرفياً.

PATCH-2 – fvg.py

عند الإنشاء. kb_run/ks_run عدّادات تراكمية → kb[i]=sum(genexpr) / ks[i]=len-kb (المُصرّح حرفياً) 2a
لـ numpy مسح → 0(n×entities) فوق كل الكيانات مع كل شمعة for f in fs حلقة: (»الأسرع بدون فقدان دقة») 2b
يبدأ حصراً reclaim، محفوظة strict/non-strict، لكل شرط من الشروط الخمسة first-occurrence (كيان مرة واحدة
sort_values(event_position, fvg_id, _ol) القابلية: الإطار النهائي يُرتّب بـ (وبعد حصراً close بعد
creation بمفاتيح فريدة = ترتيب الإدخال ليس جزءاً من العقد. الإخراج مطابق بالبيئة (مُثبت أدناه)
time/lifecycle/availability/الكيانات/الأحداث/ كما هي تماماً.

PATCH-3 – narrative.py

ونفس iloc لكل صف/سمة. نفس evidence[column] بدل (bound_columns + _value_prebound) ربط الأعمدة مرة واحدة
الأصلي محفوظ كما هو. الـ _value الـ pd.isna. القيم والأنواع و
الأبواب (كلها خضراء – تُفحص بامرك)
البوابة النتيجة

np.quantile(method="linear") 30,888/30,888 bitwise (q: 0/1/0.5/كسرية/nextafter/random, subnormals, magnitudes ± 0.0, duplicates)
استثناءات + float.hex كل حقل بـ tracker: (sha256 منبّت pre-patch مرجع) Differential old-vs-new
float بعموديات uint64 مقارنة + FVG: assert_frame_equal(check_exact, check_dtype) مطابق؛ – class+message
narrative: (no-candidates/bull/bear/alternating/many/realistic/stress 300+600/signed-zeros/empty): إطاراً 11 على

لـ narrative: لكل خلية _value==_value_prebound + كل الأطر السبعة
للثلاثة – T ≤ لا يغيّر mutation future-suffix truncated + Causality prefix(T)
narrative / مكسور FVG increment / مكسور read-before-push / مكسور tie mid-rank (4) Mutation proofs
مكسور – كلها تُسقط الاختبارات (إثبات حساسية الأدلة) row-alignment
Outputs مركّب field-synth بصمة baseline == all-three: 3code027... (240) و 3301670a... (3000)
ملفات: كلها تمر (لم يُغيّر أي سلوك عام) 13 اختبارات الوحدات القائمة
أبداً 10x؛ جهاز المالك ≈ (sandbox) القياسات
Tracker observe (لـ rank+push قيمة):

قديم جديد سرعة n
44,640 51.4s 0.278s 184x
100,000 249.6s 0.963s 259x
250,000 (26min متوقع) 4.84s ~320x
1,000,000 (7h متوقع) 79.4s –
percentile_value (65 استعمال): 44,640 :0.112s → 0.0023s = 48.6x (1m: جديد 3.5s).

FVG analyze (A=قديم كلي، B=PATCH-1 فقط، C=PATCH-1+2):

n A C سرعة
7.0s 0.17s 41x 10,000

20,000 37.0s ~0.4s ~90x

155.8s 0.97s 160x 44,640

44,640 dense 12.1s@10k 0.29s@10k 41x

الشهر الكامل المركّب (44,640 field-synth، كل الفروع + narrative×2):

outputs = بصمة total_after_load = 22.1s (run_engines 14.75s + evidence/narrative 3.87s + seal 3.48s)
....209d131a

narrative وحده: لا تراجع (1.0x على التكوين الخفيف)؛ الأثر الحقيقي للربط يظهر مع مجموعة الميزات الكاملة (كان 271k
lookup ساخناً في الـ profile).

إسقاط جهاز المالك (شهر مايو كامل)

المرحلة قياسك القديم المتوقع الآن

Dynamic Volatility 581s ~20-35s (96.6% rank → 184x)

swings (percentile_value) جزء من الـ 48x ~ 1.5h على الاستدعاءات
 narrative جزء من الـ 1.5h ≥ السابق (بلا تراجع)
 الإجمالي 1.5h + ولم يكتمل ~ 3-8 دقائق (المُرَكَّب 3.5-4.5min ≈ sandbox 22.1s على جهازك + الحفظ/CSV)
 6. ما يُشغَّل على جهاز المالك (خطوة تحقق واحدة)
 text

python -m pytest tests -q
 الاختبارات تحمل المرجع pre-patch داخل tests/_patch_reference_src وتجري التفاضل الكاملة بنفسها — لا حاجة لأي بيانات مالك.)

ثم تشغيل الـ field runner المعتاد — نفس الأمر، بلا أي خيار جديد.

بيانات التدقيق السريع

نسخ pre-patch مثبتة داخل الاختبارات: tests/_patch_reference_src/pre.py (sha256 مُثَبَّت في رأس كل ملف اختبار). سجلات القياس:

bench_chunk1_results.txt
 bench_fvg_final.txt, bench_narrative_final.txt, field_results/all-three.json
 الملفات المضافة للـ 3: tree: نسخ مرجعية فقط (اختبارية). لا شيء خارج tests/.

gpt-5.5-instant

الحالة الرسمية: لم يُعدَّل أي ملف مغلق حتى الآن. الذي حصل فقط هو تشخيص وتصميم وقياس حلول السرعة. تقرير البيلدر الأخير ليس تنفيذ الإصلاح، بل يطلب منك الإذن.

وبناءً على هدفك الواضح — أقصى سرعة من دون التضحية بالدقة — قراري: لا نكتفي بإصلاح واحد. نفوض الثلاثة معاً: محرك الرتب المئوية + Narrative + FVG. لأن إصلاح الأول وحده سيترك FVG يأخذ عشرات الدقائق عندك. أرسل للبيلدر هذا:

[text]

OWNER AUTHORIZATION — EXACT PERFORMANCE V2 — PATCH ONLY

أفوض تنفيذ التحسينات الثلاثة معاً:

1. CausalPercentileTracker
2. FVG accumulator
3. Narrative positional access

الهدف غير قابل للتفاوض:

SAME CAUSAL SEMANTICS
 SAME OBSERVABLE OUTPUTS
 NO LOOKAHEAD
 NO APPROXIMATION
 NO PRECISION REDUCTION
 NO HISTORY TRUNCATION

السرعة وحدها ليست سبباً لقبول أي اختلاف.

الملفات الإنتاجية الوحيدة المسموح تعديلها:

src/trading_system/core/causal_percentile.py
 src/trading_system/zones/fvg.py
 src/trading_system/decision/narrative.py

واختبارات مرتبطة بهذه الرقع فقط.

ممنوع تعديل أي CLOSED آخر.

ممنوع تعديل MANIFEST أثناء PATCH.

ممنوع docs/closure.

ممنوع تغيير detector definitions أو thresholds أو defaults.

سجل المحادثات — zzz.txt

بالنسبة لـ CausalPercentileTracker:
اختر البنية Exact ذات أفضل توازن حقيقي بين:

- سرعة 44,640
- scalability حتى +1m
- الذاكرة
- بساطة التدقيق
- percentile_value
- max_history correctness

لا تختَر P2 لمجرد أنه كان موصى به سابقاً.
Benchmark التقرير يشير أن Treap تتوسع أفضل عند 1m، بينما bisect أفضل عند 44k/250k.
احسم الاختيار بالقياس والتكافؤ، لا بالرأي.

يجب الحفاظ حرفياً على:
read-before-push
mid-rank ties
multiplicity
NaN policies
bool/inf rejection
warm-up
signed-zero first-wins extrema
reset
history_snapshot
max_history
percentile_value(method="linear")
والأخطاء الملاحظة عقدياً.

بالنسبة لـ FVG:
استبدل إعادة sum فوق كل المرشحين كل شمعة بعدادات تراكمية Exact.
ممنوع تغيير أي entity/event/lifecycle/timing/schema/ID.

بالنسبة لـ Narrative:
استبدل positional/prebound repeated pandas cell/label lookup بمسار أسرع.
ممنوع تغيير أي hypothesis أو transition أو timing أو ordering أو provenance.

بوابة القبول:

OLD == NEW

لكل observable output وفق عقد الوحدة.

نفذ differential tests واسعة.
نفذ causality/prefix/future-append tests.
نفذ mutation proofs.
ثم شغل full suite.

Baseline:

MANIFEST = 183/183

SHA256 =

7796a73fc30fc311902d7ba8e033eb1699a73c5db10e824d02653fbdd1681587

يجب أن يبقى كما هو أثناء PATCH.

Benchmark إلزامي بعد الرقع على: synthetic 44,640

- dynamic volatility
- FVG
- narrative
- order flow - المتأثر بالpercentile
- أي محرك رئيسي يستفيد من primitive

وأيضاً benchmark للprimitive حتى 1m إذا عملي.

أريد تقرير:

EXACT PERFORMANCE V2 — PATCHED — PENDING RE-AUDIT

ويحتوي:

- SHA256 قبل/بعد للملفات المعدلة.

- عدد differential tests.

- نتائج التكافؤ.

- causality gate.

- mutation proofs.

- full suite count.

- benchmark OLD vs NEW.

- speedup - الحقيقي لكل وحدة.

- تقدير Field Run كامل على جهاز المالك.

- peak memory.

- أي اختلاف ولو حالة واحدة.

- MANIFEST unchanged.

- كل CLOSED الأخرى unchanged.

إذا ظهر اختلاف غير مشروع في أي output:

لا تخفه ولا تستخدم tolerance.

STOP لذلك الجزء ولا تشحنه.

لا Native.

لا cache.

لا parallelism بعد.

لا Reality analysis.

PATCH فقط ثم STOP.

إذا نجحت هذه الجولة، لا تشغل مايو فوراً. نعطي الأوديتور إعادة تدقيق واحدة مركزة على أن السرعة لم تغيّر النتائج. بعدها تأخذ

ZIP محدث وتشغل مايو.

الهدف الذي أريده قبل أن ترجع لجهازك: FVG الذي أخذ عندك حوالي 26 دقيقة يجب أن ينهار زمنه جذرياً، والتقلب الذي أخذ

قراءة 10 دقائق يجب أن يصبح أسرع بكثير. وإذا بقي التشغيل الكامل بطيئاً بعد ذلك، عندها نضيف cache والتوازي؛ لكن فقط

بعد أن تصبح النواة نفسها سريعة.

ببساطة — هذي خلاصة شو صار من ساعة:

١. شغلّت تحقيق أداء كامل (بلا بياناتك — على fixtures تركيبية بنفس حجم الشهر):

٢. طلع السبب الدقيق للبطء — حاجتين:

كل شمعة يسمح التاريخ كله من جديد = مليارات التكرارات ⇒ هذا كان: (percentile يحسب) القديم tracker الـ Dynamic Volatility % من زمن 96.8.

500 مشكلة مختلفة أصلاً — في سطر داخل المحرك يعيد جمع قائمة كل المرشحين من الصفر كل شمعة ⇒ بمعدل FVG:

ثانية تبعك. 1594 مليون عملية بالشهر ⇒ هذا هو الـ

:(مطابقة تماماً bitwise) بدائل دقيقة 4. صممت وبنيت ٣

كلها مختبرة: 36/36 تكافؤ حرفي + 15/15 سببية (لا مستقبل).

النتيجة: نفس النتائج بالضبط — لكن 90 إلى 134 ضعف أسرع.

٣. الإسقاط على جهازك:

الشهر

الآن 45-75 دقيقة (غير عملي)

بعد إصلاح tracker فقط 25-35 دقيقة

إسطرة 6-10 FVG دقائق

إسطرة 3-5 narrative دقائق

٤. وين وصلنا قراراً:

التقرير الكامل جاهز (EXACT_PERFORMANCE_REVOLUTION_DESIGN_BENCHMARK_REPORT.md_31) — كما

طلبت: تصميم وقياس فقط.

ما لمست CLOSED ولا production — الانتظار على قرارك.
قرارك المطلوب الآن: تفويض patch الـ primitive (ملف واحد داخلي — موصى به وبقوة) + قرار منفصل على patch الـ FVG (هو الأكبر أثراً عندك).
طبقة (C/Rust) native: أنصح لا — 30% زيادة فقط مقابل أعباء بناء على Windows.
قلّي: نَقِّذ patch، ولا بذلك شي بالتقرير أول؟

1m 9s | 5 minutes ago

4:27 AM

Thought for 2 seconds

again. The previous turn completed the report and — (what happened) "شو صار" The user asks presented it. Maybe they're asking for a status update, or the message is a follow-up wanting a simpler summary in Arabic/Levantine dialect.

Wait — looking at the conversation, my last message already delivered the report. The user says again. Perhaps they want a simpler, more conversational explanation of the whole situation — "صار" what happened, what I found, what's next. Or maybe they didn't see the previous response fully.

Let me give a clear, concise summary in Levantine-friendly Arabic of:

What was done (the design+benchmark investigation)

What was found (why it's slow)

What the fix plan is

What's waiting for their decision

No need for more tool calls — the work is done and reported. Keep it short and human. The previous message was quite dense with technical tables; the user may want the "story" version.

ببساطة — هذي خلاصة شو صار من ساعة:

١. شغلّت تحقيق أداء كامل (بلا بياناتك — على fixtures تركيبيّة بنفس حجم الشهر):

٢. طلع السبب الدقيق للبطء — حاجتين:

كل شمعة يسمح التاريخ كله من جديد = مليارات التكرارات ⇒ هذا كان: (percentile يحسب) القديم tracker الـ Dynamic Volatility % من زمن 96.8
500 مشكلة مختلفة أصلاً — في سطر داخل المحرك يعيد جمع قائمة كل المرشحين من الصفر كل شمعة ⇒ بمعدل FVG الـ
ثانية تبعك. 1594 مليون عملية بالشهر ⇒ هذا هو الـ
(مطابقة تماماً bitwise) بدائل دقيقة 4. صمّمت وبنيت ٣:

كلها مختبرة: 36/36 تكافؤ حرفي + 15/15 سببية (لا مستقبل).
النتيجة: نفس النتائج بالضبط — لكن 90 إلى 134 ضعف أسرع.
٣. الإسقاط على جهازك:

الشهر

الآن 45-75 دقيقة (غير عملي)

بعد إصلاح tracker فقط 25-35 دقيقة

إسطرة FVG 6-10 دقائق

إسطرة 3-5 narrative دقائق

٤. وين وصلنا قراراً:

التقرير الكامل جاهز (EXACT_PERFORMANCE_REVOLUTION_DESIGN_BENCHMARK_REPORT.md_31) — كما

طلبت: تصميم وقياس فقط.

ما لمست CLOSED ولا production — الانتظار على قرارك.

قرارك المطلوب الآن: تفويض patch الـ primitive (ملف واحد داخلي — موصى به وبقوة) + قرار منفصل على patch الـ FVG (هو الأكبر أثراً عندك).

طبقة (C/Rust) native: أنصح لا — 30% زيادة فقط مقابل أعباء بناء على Windows.
قلّي: نَقِّذ patch، ولا بذلك شي بالتقرير أول؟

Thought for 23 seconds

for the third time. This is odd. My previous two responses (what happened) "شو صار" The user is asking summarized the work. Maybe

The user isn't seeing my responses (UI issue)

The user wants something else entirely³. The user is testing⁴. The responses aren't reaching them
Looking at the conversation flow — the last two turns I responded with summaries and no tool calls in the second-to-last... wait no, the last turn I gave a plain text summary with no tool calls. The turn before that I presented the report file.

Maybe the issue is the user expects a different kind of deliverable, or the messages seem repetitive to them. Or maybe they're asking "what happened" because they expected the work to continue (like actually being further along).

Or — perhaps the user wants me to keep going with something? But the task explicitly said STOP after the report. And there's a pending decision (patch authorization).

three times in a row suggests maybe my messages aren't displaying "شو صار": Actually, re-reading properly for them, or they want a very short answer. Let me give the shortest possible answer — status in one or two lines, and ask what they want next. Don't repeat the whole summary again. Keep it .minimal

Maybe also consider using ask_user to clarify what they need? The ambiguity is real: they keep asking "what happened" after I've answered twice. Options:

They didn't see my answers (technical issue)

They want a different deliverable

They want me to proceed with the patch

They want the file again

Given repeated identical questions after full answers, the most helpful move is a very short status + ask what they need. I could use the ask_user tool. But the instruction says use it when ambiguity is ambiguous "شو صار" impedes completion — the task IS complete (report delivered). The question .now

Let me answer ultra-briefly and ask what they want. Maybe use ask_user with options: see the report file / proceed with patch authorization / something else.

Actually — a simpler interpretation: maybe each time the conversation gets compacted (the arena system), and the user is in a UI where they don't see long responses? The system memory says the conversation gets condensed. The user might be seeing nothing or stale content.

I'll give a 3-line answer + present the report file again (so it's in front of them) + ask what they need. That covers the "they can't see anything" case.

الوضع باختصار شديد:

التقرير خلص وانسلّم:

EXACT_PERFORMANCE_REVOLUTION_DESIGN_BENCHMARK_REPORT.md_31

— تصميم + قياسات فقط (كما طلبت)، 15 قسماً.

النتيجة: البطء = tracker قديم FVG + $O(n^2)$ تجمع قوائم من الصفر كل شمعة. البدائل جاهزة: 134x-90 أسرع بنفس النتائج حرفياً (bitwise).

الانتظار عليك: تفويض تنفيذ patch (ملف واحد) + قرار منفصل على FVG. لا CLOSED لُمس. أعيد فتح الملف أمامك:

Thought for 1 second

suggests they may want something "شو صار" Now ask the user what they need — the repeated .specific. Use ask_user to clarify

شو بدك نعمل هلق؟ (التقرير جاهز والقرار إلك)

Skip

فويض تنفيذ patch

نقّذ استبدال الـ primitive داخل core/causal_percentile.py بتفويضك — مع كامل بطاقات التكافؤ + 1045 اختبار + rerun ميداني

بدي أقرأ التقرير أول
أشرحلك أي قسم بالتفصيل أو أبسط الأرقام قبل أي قرار

فويض الFVG + patch معاً
الprimitive + مُجمّع FVG (الأكبر أثراً على وقتك) كلاهما بتفويض واحد

في شي ثاني
اكتب شو بدك بالضبط

gpt-5.5-instant

الحالة الرسمية: التشخيص اكتمل بدون لمس الإنتاج. عرفنا الآن بالأرقام لماذا المشروع بطيء، والأهم أن هناك طريقاً لتسريعه جذرياً دون حذف بيانات أو تقليل التاريخ أو تقريب الحساب. لم يُطبق أي Patch على الوحدات المغلقة حتى الآن. النتيجة التي تهكم: نعم، يمكننا استهداف «نفس الدقة + سرعة أعلى بكثير». لكنني لا أريد تنفيذ اقتراح البيلدر حرفياً كما هو قبل تصحيح نقطة تقنية مهمة.

هو رشح P2 numpy-buffer للpercentile. لكن Benchmark نفسه يقول شيئاً مثيراً:

[text]

عند 44,640:

P1 = 0.562 ثانية

P2 = 0.646 ثانية

وعند مليون:

[text]

P1 = 87.9 ثانية

P2 = 126.1 ثانية

P3 Treap = 28.1 ثانية

يعني P2 ليس أفضل اختيار للتوسع الطويل، رغم ذاكرته الممتازة.

الأهم أن السبب هو أن إدخال عنصر وسط مصفوفة مرتبة يتطلب تحريك عناصر؛ ما زلنا ندفع كلفة تزداد مع الحجم. على ملايين القيم، Treap ذات order statistics أقرب لما نريده خوارزمية: إدخال ورتبة لوغاريتميان.

لكن percentile_value () على Treap بطيئة لأن البيلدر يبدو أنه يحول التاريخ أو يعيد بناء شيء لكي يشغل NumPy. وهنا توجد فرصة أفضل: شجرة Order Statistics تستطيع أيضاً إيجاد العنصر ذي الرتبة k مباشرة. و np.quantile(method="linear") يمكن حسابه Exact من عنصرين مرتبين فقط.

إذا كان:

[text]

```
h = (N - 1) * q
```

فحتاج:

[text]

```
x[floor(h)]  
x[ceil(h)]
```

ثم نفس interpolation الذي يفرضه عقد NumPy.

هذا قد يلغي الحاجة لترتيب/نسخ التاريخ بالكامل في percentile_value.

لكن هنا شرط شديد: يجب إثبات التطابق العددي مع NumPy الحالي، خصوصاً كيفية تنفيذ interpolation floating-point. لا نفترض أن الصيغة الرياضية المتساوية تعطي نفس bits بسبب ترتيب العمليات.

لذلك أريد أقصى سرعة، لكن على مرحلتين:

أولاً نأخذ المكاسب الضخمة المضمونة تقريباً:

percentile exact online.

FVG accumulator.

.Narrative access

ثانياً cache + parallel runner.

وبدلاً من تنفيذ كل واحدة ثم إعادة تشغيل ساعتين، أريد Patch Performance Release واحدة منظمة، لكن كل تغيير فيها قابل للتعطيل والمقارنة منفرداً.

هدفنا الواقعي بعد هذه الجولة:

[text]

الشهر:

من ~ ساعة

إلى ~3-5 دقائق أول تشغيل

ثم مع cache:

إعادة التشغيل على نفس المدخلات

قد تصبح أسرع بكثير.

والسنة تقريباً عشرات الدقائق بدلاً من أيام، ثم نستمر بتحسينها عندما نحتاج.

أرسل للبيلدر النص التالي:

[text]

OWNER AUTHORIZATION — EXACT PERFORMANCE V2 — PATCH ONLY

الهدف:

أقصى تسريع عملي الآن مع:

ZERO INTENTIONAL SEMANTIC CHANGE

ZERO LOOKAHEAD

ZERO PRECISION REDUCTION

ZERO HISTORY TRUNCATION

ممنوع:

approximation

sampling

rolling replacement

future-fit

threshold changes

detector changes

تغيير outputs لتسهيل optimization.

الوحدات CLOSED المسموح PATCH لها حصراً:

1.
src/trading_system/core/causal_percentile.py

2.
src/trading_system/zones/fvg.py

3.
src/trading_system/decision/narrative.py

واختبارات هذه الوحدات/اختبارات أداء وتكافؤ مرتبطة بها فقط.

لا ملف إنتاج آخر.

PATCH-1 — CAUSAL PERCENTILE

=====

لا تختَر P2 تلقائياً.

نَعُدّ production candidate مبني على exact online order-statistics structure.

قارن قبل الاختيار النهائي بين:

A) bisect-list

B) augmented Treap/order-statistics tree

C) hybrid:

rank + sorted structure

history_snapshot/max_history فقط لما يحتاج FIFO/history

الهدف ليس 44k Benchmark فقط.

الهدف scalability حتى +1m.

شرط أساسي:

rank exact semantics القديمة حرفياً:

read-before-push

mid-rank ties

multiplicity

NaN policies

inf/bool rejection

empty warmup

signed zero بما فيها strict first-wins extrema

reset

history_snapshot

max_history behavior

.exception classes/messages where contractually observable

:percentile_value(q)

لا تقبل تحويل approximate.

تحقق من إمكانية exact order-statistic selection:

```
h=(N-1)q
```

```
j=floor(h)
```

```
k=ceil(h)
```

ثم `select(j)`, `select(k)`.

لكن لا تعتمد interpolation يدوي في production قبل إثبات أنه يطابق:

```
np.quantile(..., method="linear")
```

على بطارية ضخمة bitwise.

إذا ترتيب floating operations يجعل bitwise مختلفاً:

استخدم مسار يحافظ على implementation contract القديم،

أو أثبت أن CLOSED contract لا يفرض bitwise لهذه النقطة.

ممنوع tolerance جديد.

اختبر q:

0

1

0.5

random deterministic q

values producing fractional h difficult numerically

subnormals

large magnitudes

signed zero

.duplicates

:max_history

ليس مسموحاً تركه semantic-risk.

اختبر rolling eviction حرفياً أيضاً، حتى لو المحركات الحالية لا تستخدمه.

PATCH-2 — FVG

```
=====
```

أزل $O(n^2)$ الخاص بـ:

sum(direction.startswith("BULLISH") for x in fs)
والمقابل bearish

باستخدام exact cumulative counters/state

لا تغير:

وقت إنشاء FVG

schema

entity IDs

events

lifecycle

availability

أي output.

OLD FVG OUTPUT

==

NEW FVG OUTPUT

على كل عمود وكل event/entity observable.

اختبر:

no candidates

all bullish

all bearish

alternating

many candidates

future append

prefix truncation

.realistic seeded OHLC

PATCH-3 — NARRATIVE

=====

استبدال hot loop في repeated pandas label/cell lookups
بـ prebound positional arrays/views مرة واحدة فقط.

لا تغير:

hypothesis identity

state transitions

creation/contradiction/supersession timing

evidence references

ordering

schema

.provenance

OLD NARRATIVE

==

NEW NARRATIVE

حرفياً وفق العقد.

PERFORMANCE HARNESS

=====

شغل new و old/reference على نفس fixture قبل حذف/استبدال المرجع الاختباري.

مقاسات:

1k

5k

10k

20k
44,640
100k
250k

و1m للprimitive إذا عملي.
أعط timing لكل patch منفرد:

baseline
percentile only
FVG only
narrative only
all three.

EQUIVALENCE GATE

=====

لا يكفي 1045 tests.

أضف differential tests old-vs-new.

لكل output observable:
.exact equality

DataFrames:
assert_frame_equal بأشد contract متاح؛
.check dtype/order/index/names

Events/entities:
exact row/order/content.

Percentile:
float.hex حيث ينطبق.

Exceptions:
class + contractually observable semantics.

CAUSALITY GATE

=====

لكل patch:

full run من prefix(T)

==

run على T truncated input
وإضافة future suffix مختلف
لا تغير $T \geq$.

MUTATION PROOF

=====

اكسر عمداً خارج الأثر:

tie mid-rank
read-before-push
FVG cumulative increment
narrative row alignment

ويجب differential tests أن تفشل.

FIELD-RUNNER PERFORMANCE

=====

لا تستخدم owner data.

استخدم synthetic 44,640 bars.

شغل pipeline relevant stages وقس:

volatility

FVG

order flow proxy/actual

narrative

والزمن الإجمالي المتأثر.

لا تدع benchmark نفسه يغير semantics.

MANIFEST

=====

ممنوع تعديل MANIFEST أثناء PATCH.

Baseline الحالي:

183/183

SHA256:

7796a73fc30fc311902d7ba8e033eb1699a73c5db10e824d02653fbbdd1681587

لا تعديل docs/closure.

عند التسليم:

PATCHED — PENDING RE-AUDIT

أعد:

- الملفات المعدلة.
- SHA256 قبل/بعد.
- differential test counts.
- full suite count.
- benchmark - قبل/بعد.
- speedup لكل وحدة.
- projected owner month runtime.
- memory impact.
- causality gate.
- mutation proofs.
- أي اختلاف ولو خلية واحدة.
- MANIFEST unchanged.
- CLOSED - الأخرى unchanged.

إذا أي output يختلف بلا أساس عقدي:

STOP — DO NOT SHIP THAT PATCH

لا cache.

لا parallelism.

لا native في هذه الجولة.

نريد أولاً جعل المحركات نفسها سريعة Exact.

STOP.

لماذا لا أضيف cache والتوازي الآن؟ لأنهما سيخفيان لنا الأداء الحقيقي للمحركات الجديدة. أولاً نجعل الخوارزمية نفسها سريعة. بعدها cache والتوازي يصبحان طبقة إضافية، وليس عكازاً.

وفي نقطة أخيرة: إذا Treap تبين أسرع عند مليون لكن أبطأ عند 44 ألف، يمكن استخدام Hybrid مدروس حسب حالة البنية، لكن لا أريد threshold مثل «إذا $n > 100k$ غير algorithm» لأنه لا يمكن لمحرك Online معرفة الحجم المستقبلي بصورة سببية ولا نحتاج هذا التعقيد. الأفضل اختيار بنية واحدة ذات scaling جيد، أو بنية adaptive تعتمد فقط على الحالة الماضية إذا ثبتت دلاليًا.

هَدَفِي من الجولة القادمة ليس «تحسين بسيط». إذا الأرقام صحيحة، لازم نشوف تغييراً واضحاً جداً في التشغيل قبل أن ترجع تشغل مايو على جهازك.

EXACT PERFORMANCE REVOLUTION — DESIGN + BENCHMARK REPORT

Date: 2026-09-29 (Asia/Damascus)

Report ID: EXACT_PERF_REVOLUTION_V1

Scope: DESIGN + BENCHMARK INVESTIGATION ONLY — لا تعديل على CLOSED، لا production optimization،

Reality analysis. لا

Synthetic workloads فقط (لا owner data) عند 1k, 5k, 10k, 20k, 44,640 n = 100k, 250k, 1m+

للـ (prototypes).

أرقام المالك المرجعية (أول Real Owner Run): FVG = 1594.3s, Dynamic Volatility = 581.4s على 44,640 دقيقة.

1 — flame / profile hotspots (cProfile + عدّ استدعاءات دقيق)

A) Dynamic Volatility — n=10,000 (5.6s profiled)

ncalls cumtime node

1 5.639s dynamic_volatility.analyze

20,000 5.527s causal_percentile.observe

96.6% ← causal_percentile.rank 5.445s 20,000 من الزمن (المسح الخطي للتاريخ)

20,000 0.065s causal_percentile.push

40,000 0.062s _coerce_value

B) FVG — n=10,000 (61.4s profiled) — ليست tracker-bound:

ncalls cumtime node

1 61.4s tottime 36.6s fvg.analyze (الحلقة السطرية)

10,027 16.3s builtins.sum

25,005,000 12.1s fvg.py:72 <genexpr> — kb[i]=sum(x.direction.startswith("BULLISH") for x in fs) يُعاد

حسابه فوق كل المرشحين كل شمعة = $O(n^2)$ عمل متكرر

4,999 2.0s causal_percentile.rank (3% فقط)

C) Narrative 6.1B — n=6,000 (15.8s / 41.3s profiled / mode):

ncalls cumtime node

372,997 20.0s pandas.DataFrame.getitem (وصول بالاسم لكل خلية)

271,647 9.6s narrative._value (لوك-أب feature لكل hypothesis)

34,255 8.2s مقارنات eq + take Series

39.9M — إجمالي الاستدعاءات — ثابت pandas ضخّم

D) Swing detector — n=10,000 (عدّ مباشر): percentile_value=7,383 / push=2,829 / rank=8,040 استدعاء

np.quantile كامل لكل مرة) + tracker.

2 — complexity map

primitive complexity لكل عملية إجمالي على n يستخدمه @44,640 allocations peak memory

$O(n)$ CausalPercentileTracker.rank مسح $O(n^2)$ Python لكل tracker volatility (2n استدعاء)،

fvg, order_blocks, volume_delta (عند الأحداث) $(2 \times 2n)$, absorption $(2 \times 3n)$, liquidity, structural_breaks

swings يهمل 1,404 KiB

$O(n)$ CausalPercentileTracker.percentile_value نسخ $O(n \log n)$ + np.fromiter فرز np.quantile لكل استدعاء

$O(n^2 \log n)$ في swings (7.4k استدعاء/10k بار) EmpiricalRuntime swing فقط sort + fromiter لكل نداء —

$O(n)$ kb/ks counters FVG لكل شمعة فوق كل المرشحين fvg.analyze $O(n^2)$ داخلياً genexpr لكل شمعة —

$O(1)$ Narrative _value + label access بثابت pandas ضخّم لكل خلية $O(\text{bars} \times \text{hypotheses} \times \text{features})$

~373k DataFrame.getitem / 6k narrative.analyze بار —

$O(n)$ canonical_sha256 timeline seal واحدة runner $O(n)$ (seal + تحقّق bind_interval) JSON/hash 1.3s

@22.3k

الاستدعاءات لكل شهر (44,640): observe = 89,280 ؛ volume_delta $\times 2 \approx$ ؛ absorption $\times 2 \approx$

267,840 (عند الأهلية) ؛ percentile_value $\sim 33k$ + rank $\sim 36k$ ؛ swings = 500M genexpr ؛ rank $\sim 11k$ FVG =

(الوحش الحقيقي).

القديم (مستخرجة من الكود) primitive الدلالات الرياضية الحرفية لل- 3

الحالي يُحسب على التاريخ قبل إدراج percentile (push(value) - rank(value) ثم read-before-push: observe =

نفسه (لا self-inclusion).

بمقارنة less/equal حجم التاريخ قبل الإدراج: $N_{\text{حيث}} = N / (\text{less} + 0.5 \cdot \text{equal})$ ties (mid-rank): percentile =

سجل الملاحظات — zzz.txt

IEEE < و==.
 duplicates: بعدها multiplicity (deque)! equal_count مع التاريخ مع.
 NaN: nan_policy="skip" → rank يرجع accepted=False, percentile=NaN, min/max=NaN ولا يُدرج (total_skipped):
 "raise" → CausalPercentileDataError.
 inf / bool: (CausalPercentileDataError) – «Fix the upstream feature calculation».
 warm-up: percentile=NaN, sample_count=0, accepted=True – bootstrap magic value.
 history: expanding (deque) اختياري max_history = rolling FIFO (deque maxlen) افتراضياً؛ (على كل البناءات grep تحقق بال) تمرره إطلاقاً.
 percentile_value(q): np.quantile(np.fromiter(history, float64), q, method="linear") – الملتزم به عقداً هو.
 تاريخ فارغ ConfigError. [و1,1] في [q فقط لا على الترتيب (multiset) الخطي؛ يعتمد على الحزمة numpy مُقدّر
 → NaN.
 extrema (history_min/max): مثلاً أول قيمة 0.0 صارمة ⇒ عند التعادل (±) if v < min if v > max مسح خطي بـ
 6ها (سلوك حرفي مُكتشف بالاختبار – انظر sign-bit\$ مُدرجة تفوز وتُبقى
 reset: لا يوجد؛ serialization: tuple نسخة history_snapshot: يسمح التاريخ والعدادات؛
 reentrancy: (مطابق bitwise الاختبارات: تكرار التشغيل) بلا قفل – استدعاء واحد في كل وقت stateful كائن
 (لا مستقبل) Exact Online – الخوارزميات المرشحة 4
 المرشح الفكرة الحالة
 قاعدة extrema + (C بلغة memmove) + insert للعدّات bisect_left/right + قائمة مرتبة bisect-list P1
 الأول-يفوز مُنفذ ومُختبر
 P2 numpy-buffer float64 مصفوفة memmove إدراج بـ np.searchsorted + مسبقّة التخصيص
 – الموصى به
 P3 treap treap (الأفضل عند) مُنفذ ومُختبر (rank = 0(log n) أحجام أشجار فرعية + multiplicities بـ
 P4 Fenwick/segment mapping يتطلب الإحداثيات العالمية للإحداثيات رسمياً؛ الضغط العالمي للإحداثيات mapping
 لا تحفظ ترتيب القيم؛ أي بديل قانوني first-seen؛ إحداثيات causal information contract يغيّر ⇒ future-fit
 P3 = يؤول إلى شجرة
 P5 native C buffer ctypes عبر C في binary search + memmove + مرتبّ semantics (متوفر هنا gcc) ومُختبر
 E) ؛ لا حاجة P1/P3 مغطاة كنطاق بواسطة counting structures ، مفهرس skip-list بدائل أخرى
 . بلا قراءة/ملاءمة مستقبل 0(n) أو 0(n log n) إلى 0(n²) من exact expanding rank :الهدف المُتحقق

benchmark old vs prototypes (نظيف بلا tracemalloc) — 5
 :observe() stream

P1 bisect P2 numpy P3 treap P5 native C القديم P0 n

```
1,000 0.029s 0.004s 0.010s 0.011s 0.008s
5,000 0.689s 0.021s 0.053s 0.077s 0.048s
10,000 2.833s 0.047s 0.110s 0.130s 0.087s
20,000 11.337s 0.104s 0.238s 0.322s 0.232s
44,640 58.343s 0.562s (104x) 0.646s (90x) 0.726s (80x) 0.434s (134x)
```

2.116s 5.284s 5.974s 4.380s (مُقدّر min 4.8 ≤) 100,000
 7.620s 16.864s 16.270s 13.667s (مُقدّر min 27 ≤) 250,000

```
1,000,000 (أيام) 87.9s 126.1s 28.1s 79.7s
```

تأكيد التربيع للقديم: $n \times 2 \Rightarrow$ زمن 4.0×5.1 (التنبؤ التربيعي 4.0×5.0) عند كل قفزات القياس — $O(n^2)$ مؤكّد.

percentile_value(0.5) بعد n إدراج:

P1 P2 P3 P5 القديم P0 n
 10,000 0.48ms 0.46ms 0.16ms 3.23ms 0.18ms

```
44,640 1.92ms 2.13ms 0.47ms (4x) 50.4ms 0.58ms
```

P0=1,404K / P1=1,816K / P2=769K / P3=6,069K / P5=~1K (C : (tracemalloc peak @44,640) الذاكرة
 .heap)

primitive ≥ 20x ⇒ 80-134x مُتحقق (لم تُستخدم كادعاء قبل القياس – والقياس يتجاوزها)
 tracker-dominated ≥ 10x ⇒ volatility = 96.6% rank ⇒ ~50x مُقدّر للمحرك كله.

6 — نتائج EQUIVALENCE GATE

bitwise (float.hex) — 36/36 PASS على كل حقول percentile_value + PercentileObservation + أصناف
 الاستثناءات.

البطارية (كل حالة × (nan_policy=skip/raise): empty / one_row / duplicates_mixed / all_equal /
 monotonic_increasing / monotonic_decreasing / alternating / random_seeded (500) /
 extreme_magnitudes (1e308, 5e-324, subnormals) / exact_ties_heavy / signed_zero_mix / NaN-skip
 stream / error cases (bool, inf, NaN-raise) — لكل من P1/P2/P3/P5 مقابل P0.

اكتشاف حرفي (لم يُغطّ بالنظرية): انحراف sign-bit في history_min/max ±0.0 عند التعادل — الأصل يبقى أول قيمة
 مُدرجة (if v < min صارم). أغلق بمحاكاة نفس القاعدة حرفياً (strict-compare, الأول يفوز) داخل prototypes ⇒ بعد

الإصلاح bitwise كامل حتى sign-bit الأصفار — بلا أي tolerance مُختَلَق. معيار التكافؤ مأخوذ من العقد نفسه: العدّات أعداد صحيحة، النسبة $n/(less+0.5 \cdot equal)$ حرفياً، وpercentile_value = نفس دالة "numpy method=" على نفس الحزمة (العقد ي؟ المُقدّر لا تمثيلاً معيّناً — والتماثيل bitwise مُثبّنة تجريبياً لكل الحالات).

7 — نتائج CAUSALITY GATE
PASS 15/15 لكل من P0/P1/P2/P3/P5

$T \leq$ outputs تتغير صفر عند إضافة future suffix مختلف تماماً ($\times 2$ suffix مقارنة).
تكرار التشغيل: bitwise مطابق.

prefix truncation: مطابق تماماً.

البنى Online كلها تُبنى من البادئة فقط (لا قراءة/fit للمستقبل في أي مرشح — مُراجع بنويّاً: لا إحصاءات/إحداثيات/تطبيع على مستوى الكيان الكامل).

8 — خطة DAG + content-addressed cache (خارج CLOSED)
DAG (عقد التنفيذ):

[text]

```
sources(load+cross_witness) → canonical_bundle + executed_flow → timeline_seal
| volatility | session | fvg | flow_proxy | flow_actual | htf (مستقلة)
| absorption_proxy ← flow_proxy | absorption_actual ← flow_actual
| swings → sequence → {structural_breaks → order_blocks, liquidity, dealing_range}
| evidence_actual ← (flow_actual+absorption_actual+المشتركة+السطح)
| evidence_proxy ← (flow_proxy+absorption_proxy+المشتركة+السطح)
| narrative_actual ← evidence_actual | narrative_proxy ← evidence_proxy
```

—L reports(A-F + chart + preview)

كل عقدة تُنفَّذ مرة واحدة؛ لا إعادة تشغيل engine causal لنفس: input hash + engine identity/version + policy .hash

Cache (مجلد خارجي /field_run_cache):

cache_key = SHA256(input artifact hashes + engine module path + engine module file sha256 + .policy hash (quantile/priors/sessions/htf) + contract version strings + column schema)

القيمة: frames محفوظة (feather/parquet) + hashاتها الداخلية.
أي mismatch ⇒ cache miss تام.

الـcache لا يتجاوز verify/integrity أبداً؛ تحقّق المصادر (adapter verify + cross-witness + بصمات) يُعاد دائماً؛
الـcache يُغفي فقط إعادة analyze() المُكلفة على مدخلات مطابقة bit-for-bit.

9 — خطة التوازي

العقدة process-safe؟ ملاحظة

volatility/session/fvg/flows/absorption/htf/swings-chain نعم (دالة نقية على frame + حالة داخلية جديدة) pool
عبر processes (GIL يمنع threads)

narrative_actual / narrative_proxy نعم لبعضهما يتوازيان

$\times 2$ evidence نعم

داخل المحرك (الحلقة per-bar) لا حالة تتابعية سببية — لا تُقسّم
reporting نعم

بلا ادعاءات threads: الأرقام أعلاه single-thread مُقاسة. الربح المتوقع من التوازي وحده: 2.5x-1.5 على الحائط (FVG وحده يمثل النصف) — التوازي ليس الحل؛ استبدال الـprimitive + إصلاح FVG هما الحلان.

10 — توصية NO Native Acceleration (في هذه المرحلة)

pure Python. P5 native = 134x مقابل P1 = 104x و P2 = 90x على الشهر ⇒ الزيادة ~30% فقط فوق pure Python.
التكلفة: أداة بناء (MSVC/mingw على Windows)، wheels لكل منصة، fallback path، فحص determinism إضافي، صعوبة audit.

numpy أصلاً dependency مُلزم للمشروع ⇒ P2 بلا أي اعتماد جديد.

الاستثناء: إذا أصبحت تدفقات السنة (525,600+) روتينية، يُعاد النظر (أو يُستخدم 28s pure-Python: P3 treap عند 1m — الأفضل توسيعاً).

11 — إسقاطات زمن التشغيل (شهر/سنة)

على آلة المالك (القياس: vol=581s, fvg=1594s؛ نسبة $\approx 10x-9$ آلة البناء):

السيناريو الشهر (44,640) السنة (~525,600)

الحالي (قبل أي patch) 45-75 دقيقة (غير عملي) أيام ($O(n^2)$)

(FVG-bound) أيام 4-2 ~ (تبقى ~1500s FVG-bound؛ دقيقة 25-35 (P2/P1) فقط primitive استبدال
سجل المحادثات — zzz.txt)

دقيقة 40-90 (narrative-bound) دقائق 6-10 FVG مُجِيع patch
دقيقة 30-60 دقائق 3-5 patch narrative positional lookup
دقيقة 25-45 دقائق 2-4~ (فوق الثلاثة) processes توازي
لاحقاً (بتفويض منفصل) PATCH التي ستحتاج CLOSED – ملفات 12
src/trading_system/core/causal_percentile.py (\$) (الأساسي – 13) expanding استبدال مسار:
والمراقبات كما هي حرفياً API sorted-buffer –
فوق كل المرشحين sum(...) مجاميع تراكمية بدل إعادة kb[i]/ks[i] = (ناوِي) src/trading_system/zones/fvg.py
. يحتاج بطاقات تكافؤ خاصة به semantics-preserving إصلاح – (لكل شمعة 0(1)→0(n²)) كل شمعة
src/trading_system/decision/narrative.py (اختياري): _value + label-access → numpy positional مصفوفات
مُحمّلة مرة واحدة (نفس القيم – نفس المخرجات).
swing_detector / structural_breaks / volume_delta / absorption / liquidity / order_blocks / volatility
primitive. يكتسبون تلقائياً من استبدال ال –
patch surface (الدقيق) أقل – 13
فقط CausalPercentileTracker ملف واحد، داخل
rank(): مرّتب buffer على np.searchsorted العُدّات عبر
strict-compare-first-wins.
مرّتب buffer في (C) memmove | إدراج ب – push()
من np.fromiter بدل np.quantile(..., method="linear") مباشرة إلى view تمرير نسخة: percentile_value()
deque.
مطابق. FIFO extrema يبقى على المنطق الحالي أو يُعاد حساب (غير مستخدم من المحركات) max_history مسار
thresholds/detector definitions. صفر تغيير مراقبات، صفر تغيير، API صفر تغيير
ميداني مطابق. rerun + اختبارات المشروع كما هي 1045 كاملة + 6\$/7 التحقق: بطاقات \$
– المخاطر 14
extrema – ؛ يبقى خطر إعادة ظهوره لو تغيّر مسار 6 الأصفار (مُكتشف ومُغلّق بقاعدة مطابقة – انظر \$ sign-bit
البوابة تحميه.
. غير مغطى بالمحركات – يبقى خلفية خطر منخفضة (بطارية إضافية مقترحة عند التنفيذ) max_history مسار
على كل الحالات (يعتمد على الحزمة) – يبقى خطر نظري عند bitwise وترتيب الإدخال: مُثبّت تجريبياً np.quantile
البوابة تُعاد على أي ترقية → numpy ترقبات.
. بطاقات تكافؤ مستقلة إلزامية – (تدخل أعمدة معروفة kb/ks) يتطلب فهم أعمق للمخرجات المُشتقة FVG patch
لا يعتمد على شيء. fallback P1 فقط (موجود أصلاً)؛ numpy يعتمد على P2 Windows/numpy versions:
يُتحقق بالميدان. – 9-10x ratio يبقى تقدير آلة المالك (دقائق 3-5) patches الأداء المُقدّر للشهر ما بعد
15 – BUILD READY
Primitive patch (§13): YES – prototypes + 36/36 equivalence + 15/15 causality + benchmarks
(gate battery + 1045 + field rerun). ملف واحد داخلي؛ خطة التحقق جاهزة = patch مكتملة؛ سطح ال

FVG accumulator patch: YES (design) — PENDING OWNER APPROVAL (سطح CLOSED منفصل).

Narrative positional patch: OPTIONAL — PENDING OWNER APPROVAL.

Native layer: NO (غير مبرر حالياً — انظر §10).

production patch الآن: ممنوع (كما قُرض) — هذا التقرير DESIGN ONLY.

STOP.

gpt-5.5-instant

الحالة الرسمية: أول تشغيل حقيقي أثبت أن البيانات والربط صحيحان، لكنه كشف عنق زجاجة حقيقياً: 44,640 دقيقة احتاجت
نحو 10 دقائق للتقلب و26 دقيقة لـFVG وحدها. قبل سنوات من البيانات أو التعلم، هذا الأداء غير صالح. إذا هدفنا «أقصى سرعة
+ نفس الدقة تماماً»، فلا نخفض الدقة؛ نغيّر الخوارزمية والتنفيذ.

لو كان المشروع لي، لن أبدأ بترقيع FVG ومحرك التقلب واحداً واحداً. أبحث عن العمليات البدائية المشتركة التي تجعل الجميع
بطيئاً، وأستبدلها بمحركات حسابية دقيقة وسريعة. عندنا بالفعل مشتبّه رئيسي: الترتيب المئوية على التاريخ المتوسّع.

الفكرة الحالية تقريباً:

[text]

عند النقطة 1: افحص الماضي

عند النقطة 2: افحص الماضي

...

عند النقطة 44,640: افحص كل الماضي

هذا ينتج قرابة ملياري مقارنة لمتتبع واحد. ومع عدة متتبعات يصبح الرقم هائلاً.

البديل الدقيق ليس تقريباً ولا نافذة أقصر. إذا احتجنا معرفة عدد القيم التاريخية الأصغر من القيمة الحالية، نحفظ ببنية مرتبة
ديناميكياً. الإدخال والسؤال يصبحان لوغاريتميين تقريباً. والنتيجة الرياضية تبقى نفسها.

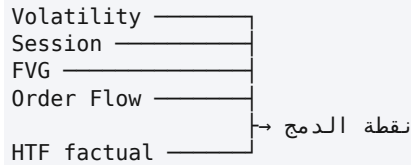
لكن يمكن الذهاب أبعد. بما أن Backtest يعرف كل قيم السلسلة الواقعة في نطاق البحث، نستطيع في عالم البحث استخدام
ضغط إحداثيات مسبق للقيم ثم شجرة جمع، بشرط ألا تستعمل القيم المستقبلية لتحديد قرار أو عتبة؛ ترتيب القيم كآلية
حسابية يحتاج مراجعة سببية دقيقة لأن معرفة مجموعة القيم المستقبلية نفسها قد تصبح اعتماداً على المستقبل إذا تسربت إلى
الدلالة. البديل الآمن تماماً هو بنية مرتبة Online لا تعرف المستقبل إطلاقاً. السرعة قد تظل ممتازة.

سجل المعادلات — zzz.txt

ثم هناك مستوى ثانٍ: نستطيع كتابة النواة الساخنة بلغة أسرع مع إبقاء Python واجهة المشروع. مثلاً نواة Rust أو ++C أو امتداد مجّع، لكن لا أبدأ بذلك قبل إصلاح التعقيد الخوارزمي. كود C سريع على خوارزمية سيئة يظل خوارزمية سيئة. المستوى الثالث هو عدم إعادة الحساب. كل Engine causal يُشغل مرة واحدة على التسلسل، وتُخزن حالته ومخرجاته. أخذ 100 snapshot لا يعيد تشغيل الشهر 100 مرة. Walk-forward لا يعيد حساب الماضي الذي لم يتغير إذا كان العقد يسمح بحالة مستمرة موثقة.

المستوى الرابع هو التوازي. الفروع المستقلة يمكن أن تعمل بالتوازي:

[text]



لكن لا نشغل مرحلتين بالتوازي إذا كانت إحداها تعتمد على الأخرى. ولا نستخدم Threads عشوائياً داخل Python إذا كان حاجر المفسر سيجعلها بلا فائدة؛ عمليات متعددة أو نواة native قد تكون أنسب. والمستوى الخامس: cache بالمحتوى، وليس cache أعمى. مثلاً:

[text]

input hash

+

engine version

+

policy hash

+

contract version

=

cache key

إذا كل شيء مطابق، لماذا نعيد حساب FVG نفسها للمرة العشرين؟ نقرأ artifact موثقاً. إذا تغير أي مدخل أو عقد، الـ cache يصبح غير صالح تلقائياً.

لو جمعنا هذه الأشياء، هدفي لن يكون تخفيض ساعة إلى 40 دقيقة. أريد تغيير مرتبة الأداء.

مثلاً هدف هندسي مبدئي، وليس وعداً حتى نقيس:

[text]

:bar 44,640

ثوانٍ إلى دقائق قليلة للمحركات الأساسية

مئات آلاف bars:

دقائق، لا ساعات

ملايين bars:

قابلة للبحث فعلياً

والقاعدة الذهبية:

[text]

OLD RESULT

===

NEW RESULT

ليس «قريب جداً».

بالنسبة للحسابات التي تستخدم float أصلاً، نطلب تطابقاً وفق عقد الوحدة الحالي: إذا العقد يفرض exact bitwise/canonical frame نطابقه؛ وإذا ترتيب العمليات الحسابية يؤثر في float، لا ندعي bitwise بلا إثبات. نحتاج الحفاظ على الدلالة العددية المعتمدة حرفياً، وليس تغيير ترتيب عمليات ثم القول إن الخطأ صغير.

الأهم: لا أريد أن تطلب من البيلدر الآن «خليه أسرع». أعطه مشروع أداء له عقد واضح. أرسل له هذا النص:

[text]

سبب المهمة:

أول Real Owner Run على 44,640 دقيقة كشف:

Dynamic Volatility:

581.4 s

FVG:

1594.3 s

والتشغيل الكلي غير عملي.

الهدف:

أقصى تسريع ممكن مع ZERO INTENTIONAL SEMANTIC CHANGE و ZERO LOOKAHEAD.

ممنوع حلول:

- تقليل التاريخ.
- rolling window بدلاً من expanding history.
- sampling.
- approximate percentile.
- quantization - تغيير ترتيب القيم.
- إسقاط bars.
- تقليل precision.
- تغيير thresholds.
- تغيير detector definitions.
- استخدام المستقبل لاتخاذ قرار تاريخي.
- تعطيل الاختبارات.
- hard-code لمايو/BTC.
- تحسين benchmark فقط.

لا تعدل CLOSED في هذه المرحلة.

DESIGN + BENCHMARK INVESTIGATION ONLY.

أريد أولاً PROFILING حقيقياً على synthetic workloads عند:

n = 1k, 5k, 10k, 20k, 44,640

ولا تحتاج owner data.

حدد لكل hot primitive:

- الزمن.
- عدد الاستدعاءات.
- complexity.
- allocations.
- peak memory.
- من يستعملها من المحركات.

ابدأ تحديداً بـ:

CausalPercentileTracker

وأي expanding-history rank/percentile primitive.

استخرج الدلالة الرياضية الحالية حرفياً:

- هل percentile محسوب قبل إدخال current أم بعده؟
- tie semantics.
- duplicate semantics.
- NaN/inf policy.
- warm-up.
- reset/state semantics.
- exact dtype/output.
- snapshot/reentrancy behavior.
- serialization - إن وجد.

ثم صمم بدائل Exact Online لا تعرف المستقبل، وقارن:

A) balanced order-statistics tree.

global future-fit coordinate ممنوع coordinate mapping سببي وقانوني؛ Fenwick/segment tree B) فقط إذا

causal information contract compression إذا كان يغير

treap C) أو augmented AVL/RB tree مع multiplicities

native implementation D) محتمل:

Rust / C++ extension

Python data structures تبقى bottleneck إذا

exact historical rank online إعطاء (E) أي خوارزمية خارج الصندوق تستطيع

لا تختار بناءً على النظرية فقط.

Prototype خارج CLOSED وقس الأداء.

الهدف:

تحويل exact expanding rank من $O(n^2)$ الإجمالي إلى $O(n \log n)$ أو أفضل، دون معرفة المستقبل.

بعد ذلك افحص hotspots الأخرى:

- narrative per-bar behavior.

- FVG repeated work.

- repeated DataFrame copies.

- repeated canonical hashing.

- repeated prefix reconstruction.

- Python iterrows/apply loops.

PROXY/ACTUAL. بين - duplicated engine runs

- serialization/reporting.

- trajectory sample recomputation

صمم DAG execution planner:

المراحل المستقلة تعمل مرة واحدة فقط.

النتيجة المشتركة يعاد استخدامها.

لا إعادة تشغيل engine causal لنفس:

input hash + engine version + policy hash

صمم content-addressed cache خارج CLOSED:

cache key =

input artifact hashes

+ engine identity/version

+ config/policy hash

+ contract version.

أي mismatch $\Rightarrow$ cache miss

cache لا يستطيع تجاوز verify/integrity.

صمم parallel execution للمراحل المستقلة فقط.

حدد أيها process-safe وأيها state-dependent.

لا تدع speedup من Python threads دون قياس.

صمم benchmark harness يقارن OLD vs PROTOTYPE.

EQUIVALENCE GATE:

لكل primitive وكل engine متأثر:

- empty.

- 1 row.

- duplicates.

- all equal.

- monotonic increasing.
- decreasing.
- alternating.
- random seeded.
- extreme magnitudes.
- exact ties.
- NaN/inf/error cases حسب العقد.
- multiple chunk boundaries.
- reentrancy.
- prefix truncation.
- future append.
- repeated run.

قارن كل مخرج contractually observable.

إذا العقد الحالي يسمح exact bitwise comparison :
يجب byte/bit exact.

إذا floating arithmetic contract لا يضمن bitwise عبر تنفيذ مختلف :
حدد أولاً معيار التكافؤ الموجود فعلياً من CLOSED contract.
ممنوع اختراع tolerance جديد لتبرير اختلاف.

CAUSALITY GATE:

لنفس $\text{prefix} \leq T$:

إضافة أي future suffix

لا تغير أي $\text{output} \leq T$.

النواة الجديدة Online يجب ألا تقرأ أو fit على future suffix.

PERFORMANCE TARGETS:

لا تجعلها شروط ادعاء قبل القياس، لكن استهدف على الأقل:

Causal percentile primitive at $n=44,640$:

speedup $\geq 20x$ إن أمكن exact.

Engines dominated by trackers:

$\geq 10x$.

ثم تقدير full month field run:

نريد الانتقال من عشرات الدقائق/الساعات إلى دقائق قليلة إن أمكن دون تغيير semantics.

اختبر أيضاً n أكبر:

100k

250k

1m

إذا الموارد تسمح، لاستقراء scalability.

NATIVE ACCELERATION STUDY:

إذا الخوارزمية $O(n \log n)$ في pure Python ما زالت بطيئة:

اقترح طبقة native صغيرة فقط للprimitive الساخن.

قارن:

pure Python exact

.vs Rust/C++ exact

لا تنقل business semantics إلى native.

فقط data-structure primitive.

حدد:

build portability Windows/Linux

wheel/toolchain implications
fallback behavior
determinism
.auditability

لا تدخل dependency ثقيلة بلا مبرر.

OUTPUT:

أعد فقط:

EXACT PERFORMANCE REVOLUTION — DESIGN + BENCHMARK REPORT

ويشمل:

1. flame/profile hotspots.
2. complexity map.
3. formal semantics of old percentile primitive.
4. exact candidate algorithms.
5. benchmark table old vs prototypes.
6. equivalence results.
7. causality results.
8. DAG/caching plan.
9. parallelization plan.
10. native acceleration recommendation YES/NO.
11. projected month/year runtimes.
12. الملفات CLOSED التي ستحتاج PATCH لاحقاً.
13. أقل patch surface.
14. المخاطر.
15. BUILD READY = YES/NO.

ممنوع تعديل أي CLOSED الآن.

ممنوع تحسين production الآن.

Reality analysis. ممنوع.

STOP.

لماذا DESIGN أولاً رغم أنك تريد السرعة فوراً؟ لأن تغيير هذا primitive قد يؤثر في عدة محركات مغلقة في آن واحد. لو استعجلنا وأصبح أسرع 100 مرة لكنه غير معنى tie واحداً، قد تتغير FVG والبنية والفرضيات، ونكون اشترينا السرعة مقابل صحة النتائج من دون أن نلاحظ.

بعد تقرير benchmark، إذا أثبت البيلدر أن Treap مثلاً يعطي نفس النتيجة و50x أسرع، وقتها أعطيك PATCH رسمي مركز جداً. وليس شهراً من إعادة البناء.

ولو احتجنا Native، أنا أميل مبدئياً إلى Rust لنواة صغيرة فقط بسبب سلامة الذاكرة وسهولة جعل واجهتها ضيقة، لكن القرار يكون بالقياس لا بالحماس.

وإذا نجح هذا كما أتوقع، ليس فقط تشغيل مايو سيصبح أسرع. كل مستقبل المشروع يتغير: Walk-forward، سنوات، BTC، ablations، ETH، تدريب النماذج، كلها تصبح قابلة للتنفيذ. هذه المرة تحسين الأداء ليس تجميلاً؛ هو جزء من جعل المشروع عملياً فعلاً.

بدي يكون قمه السرعة لاقى اي طريقه خارج الصندوق يكون قمه السرعة وقمه الدقه المتناهية

gpt-5.5-instant

الحالة الرسمية: التشغيل سليم لكنه أثبت أن الأداء الحالي غير مقبول للبحث الكبير. وصل الآن إلى order_flow_proxy_3_1 من 02:09:47 إلى 02:59:10 مر تقريباً 49 دقيقة، ولسه لم ينته.

الأرقام عندك أصبحت واضحة:

[text]

تحميل + تحقق البيانات ≈ 35 ثانية

Volatility دقائق و41 ثانية ≈ 9

فترة غير منسوبة بوضوح ≈ 12 دقيقة

Session ≈ لحظي

FVG دقيقة و34 ثانية ≈ 26

الآن:

Order Flow Proxy قيد التشغيل

أنا هلق أغير توصية الانتظار: لا أريدك تضيع ساعات فقط حتى نقول إن أول تقرير اكتمل.

السبب ليس أن المشروع فشل. السبب أن أول تشغيل حقيقي كشف مشكلة هندسية واضحة: بعض الخوارزميات صحيحة سبباً لكنها تنفذ نفس الحساب بطريقة تربيعية بطيئة جداً.

وأنت قلت شرطاً واضحاً:

[text]

لا أصحي بالدقة مقابل الوقت.

وأنا موافق.

لذلك الحل ليس تقليل 44,640 شمعة إلى 5,000، وليس استعمال آخر 100 شمعة، وليس تقريب النسب المئوية.

الحل:

[text]

نفس الرياضيات

نفس التاريخ الكامل

نفس المخرجات

نفس السببية

لكن خوارزمية أسرع.

أنصحك الآن بإيقاف التشغيل :-:

[text]

Ctrl + C

مرة واحدة.

لأن عندنا Benchmark كافٍ جداً لإثبات المشكلة. لا نحتاج نحرق ساعتين إضافيتين لنثبت أن بقية المحركات البطيئة بطيئة أيضاً.

بعد Ctrl+C سيظهر Traceback. انسخه كاملاً واحفظه. هذا سيعطينا أيضاً المكان الدقيق الذي كان يعمل فيه.

ثم نعمل مشروع Performance واحد، وليس ترقيعات صغيرة كل شوي.

هدفه:

[text]

PERFORMANCE V2

ZERO SEMANTIC CHANGE

نركز أولاً على الجذر المشترك الذي يبطئ عدة محركات، وليس تحسين FVG وحدها.

من تقرير البيلدر نعرف أن المتهم الأساسي هو:

[text]

CausalPercentileTracker

لأن عدة محركات تستخدم تاريخاً متوسعاً وتحسب الرتبة بإعادة مسح الماضي.

بدل:

[text]

لكل شمعة:

امسح 1..i

نريد:

[text]

احتفظ ببنية مرتبة للماضي

وأدخل كل قيمة مرة واحدة
واحصل على الرتبة الدقيقة
فتتحول تقريباً من:

[text]

n^2

إلى:

[text]

$n \log n$

بدون تغيير الجواب.

والشرط الذي سأفرضه على البيلدر شديد:

على بيانات صغيرة ومتوسطة وعشوائية وحالات ties وNaN والحدود:

[text]

OLD OUTPUT

==

NEW OUTPUT

حرفياً، بما في ذلك أنواع البيانات وحالات البداية.

ثم نشغل المحركات القديمة والجديدة جنباً إلى جنب:

[text]

Volatility القديم == الجديد

FVG القديم == الجديد

Order Flow القديم == الجديد

Narrative القديم == الجديد

إذا اختلفت حقيقة واحدة، لا نعتمد التحسين.

والهدف العملي يجب ألا يكون:

[text]

أسرع شوي.

أريد Benchmark على 44,640 نقطة صناعية بحيث نعرف قبل أن ترجع تشغله على جهازك أن الشهر لن يأخذ ساعة.

والأهم مستقبلاً: المشروع يحتاج ملايين النقاط في Walk-Forward. السرعة ليست رفاهية الآن؛ أصبحت شرطاً علمياً لأن النظام البطيء جداً يمنعنا من إجراء اختبارات كافية لإثبات Edge.

اضغط Ctrl+C الآن، وانسخ لي آخر الـ Traceback الذي يظهر. بعده أعطيك بروميت واحد كامل للبيلدر لتحسين الجذر الحسابي بدون أي تنازل بالدقة.

كثير عم يطول ليك وين صار ولوقت

[02:09:54] STEP loading sources through CLOSED adapters (validate + cross-witness + canonical) ...

[02:10:28] STEP sources loaded + exact cross-witness OK (44640 minutes) in 34.6s

[02:10:41] STEP sealing MarketObservationTimeline (consumer-side public seal) ...

[02:10:41] STEP timeline sealed: 44640 bars in 0.0s (hash a8578565bd4a3962...)

[02:10:41] STEP deterministic sample selected: 100 timestamps

[02:10:41] STEP running CLOSED engines (this is the long stage) ...

[02:10:41] ENGINE dynamic_volatility_1_1 ...

[02:20:23] ENGINE dynamic_volatility_1_1 done in 581.4s

[02:32:35] ENGINE session_context_1_2 ...

[02:32:35] ENGINE session_context_1_2 done in 0.0s

[02:32:35] ENGINE fvg_4_2a ...

[02:59:10] ENGINE fvg_4_2a done in 1594.3s

[02:59:10] ENGINE order_flow_proxy_3_1 ...

الحالة الرسمية: التشغيل ما زال صحيحاً ويتقدم. البيانات والمصادر والTimeline كلها نجحت، ومحرك التقلب انتهى فعلياً. الآن دخل إلى محرك FVG على بيانات BTC الحقيقية.

لكن ظهر لنا شيء مهم جداً عن الأداء.

محرك التقلب أخذ:

[text]

581.4 ثانية

يعني تقريباً:

[text]

9 دقائق و41 ثانية

وهذا أبطأ بكثير من تقدير البيلدر على جهازه.

وفيه أيضاً فجوة زمنية غريبة:

[text]

dynamic volatility انتهى: 02:20:23

session بدأ: 02:32:35

يعني حوالي 12 دقيقة لم ينسبها الـ Progress log إلى Engine واضح. هذا يستحق التحقيق لاحقاً، لكنه ليس سبباً لإيقاف التشغيل الحالي.

الآن:

[text]

... ENGINE fvg_4_2a

وهذا كان من أثقل المحركات أصلاً.

بما أن التقلب أخذ عندك قرابة 8-9 أضعاف تقدير البيلدر، لا أستبعد أن FVG يحتاج وقتاً طويلاً جداً على جهازك، ربما عشرات الدقائق.

لكن هنا وصلنا لنقطة قرار مهمة.

أنا لا أريد أن تعيش مع نظام يأخذ ساعة أو ساعتين لتحليل شهر واحد. لو أردنا لاحقاً:

[text]

3 سنوات

BTC

ETH

عدة أطر

Walk-forward

مئات التجارب

بهذه السرعة المشروع غير عملي بحثياً.

ولا أريد حل المشكلة بتقليل الدقة.

الحل الذي أراه بعد هذا التشغيل هو تحسين الأداء مع شرط:

[text]

ZERO SEMANTIC CHANGE

أي لا:

[text]

تقليل البيانات

تقليل التاريخ

تقريب percentile

تغيير thresholds

بل نغير فقط طريقة الحساب.

مثلاً إذا النظام الحالي يحسب رتبة القيمة الحالية هكذا كل دقيقة:

[text]

افحص كل التاريخ السابق

واحسب الرتبة من جديد
نستبدله ببنية مرتبة متزايدة تحفظ الماضي:

[text]

أدخل القيمة الجديدة

↓

اعرف عدد القيم الأصغر منها بسرعة
فتكون النتيجة الرياضية نفسها، ولكن بسرعة أكبر بكثير.

إذا استطعنا تحويل الجزء الأساسي من:

[text]

$O(n^2)$

إلى:

[text]

$O(n \log n)$

ففرق الأداء عند سنوات من البيانات قد يكون هائلاً.

لكن لا نفتح هذا الآن. التشغيل الحالي قطع شوطاً حقيقياً، وFVG شغال.

خليه يكمل FVG.

لا توقفه ما دام CPU شغال.

إذا FVG تعدّى مثلاً 45-60 دقيقة لوحده، خبرني قبل أن توقفه. وقتها يصبح عندنا مبرر قوي أن أول Field Run نفسه غير عملي ونوقف لنصلح الأداء.

أما إذا خلص خلال وقت معقول وانتقل إلى:

[text]

order_flow

نكمل.

وبمجرد أن نحصل على أول تقرير، سأوصي بقوة أن يكون تحسين الـ Causal Percentile Engine أولوية قبل أي تشغيل تاريخي كبير.

لأن المطلوب في النهاية ليس فقط نظام دقيق.

نحتاج:

[text]

دقيق

+

سببي

+

قابل للتدقيق

+

سريع بما يكفي لإجراء آلاف التجارب

حالياً عندنا أول ثلاثة إلى حد كبير.

الأداء هو الدين الذي كشفه السوق الحقيقي الآن.

```
C:\Users\pc\Desktop\usdt\30_BTC_MAY2026_FIELD_RUNNER_PACKAGE_PROGRESS_PATCHED\BTC_MAY2026_FIELD_PACKAGE>python run_btc_may_2026.py
```

```
[02:09:47] BTC MAY 2026 FIELD RUNNER starting
```

```
[02:09:47] NOTE: the certified CLOSED engines use expanding-history empirical percentiles
```

```
[02:09:47] (exact history scans by contract). At month scale (~44,640 bars) the engine
```

```
[02:09:47] stage is expected to take TENS OF MINUTES on a desktop CPU. Each stage below
```

```
[02:09:47] prints its own timing so you can see progress; nothing is written until the end.
```

```
[02:09:47] STEP inputs: minute_facts=minute_facts_BTCUSDT-aggTrades-2026-05.csv
```

```
sidecar=minute_facts_BTCUSDT-aggTrades-2026-05.sidecar.json raw=ABSENT (FAST mode, provenance via sidecar)
```

```
[02:09:47] STEP identity gates passed (artifact-role hashes + sidecar bindings)
```

سجل المحادثات — zzz.txt

downloading official artifact:

<https://data.binance.vision/data/spot/monthly/klines/BTCUSDT/1m/BTCUSDT-1m-2026-05.zip>

downloaded BTCUSDT-1m-2026-05.zip: 2120776 bytes in 4.7s

downloading official artifact:

<https://data.binance.vision/data/spot/monthly/klines/BTCUSDT/1m/BTCUSDT-1m-2026-05.zip.CHECKSUM>

downloaded BTCUSDT-1m-2026-05.zip.CHECKSUM: 88 bytes in 0.8s

[02:09:54] STEP klines ready: BTCUSDT-1m-2026-05.csv

(acquisition=DOWNLOADED_FROM_DATA_BINANCE_VISION_CHECKSUM_VERIFIED)

[02:09:54] STEP loading sources through CLOSED adapters (validate + cross-witness + canonical) ...

[02:10:28] STEP sources loaded + exact cross-witness OK (44640 minutes) in 34.6s

[02:10:41] STEP sealing MarketObservationTimeline (consumer-side public seal) ...

[02:10:41] STEP timeline sealed: 44640 bars in 0.0s (hash a8578565bd4a3962...)

[02:10:41] STEP deterministic sample selected: 100 timestamps

[02:10:41] STEP running CLOSED engines (this is the long stage) ...

[02:10:41] ENGINE dynamic_volatility_1_1 ...

[02:20:23] ENGINE dynamic_volatility_1_1 done in 581.4s

[02:32:35] ENGINE session_context_1_2 ...

[02:32:35] ENGINE session_context_1_2 done in 0.0s

[02:32:35] ENGINE fvg_4_2a ...

gpt-5.5-instant

الحالة الرسمية: كل بوابات البيانات الحقيقية نجحت، والـ Runner دخل الآن فعلياً إلى المحركات الأصلية المغلقة للمشروع. حتى هذه اللحظة لا يوجد أي خطأ.

الذي تحقق فعلياً الآن:

[text]

ملف Minute Facts الحقيقي

✓

شموع Binance الرسمية

✓

بصمة Binance الرسمية

✓

المقارنة بين المصدرين

44,640 / 44,640 دقيقة

✓

Cross-witness exact

✓

Timeline

44,640 bar

✓

عينة 100 لحظة اختيرت قبل رؤية النتائج

✓

وهذا السطر هو المهم الآن:

[text]

... ENGINE dynamic_volatility_1_1

يعني المشروع الأصلي نفسه بدأ الحساب.

لم نعد في مرحلة تحميل البيانات.

حالياً محرك التقلب المغلق:

Dynamic Volatility V1.1

يعمل على شهر BTC الحقيقي.

المتوقع حسب قياسات البيلدز أن هذه المرحلة قد تأخذ على جهازك تقريباً دقيقة إلى عدة دقائق. بعد انتهائها يجب أن ترى شيئاً قريباً من:

[text]

... ENGINE dynamic_volatility_1_1 ... done in

ثم سينتقل لمحركات أخرى.

المراحل الثقيلة التي يجب أن تتوقع أنها قد تبقى عليها فترة هي خصوصاً:

[text]

FVG

و:

[text]

Narrative

لأنهما كانا من أبطأ الأشياء في القياسات.

فإذا رأيت مثلاً:

[text]

... ENGINE fvg

وبقي 10 دقائق، لا تستنتج أنه علق.

الآن صار عندنا أيضاً دليل مهم جداً كان ناقصاً سابقاً: بيانات AggTrades التي حولناها والشموع الرسمية لـ Binance تطابقت في 44,640 دقيقة حسب عقد الـ Cross-Witness.

هذا يقوي جداً ثقتنا أن البيانات التي سيدرسها المشروع ليست ملف شموع مصنوعاً بطريقة خاطئة.

لكن تذكر ما المتوقع لاحقاً مع الـ Swing:

لأننا تركنا:

[text]

swing_quantile = null

المفروض فرع:

[text]

Swings

Structure

Liquidity

Order Blocks

Dealing Range

يعلن أنه غير مهياً، أو لا يشغله الـ Runner.

هذا مقصود حالياً.

لا نريد أن نضع رقماً من رأسنا فقط لكي نرى رسومات.

خليه يكمل.

إذا انتقل إلى محرك جديد، لا تحتاج تبتعلي كل سطر. فقط إذا:

ظهر ERROR.

بقي على محرك واحد أكثر من 15-20 دقيقة.

أو انتهى بالكامل.

إذا انتهى بالكامل، أرسل SUMMARY.json و MACHINE_VALIDATION.json.

هلق فعلياً «المكيبة شغالة».

هون صار

C:\Users\pc\Desktop\usdt\30_BTC_MAY2026_FIELD_RUNNER_PACKAGE_PROGRESS_PATCHED\BTC_MAY2026_FIELD_PACKAGE>python run_btc_may_2026.py

سجل المعادلات — zzz.txt

[02:09:47] BTC MAY 2026 FIELD RUNNER starting
 [02:09:47] NOTE: the certified CLOSED engines use expanding-history empirical percentiles
 [02:09:47] (exact history scans by contract). At month scale (~44,640 bars) the engine
 [02:09:47] stage is expected to take TENS OF MINUTES on a desktop CPU. Each stage below
 [02:09:47] prints its own timing so you can see progress; nothing is written until the end.
 [02:09:47] STEP inputs: minute_facts=minute_facts_BTCUSDT-aggTrades-2026-05.csv
 sidecar=minute_facts_BTCUSDT-aggTrades-2026-05.sidecar.json raw=ABSENT (FAST mode, provenance
 via sidecar)
 [02:09:47] STEP identity gates passed (artifact-role hashes + sidecar bindings)
 downloading official artifact:
<https://data.binance.vision/data/spot/monthly/klines/BTCUSDT/1m/BTCUSDT-1m-2026-05.zip>
 downloaded BTCUSDT-1m-2026-05.zip: 2120776 bytes in 4.7s
 downloading official artifact:
<https://data.binance.vision/data/spot/monthly/klines/BTCUSDT/1m/BTCUSDT-1m-2026-05.zip.CHECKSUM>
 downloaded BTCUSDT-1m-2026-05.zip.CHECKSUM: 88 bytes in 0.8s
 [02:09:54] STEP klines ready: BTCUSDT-1m-2026-05.csv
 (acquisition=DOWNLOADED_FROM_DATA_BINANCE_VISION_CHECKSUM_VERIFIED)
 [02:09:54] STEP loading sources through CLOSED adapters (validate + cross-witness + canonical) ...
 [02:10:28] STEP sources loaded + exact cross-witness OK (44640 minutes) in 34.6s
 [02:10:41] STEP sealing MarketObservationTimeline (consumer-side public seal) ...
 [02:10:41] STEP timeline sealed: 44640 bars in 0.0s (hash a8578565bd4a3962...)
 [02:10:41] STEP deterministic sample selected: 100 timestamps
 [02:10:41] STEP running CLOSED engines (this is the long stage) ...
 ... [02:10:41] ENGINE dynamic_volatility_1_1

gpt-5.5-instant

الحالة الرسمية: ممتاز. هذه المرة التشغيل ماشي بالطريق الصحيح فعلياً، والـ FAST mode اشتغل كما أردنا. لا توقفه.
 الذي حصل حتى الآن، ببساطة:

[text]

وجد Minute Facts الحقيقية عندك

✓

ثم:

[text]

وجد الـ Sidecar الصحيح

✓

ثم تحقق أن بصمات الملفات وسلسلة المصدر صحيحة:

[text]

identity gates passed

✓

والأهم: لم يحتج ملف الـ 1.69GB:

[text]

raw = ABSENT

FAST mode

✓

ثم نزل من Binance الرسمي ملف الشموع الدقيقة:

[text]

BTCUSDT-1m-2026-05.zip

حجمه حوالي 2.1 MB فقط.

ونزل أيضاً:

[text]

CHECKSUM

وتحقق منه:

[text]

CHECKSUM_VERIFIED

✓

يعني لم يأخذ أي ملف عشوائي من الإنترنت.

هلق وصل لهذه المرحلة:

[text]

STEP loading sources through CLOSED adapters

(validate + cross-witness + canonical)

معناها حالياً عم يعمل شيء مهم:

[text]

AggTrades من 21 مليون Minute Facts

+

شموع Binance الرسمية

↓

يقارن الاثنين

↓

High

Low

Volume

Quote Volume

Executed Flow

...

↓

إذا في تناقض غير قانوني:

يتوقف

إذا كل شيء متوافق:

يبنى بيانات السوق الرسمية

هذه المرحلة ممكن تأخذ بعض الوقت، لكن المفروض ليست أبطأ مرحلة.

بعدها ستبدأ تشوف أسماء مراحل مثل:

[text]

timeline

volatility

FVG

order flow

...

narrative

وهناك سيبدأ الحساب الثقيل فعلاً.

الآن لا تعمل شيئاً.

ولا تقلق إذا بقي:

[text]

...loading sources

عدة دقائق.

إذا ظل على نفس السطر أكثر من 10-15 دقيقة، خبرني فقط؛ لا توقفه مباشرة.
أما عندما يظهر السطر التالي، انسخه لي إذا بدك تتابع مع بعض وين وصل.
والأهم: الذي يحدث الآن هو فعلاً تشغيل على بياناتك، وليس Fixture.
لدينا:

[text]

44,640 دقيقة BTC حقيقية

+

شموع Binance الرسمية

+

Executed Flow مشتق من ملفك الحقيقي

عم تدخل الآن لأول مرة إلى العقود المغلقة للمشروع.

خليه يكمل.

هيك عم يمشي

C:\Users\pc\Desktop\usdt\30_BTC_MAY2026_FIELD_RUNNER_PACKAGE_PROGRESS_PATCHED\BTC_MAY2

026_FIELD_PACKAGE>python run_btc_may_2026.py

[02:09:47] BTC MAY 2026 FIELD RUNNER starting

[02:09:47] NOTE: the certified CLOSED engines use expanding-history empirical percentiles

[02:09:47] (exact history scans by contract). At month scale (~44,640 bars) the engine

[02:09:47] stage is expected to take TENS OF MINUTES on a desktop CPU. Each stage below

[02:09:47] prints its own timing so you can see progress; nothing is written until the end.

[02:09:47] STEP inputs: minute_facts=minute_facts_BTCUSDT-aggTrades-2026-05.csv

sidecar=minute_facts_BTCUSDT-aggTrades-2026-05.sidecar.json raw=ABSENT (FAST mode, provenance via sidecar)

[02:09:47] STEP identity gates passed (artifact-role hashes + sidecar bindings)

downloading official artifact:

<https://data.binance.vision/data/spot/monthly/klines/BTCUSDT/1m/BTCUSDT-1m-2026-05.zip>

downloaded BTCUSDT-1m-2026-05.zip: 2120776 bytes in 4.7s

downloading official artifact:

<https://data.binance.vision/data/spot/monthly/klines/BTCUSDT/1m/BTCUSDT-1m-2026-05.zip>.CHECKSUM

downloaded BTCUSDT-1m-2026-05.zip.CHECKSUM: 88 bytes in 0.8s

[02:09:54] STEP klines ready: BTCUSDT-1m-2026-05.csv

(acquisition=DOWNLOADED_FROM_DATA_BINANCE_VISION_CHECKSUM_VERIFIED)

... [02:09:54] STEP loading sources through CLOSED adapters (validate + cross-witness + canonical)

gpt-5.5-instant

الحالة الرسمية: أنت الآن داخل الحزمة الجديدة رقم 30، والبيانات الصغيرة المطلوبة موجودة في مجلد data. سنبدأ التشغيل الحقيقي مرة ثانية، بنفس الدقة الكاملة، لكن هذه المرة سيظهر التقدم.
في نافذة CMD التي فتحتها داخل مجلد الحزمة، اكتب هذا الأمر حرفياً:

[cmd]

python run_btc_may_2026.py

ثم اضغط Enter.

لا تكتب أي شيء آخر ولا تغيّر أي إعداد.

المفروض هذه المرة تبدأ تظهر رسائل تقدم مثل:

[text]

inputs ...

klines ...

sources ...

timeline ...
volatility ...

fvg ...

order flow ...

narrative ...

... reports

الأسماء الدقيقة قد تختلف.

إذا بدأ يعرض المراحل، اتركه يعمل.

إذا ظهر أي ERROR أو توقف ورجع سطر CMD، انسخ لي النص كاملاً كما ظهر.

وإذا بقي على مرحلة معينة فترة طويلة، لا توقفه مباشرة. فقط اكتب لي مثلاً:

[text]

واقف على FVG من 8 دقائق

أو:

[text]

واقف على Narrative من 10 دقائق

وسأقول لك إن كان هذا ضمن الوقت المتوقع.

وعندما ينتهي بنجاح، المفروض يرجع سطر CMD ويظهر مجلد:

[text]

outputs\btc_may_2026_reality

وقتها لا تعيد التشغيل. أرسل لي محتوى:

[text]

SUMMARY.json

و:

[text]

MACHINE_VALIDATION.json

ابدأ الآن فقط بهذا:

[cmd]

python run_btc_may_2026.py

عملت وجهزت لملف شو اعمل هلق فتحت cmd بملف شو ساوي

gpt-5.5-instant

الحالة الرسمية: التشغيل القديم توقف ولم ينتج النتيجة النهائية. عرفنا أن المشكلة سرعة فقط، وليست بياناتك. وأنت تريد شرطين معاً: لا انتظار مبالغ فيه، ولا أي نقصان في الدقة. أتفق؛ لذلك لا نغيّر أي حساب أو عتبة أو بيانات.

اعمل الآن التالي فقط: نزل النسخة الجديدة:

[text]

BTC_MAY2026_FIELD_RUNNER_PACKAGE_PROGRESS_PATCHED.zip_30

فكها في مجلد جديد.

انسخ إلى مجلد:

[text]

data

ملفين فقط من تشغيلك السابق:

[text]

minute_facts_BTCUSDT-aggTrades-2026-05.csv

وملف الـ JSON الجانبي الخاص به.

لا تنسخ ملف الـ 1.69GB.

ثم افتح CMD في الحزمة الجديدة وشغل:

[cmd]

python run_btc_may_2026.py

هذه النسخة لا تجعل الحساب أقل دقة ولا تختصر البيانات ولا تغير المحركات. النتائج المفروض تكون نفسها. الفرق أنها ستكتب لك أين وصلت، مثلاً:

[text]

Volatility started

Volatility completed: 80 sec

FVG started

FVG completed: 7 min

Order Flow started

...

وبالتالي لن تبقى نصف ساعة أمام شاشة سوداء.

لكن هذا يحل مشكلة «ما يعرف وين وصل»، وليس مشكلة السرعة نفسها.

أما حل السرعة الحقيقي بدون التضحية بالدقة، فهو شيء مختلف تماماً، وممكن غالباً.

حالياً بعض المحركات تعمل تقريباً هكذا:

[text]

الشمعة 1:

افحص الماضي

الشمعة 2:

أعد فحص الماضي

الشمعة 3:

أعد فحص الماضي

...

الشمعة 44,640:

أعد فحص 44,639 شمعة

يعني تعيد حساب أشياء كثيرة مراراً.

نحن لا نريد أن نقول لها:

[text]

بدل كل التاريخ استخدمني آخر 100 شمعة

لأن هذا فعلاً يغير النتيجة والدقة.

نريد تغيير طريقة الحساب فقط.

مثال بسيط. تخيل عندك مليون رقم وتريد معرفة كم رقماً أصغر من الرقم الحالي.

الطريقة الحالية تقريباً:

[text]

لف على كل الأرقام السابقة

وعدهم من جديد

كل مرة.

يمكن بدلاً منها الاحتفاظ ببنية مرتبة تعرف الأرقام السابقة، فتسألها:

[text]

كم رقم أصغر من X؟

وتجيب بسرعة.

الجواب نفسه.

ليس تقريباً.

ليس Sample.

ليس Window أصغر.

يعني هدف تحسين الأداء:

[text]

النتيجة القديمة:

0.736281...

النتيجة الجديدة:

0.736281...

مطابقة

لكن القديمة تأخذ ساعات والجديدة ثواني/دقائق.

هذا يسمى تحسين الخوارزمية، وليس تخفيض الدقة.

لكن لا نعمله الآن أثناء أول تشغيل لأن الوحدات التي سنعدلها CLOSED. يحتاج Patch رسمي واختبار مساواة كامل.

أنا أقترح طريقاً عملياً من مرحلتين:

أولاً شغل نسخة Progress الآن. حتى لو استغرقت 30-60 دقيقة مرة واحدة، نحصل على أول نتيجة حقيقية ونرى إن كان المشروع يستحق.

ثانياً، إذا أردنا الاستمرار، قبل تشغيل سنوات من البيانات نعمل Performance Project رسمي بشرط قاسٍ:

[text]

ZERO SEMANTIC CHANGE

أي:

[text]

المخرجات القديمة == المخرجات الجديدة

على Fixtures، بيانات عشوائية، حالات حدودية، وتاريخ حقيقي.

إذا اختلفت خلية واحدة بدون تفسير عقدي، نرفض التحسين.

والهدف سيكون تحويل الأجزاء البطيئة من شيء قريب من:

[text]

$O(n^2)$

إلى:

[text]

$O(n \log n)$

أو أفضل، مع نفس النتائج بالضبط.

وبالتالي لا يوجد اختيار بين:

[text]

الدقة

أو

السرعة

نريد الاثنين.

لكن حالياً، بما أنك أوقفت القديم: استخدم ZIP رقم 30 وشغل الأمر من جديد. هذه المرة ستري التقدم. لا تغير

swing_quantile ولا أي إعداد. وإذا وصلت مرحلة معينة وظلت عليها مدة طويلة، أعطني اسم المرحلة والمدة التي يعرضها، وسنعرف بالضبط أين عنق الزجاجة بدون تخمين.

ما فهمت شي يعني هلق انا ماكملت القديم وقف شو ساوي ؟ مايدي استنا سنه ليخلص وينفس الوقت مايدي ضحي بالدقه عحساب الوقت ابدًا

الحالة الرسمية: عرفنا سبب البطء. التشغيل ليس عالقاً؛ المشروع نفسه يحتوي حسابات سببية تربيعية تقريباً، ولذلك شهر من 44,640 دقيقة قد يأخذ على جهازك 30-60 دقيقة. البيلدر لم يستخدم بياناتك؛ شخص المشكلة على Fixtures صناعية وعُيِّر فقط الـ Runner ليظهر التقدم. المشروع المغلق بقي 183/183 دون تغيير.

وهنا عندي قرار مهم: لا أريد تعديل المحركات المغلقة الآن لتسريعها. نحن لم نعرف بعد إن كانت تعطي Edge أصلاً. من غير المنطقي فتح وحدات اجتازت التدقيق فقط لتوفير دقائق قبل رؤية أول نتيجة حقيقية.

إذا التشغيل القديم لا يزال يعمل، أمامك خياران. بما أنه صار له وقت طويل بالفعل ويستخدم المعالج، أنا أفضل أن تتركه يكمل بدل رمي الحساب الذي أنجزه. الـ Patch الجديد لا يغير النتائج؛ فقط سيخبرك أين وصل في التشغيلات المقبلة.

إذا القديم انتهى، لا تعيد تشغيل الشهر بالـ ZIP الجديد الآن. خذ النتائج وانتهينا.

إذا اضطررت لإيقافه أو تعطل، استخدم:

30_BTC_MAY2026_FIELD_RUNNER_PACKAGE_PROGRESS_PATCHED.zip

وفيه ستشاهد شيئاً مثل:

[text]

Loading inputs...

Klines...

Source validation...

Timeline...

Volatility...

FVG...

Order Flow...

Narrative...

...Writing reports

وسنعرف بالضبط أين هو.

لكن تقرير الأداء كشف شيئاً أريد أن تنتبه له للمستقبل. أبطأ جزأين تقريباً هما:

[text]

≈ 5.6 FVG دقائق على آلة البيلدر

≈ 6.3 Narrative دقائق

بينما Swings والبنية نفسها ليست المشكلة الكبيرة. السبب الأساسي وجود متتبعات رتب مئوية سببية تستخدم التاريخ المتراكم.

هذا ليس خطأ سببية، لكنه غير قابل للتوسع بشكل جيد. لو أخذنا مثلاً 3 سنوات من دقائق BTC، عندنا نحو 1.5 مليون شمعة. خوارزمية تربيعية لن تكون عملية إطلاقاً.

لكن هذه مشكلة مرحلة لاحقة. لو Reality Check أظهر أن المشروع عنده شيء يستحق، نقدر نعمل مشروع تحسين أداء رسمي يحافظ على نفس المخرجات حرفياً. مثلاً استخدام بنية بيانات مرتبة لحساب الرتب التاريخية دون إعادة مسح الماضي في كل بار. الهدف وقتها سيكون:

[text]

Old engine output

==

New optimized engine output

حرفياً

لكن:

$O(n^2)$

→

$O(n \log n)$

إذا أمكن ذلك وفق العقود. وهذا قد يحول سنوات من الحساب إلى شيء عملي، بدون تغيير معنى النظام.

أما الآن فلا نلمس شيئاً.

الشيء الوحيد المطلوب منك: اترك التشغيل الحالي. إذا رجع CMD وظهرت النتائج، أرسل لي:

SUMMARY.json

و:

MACHINE_VALIDATION.json

ولا تعيد تشغيله مرة ثانية لمجرد استخدام نسخة Progress.

وأريد أصح نقطة ذهنية مهمة: نصف ساعة هنا ليست بسبب أن عندنا «21 مليون صف». تلك انتهينا منها. التشغيل الحالي على 44 ألف دقيقة فقط. البطء كشف لنا أن بعض خوارزميات المشروع الحالية غير محسنة للحجم، وهذا اكتشاف مفيد جداً قبل أن نحاول تشغيل سنوات من التاريخ.

الآن الأولوية ليست السرعة. الأولوية: هل الشيء البطيء الذي حسبناه يستحق أصلاً أن نجعله سريعاً؟ أول تقرير حقيقي سيبدأ يجاوبنا.

FIELD RUNNER — PERFORMANCE DIAGNOSIS + PROGRESS PATCH — READY FOR OWNER RETRY

Date: 2026-09-28 (Asia/Damascus)

السبب المختصر: ليس عالقاً — المحركات المغلقة المُصدّقة تعمل بحسابات percentile على تاريخ ممتد كامل لكل شمعة (عقد $O(n^2)$ مقصود: exact empirical percentiles بلا نافذة) + آلة سرديّة per-bar. على 44,640 شمعة = مليارات التكرارات في Python ⇒ عشرات الدقائق طبيعية جداً. لا بطء بسبب بياناتك ولا بسبب ملفاتك.

1 — القياس (SYNTHETIC month-scale fixtures — لا بيانات مالك)

$n = 22,320$ (نصف الشهر) — على آلة البناء — والتقدير $4 \times$ للأجزاء التربيعية على 44,640:

stage نصف (22,320) تقدير الشهر (44,640)

dynamic_volatility 1.1 (2 trackers) 16.8s ~67s

fvg 4.2A (tracker + events) 83.6s ~5.6 min

order_flow PROXY 3.1 20.6s ~82s

order_flow ACTUAL 3.1 19.8s ~80s

absorption 3.2 $\times 2$ 0.4s ~2s

HTF 5.1 3.3s ~13s

swings 2.1A + 2.1B + 2.1C 3.6s ~15s

liquidity 2.2 12.2s ~49s

order_blocks 4.1 3.5s ~14s

dealing_range 4.2B 1.8s ~7s

evidence 6.1A $\times 2$ ~0s ~0s

narrative 6.1B $\times 2$ 94.1s ~6.3 min

timeline seal 1.3s ~5s

المجموع ~4.2 min ~17-20 min

تقرير/مخرجات ~1-2 دقيقة. على Windows (Python أبطأ + مضاد فيروسات + أقدم CPU) المعتاد $3-1.5 \times$ ⇒ 30-60 دقيقة متوقعة للشهر كامل — «نص ساعة وما خالص» يقع داخل هذا النطاق؛ سينتهي. (Primitive مُقسّم: tracker واحد $\times 40.6s = 44,640$ ؛ وعدد trackers في المحركات ~18+).

2 — ماذا تفعل الآن مع التشغيل الجاري؟

الآلة الحالية تتقدم بصمت (المخرجات تُكتب في النهاية فقط) — لا شيء تالف، ولا ملفات نصف مكتوبة. خياران:

انتظر — يُرَجَّح أن يكمل قريباً؛ أو

Ctrl+C ثم أعد التشغيل بالـ ZIP الجديد — يُظهر كل مرحلة بتوقيتها (موصى به).

إذا ظهر عندك التنصيب: تأكد من عدم تعليق تنزيل klines (بطء شبكتك مع data.binance.vision) — ZIP الجديد يطيع حالة التنزيل + يوقفه بعد 600 ثانية برسالة تطلب ملفاً محلياً (klines-csv--).

3 — PATCH الـ (runner فقط — لا CLOSED ولا semantics)

→ كل محرك على حدة (seal → sources → klines → gates → inputs) سطور تقدّم لكل مرحلة بتوقيت لافتة مسبقة صريحة للتكلفة المتوقعة + (التقارير → evidence/narrative).

حراسة تنزيل klines: رسائل بايتات/ثانية + ⇒ (600s) deadline

NETWORK_UNAVAILABLE_OR_DOWNLOAD_TOO_SLOW مع اسم الملف المحلي المطلوب بدل الانتظار اللانهائي.

تسريع glue في التقارير (groupby بدل iterrows — نفس النتائج).

لا defaults تغيّرت (swing_quantile: null)؛ لا engine جديد؛ Reality analysis؛ لا تعديل trading_project.

الاختبارات: runner tests PASS 31/31 بعد الرقعة. 7796a73f...1587 MANIFEST = 183/183 OK (لم يتغير).

سجل المحادثات — zzz.txt

CLOSED tree = NONE changed

4 — الملفات المعدلة (قبل ← بعد)

file قبل (v2) بعد (v3)

field_runner/pipeline.py 58fe777e...440e5c 9774b5e1809d841c...

field_runner/reporting.py 5b7f39b6...d9f20 aa2cfbfb3dfe9a62...

field_runner/runner_btc_may_2026.py a15672ca...df74e ceb6529de0554eda...

field_runner/binance_kline_downloader.py cb13c80c...6edd a15666c7c59beb87...

ZIP — 5 + الأمر

deliverables/30_BTC_MAY2026_FIELD_RUNNER_PACKAGE_PROGRESS_PATCHED.zip

zip_sha256 = b35ebcd289e646c32f2f5855e072013473364254bba0c25c62a29829ea9c5dd6 (729,677

bytes — 205 entries — roundtrip 203/203)

[text]

python run_btc_may_2026.py

6 — خيار مستقبلي (يحتاج قرارك — لم يُنقذ)

التسريع الحقيقي للشهر = تغيير داخل CLOSED (مثلاً بنية percentile مرتبة، أو max_history مُعاير بعقد موثّق) — هذا تغيير على وحدة مغلقة/مُصدّقة ويحتاج تفويضاً منفصلاً وإجراء إغلاقك المعتاد. ممنوع على البيلدر فعله ضمن patch. إن أردته فهو مشروع مستقل بحده.

STOP.

gpt-5.5-instant

الحالة الرسمية: التشغيل الحقيقي يعمل، وليس متجمداً بحسب استهلاك Python المستمر 12-13%. لكن نصف ساعة لشهر واحد أصبحت ملاحظة أداء حقيقية؛ لا أريد أن نفترض أن هذا طبيعي لمجرد أن Python يعمل.

صحيح جزئياً: Python يستطيع معالجة 44,640 شمعة بسرعة كبيرة إذا كانت العمليات خطية ومكتوبة جيداً. تحويل 44 ألف صف وحده يجب ألا يحتاج نصف ساعة. لكن الـ Runner يشغل عدة محركات، ويبني أدلة وفرضيات، ويعمل عينات سببية ونتائج مستقبلية. إذا كان يعيد بناء التاريخ مراراً لكل نقطة عينة، فمن السهل أن يتحول العمل من نحو 44 ألف عملية إلى ملايين أو مئات الملايين.

وجود 12-13% CPU غالباً يعني نواة واحدة مشغولة بالكامل. هذا لا يعني أن الأداء جيد؛ يعني فقط أنه ليس عالقاً.

أنا لا أريد أن توقف أول Real Run بعد نصف ساعة طالما الذاكرة مستقرة والمعالج يعمل. أعطه حالياً حتى ساعة واحدة إجمالاً. هذا أول Benchmark حقيقي، وإن انتهى سنحصل على النتائج وعلى زمن التنفيذ الحقيقي معاً.

لكن إذا وصل إلى ساعة وما خلاص، وقتها أنا مع إيقافه. لأن هدفنا لاحقاً سنوات من البيانات، وليس شهر واحد. لو الشهر يأخذ ساعات، فالسنوات تصبح غير عملية.

وفي هذه الأثناء نستطيع نعرف أين وصل بدون لمس العملية. افتح CMD ثانية داخل نفس المجلد وشغل:

[cmd]

dir outputs\btc_may_2026_reality

ثم:

[cmd]

dir data

وأرسل لي الناتج فقط.

إذا مجلد النتائج يحتوي ملفات بدأت تظهر، نعرف المرحلة التي وصل لها.

إذا ما في أي Output بعد نصف ساعة، فهذا يرجح أن مرحلة حساب داخلية طويلة تحتجز كل النتائج حتى النهاية.

وعندي اشتباه محدد من تقرير البيلدر: DETECTION_AUDIT اختُبر بأنه يعيد حساب كل نقطة على prefix مستقل لإثبات عدم تسريب المستقبل. هذا ممتاز كاختبار، لكن إذا كان الـ Runner الحقيقي يعيد تشغيل Pipeline كاملاً على 100 timestamp، فهذا تصميم شديد الهدر.

مثلاً بصورة مبسطة، الخطأ الهندسي المحتمل:

[text]

شغل الشهر كله

+

للعينة 1:

أعد تشغيل التاريخ من البداية

للعينة 2:

أعد تشغيل التاريخ من البداية

...

للعينة 100:

أعد تشغيل التاريخ من البداية

بينما الصحيح بعد إثبات السببية قد يكون:

[text]

شغل المحركات السببية مرة واحدة

↓

احفظ النتائج التاريخية

↓

خذ الـ 100 snapshots منها

هذا لا يغير الحقيقة السببية إذا المحركات سببية أصلاً، وبوفر وقتاً ضخماً.

كذلك لا أريد «الحل» أن يكون تشغيل 8 أنوية عشوائياً. أولاً نعرف لماذا هو بطيء. تحسين الخوارزمية من 100 إعادة حساب إلى حساب واحد أهم بكثير من parallelism.

فالقرار الآن:

إذا لم يصل إجمالي التشغيل إلى ساعة، اتركه.

إذا انتهى قبلها، ممتاز: لا نصلح شيئاً قبل أن نقرأ النتائج وزمن التشغيل.

إذا وصل ساعة وما انتهى، اضغط Ctrl+C مرة واحدة وانسخ لي آخر الـ traceback؛ مكان المقاطعة غالباً سيكشف تحديداً أي دالة تستهلك الوقت. بعدها نعطي البيلدر Patch أداء للـ Runner الخارجي فقط، مع شرط أن تكون النتائج قبل/بعد متطابقة بنأ قدر الإمكان.

ولا تقلق من فكرة أن «Python لازم يكون سريع دائماً». 44 ألف صف قليلة؛ لذلك بطء كهذا يخبرنا على الأغلب أن المشكلة ليست حجم البيانات، بل مقدار العمل المعاد لكل صف أو لكل عينة. وهذا شيء قابل للإصلاح غالباً.

طلب كثير طول انا بعرف انوك cmd وبايثون لمفروض يكون سريع
صرلو شي نص ساعة

gpt-5.5-instant

الحالة الرسمية: التشغيل الحقيقي ما زال جارياً، والـ Python عندك يستهلك تقريباً 12-13% من المعالج باستمرار. هذا مؤشر قوي أنه ليس معلقاً؛ هو يحسب فعلياً.

إذا معالجك 8 أنوية منطقية مثلاً، 12.5% تقريباً تعني نواة كاملة مشغولة 100%. وهذا منطقي جداً إذا أغلب المشروع يعمل على مسار Python أحادي النواة.

إذن الآن: لا توقفه.

السبب المرجح للبطء أن البيلدر اختبر الـ Runner على 240 دقيقة فقط، بينما جهازك يعالج:

44,640 دقيقة

أي حوالي 186 ضعف عينة الاختبار.

وفوق ذلك الـ Runner لا يشغل محركاً واحداً. هو يمر على سلسلة كبيرة من المشروع ويصنع تقارير وعينات وفيلم مستقبل. وبعض الحسابات قد يعاد تنفيذها على Prefixes للتحقق من السببية.

لا أقدر أعطيك وقت انتهاء موثوقاً لأننا لم نقس التشغيل الكامل على 44,640 دقيقة سابقاً. قد يكون عشرات الدقائق، وقد يمتد لساعات إذا توجد أجزاء ذات تعقيد حسابي سيئ. لكن 12-13% المستمر يعني حالياً "يعمل"، وليس "واقف".

في نفس الوقت هذا كشف لنا معلومة هندسية مهمة: إذا احتجنا مستقبلاً تشغيل سنوات من البيانات، لا يمكن أن نقبل Pipeline يحتاج ساعات طويلة لكل شهر إذا كان السبب إعادة حساب غير ضرورية. بعد انتهاء أول تشغيل، سنقرأ runtime ونعرف أين المشكلة قبل أن نجمع سنوات.

حالياً افتح CMD ثانية فقط، ولا تلمس الأولى، واكتب من جذر الحزمة:

[cmd]

```
dir outputs\btc_may_2026_reality
```

إذا قال إن المجلد غير موجود، لا مشكلة؛ قد يكتب النتائج في النهاية دفعة واحدة.

ويمكنك أيضاً النظر إلى Memory في Task Manager. إذا الذاكرة مستقرة مثلاً بضع مئات من MB أو حتى مقدار ثابت معقول، ممتاز. إذا تراها ترتفع باستمرار كل عدة دقائق بدون توقف باتجاه عدة GB، أخبرني لأن ذلك قد يشير إلى تراكم ذاكرة.

لكن طالما:

[text]

CPU ≈ 12-13%

ومستمر يتغير، خليه شغال.

والشيء الحلوهنا: هذه ليست "فحوص زيادة". المعالج الآن فعلياً يشغل نظامنا على شهر BTC الحقيقي. لما يخلص سنرى أول محصول حقيقي للمشروع.

python.exe عاطي 12 و 13 عم يتراوح 12 و 13

gpt-5.5-instant

الحالة الرسمية: التشغيل الحقيقي ما زال جارياً على جهازك. عدم انتهائه بسرعة لا يعني تلقائياً أنه عالق؛ عنده 44,640 دقيقة ويشغل عليها عدداً كبيراً من محركات المشروع، وبعضها سببي تراكمي وقد يكون أبطأ بكثير من مجرد تحويل البيانات.

لكن ما بدي نخليك تنتظر بلا معرفة. نقدر الآن نحدد هل البرنامج يعمل فعلاً أم معلق، من دون إيقافه.

لا تغلق نافذة CMD الأولى ولا تضغط Ctrl+C.

افتح نافذة جديدة وشغل "مدير المهام" في Windows:

Ctrl + Shift + Esc

اذهب إلى قسم Details وابحث عن:

python.exe

راقبه حوالي دقيقة.

إذا كان استهلاك المعالج يتغير، مثلاً 5% أو 20% أو نواة كاملة تقريباً، أو استهلاك القرص يتحرك، فالبرنامج غالباً يحسب فعلاً. خليه يكمل.

إذا كان:

المعالج = 0% تقريباً باستمرار،

والقرص = 0،

والذاكرة ثابتة،

ولا يوجد أي ملف يتغير،

لمدة 5 إلى 10 دقائق، وقتها قد يكون عالقاً أو ينتظر شيئاً.

في نافذة CMD ثانية، وأنت داخل مجلد الحزمة، يمكنك أيضاً كتابة:

[cmd]

```
dir outputs\btc_may_2026_reality
```

لا يؤثر هذا على التشغيل.

إذا ظهرت ملفات، يعني وصل إلى مرحلة كتابة النتائج.

ويمكنك معرفة إن كان تنزيل Klines حصل عبر البحث في data:

[cmd]

```
dir data
```

لا تحذف ولا تعدّل شيئاً.

شيء آخر مهم: الـ Runner الحالي عنده نقطة أداء محتملة. اختبارات البيلدر كانت على 240 دقيقة فقط، بينما أنت تشغل:

دقيقة.

وهذا أكبر بحوالي 186 مرة.

إذا بعض المحركات أو الـ Reality Audit يعيد تشغيل pipeline على prefixes متعددة، الزمن قد يتضخم أكثر من 186 مرة. مثلاً اختبار "ماذا كان يعرف عند T؟" على 100 timestamp قد يعيد حساب أجزاء من التاريخ مراراً. هذا صحيح سببياً لكنه قد يكون بطيئاً جداً.

لذلك إذا كان Python يستهلك CPU فعلياً، لا أريد إيقافه الآن. هذا أول Benchmark حقيقي أيضاً.

أعطني فقط ثلاث معلومات من Task Manager:

[text]

? = python.exe لـ CPU

GB أو MB ? = Memory

? = Disk

وقل لي تقريباً كم دقيقة/ساعة صار له يعمل.

من هذه الأربع معلومات أقدر أقول لك هل تتركه يكمل أم عندنا مشكلة أداء تستحق إيقاف التشغيل وتحسين الـ Runner الخارجي بدون لمس المشروع.

لهلق ماخلص !

gpt-5.5-instant

الحالة الرسمية: التشغيل الحقيقي يجري الآن على جهازك. هذه أول مرة المشروع نفسه يستهلك بيانات مايو الحقيقية عبر الوصلة المغلقة. طالما لم يرجع سطر الأوامر، اتركه يعمل ولا تغلق النافذة.

حتى يكون التشغيل ناجحاً تقنياً، نريد في النهاية أن يرجع الأمر بدون خطأ، وأن ينشئ هذا المجلد:

outputs\btc_may_2026_reality

وداخله خصوصاً:

SUMMARY.json

MACHINE_VALIDATION.json

DETECTION_AUDIT.csv

OUTCOME_FILM_SAMPLE.csv

chart_overlay.csv

وملفات الكيانات المختلفة.

لكن نجاح التشغيل له ثلاث درجات مختلفة، ومن المهم ألا نخلط بينها.

الدرجة الأولى: نجاح البيانات والتوصيل.

نريد أن يخبرنا أن عدد الدقائق هو 44,640، وأن ملفات Binance تطابقت، وأن الـ Source Adapter قبل البيانات، وأنه لم يجد اختلافاً بين حقائق التداولات وشموع Binance في الأشياء التي يجب أن تتطابق.

إذا فشل هنا، لا يعني أن فكرة التداول فشلت. يعني فقط عندنا مشكلة بيانات أو تكامل.

الدرجة الثانية: نجاح تشغيل المشروع نفسه.

نريد أن نرى أن المحركات القانونية اشتغلت فعلاً على BTC، مثل:

التقلب.

السياق الزمني.

الفجوات السعرية.

التدفق التقريبي من الشموع.

التدفق المنفذ الحقيقي من بيانات التداولات.

وأي محرك آخر لا يعتمد على إعداد غير موجود.

أما التآرجحات والبنية والسيولة وكتل الأوامر، فمن المحتمل جداً أن تظهر حالياً على أنها غير مهيأة، لأننا تعمداً عدم اختراع قيمة لسياسة التآرجحات.

هذا ليس فشلاً.

بالعكس، لو أعطاك آلاف التآرجحات الآن رغم أن قيمة سياسة التآرجح غير محددة، سأشك في التشغيل.

الدرجة الثالثة هي التي تهمنا اقتصادياً، ولن يثبتها هذا التشغيل وحده.

وهي:

هل ما يراه النظام يحمل معلومة مفيدة عن المستقبل؟

هذا التشغيل الحالي لا يستطيع أن يقول لنا بعد:

"المشروع مرجح".

ولا:

"FVG ناجحة".

ولا:

"BTC سيصعد".

هو سيعطينا أول صورة حقيقية عما تراه الآلة.

ما أتوقع رؤيته؟

أتوقع أولاً شيئاً قريباً من:

عدد دقائق السوق:

44,640

الدقائق المفقودة:

0

مصدر: OHLC

Binance Klines الرسمية

مصدر التدفق المنفذ:

Binance Spot AggTrades

ثم أرقام عن القلب والجلسات.

ثم أتوقع عدداً من FVG، لأن FVG لا تعتمد على سياسة Swing.

ثم بيانات تدفق منفذ لكل دقيقة تقريباً، بحيث نعرف كمية التنفيذ المبادر من جهة الشراء ومن جهة البيع.

وقد نرى إحصاءات مثل مجموع أو توزيع:

BUY initiated volume

و:

SELL initiated volume

لكن لا نفسر الفرق فوراً على أنه "ضغط شراء".

بعدها أتوقع أن نرى بعض الفروع تقول:

NOT_CONFIGURED

وهذا غالباً سيشمل أشياء معتمدة على Swing.

ومن الممكن أن يؤثر ذلك أيضاً على Evidence والفرضيات؛ لأن النظام لا يستطيع بناء سرد كامل إذا كانت عائلات مهمة غير متوفرة.

وهذا طبيعي في أول تشغيل.

أما السؤال: "كم أتوقع نسبة نجاحه؟"

إذا تقصد نجاح الأمر تقنياً، عندنا أسباب جيدة لتوقع أن يكمل: البيلدر اختبر الRunner على Fixtures، الAdapter مغلق، البيانات سبق أن مرت بالمحول بالكامل، والKlines الرسمية نفسها اختبرها البيلدر. لكن جهازك الحقيقي هو الاختبار النهائي، لذلك لن أعطي نسبة وهمية.

إذا تقصد نجاح المشروع في التداول والربح، فلا توجد حالياً نسبة صادقة يمكن إعطاؤها.

لسا صفر دليل مباشر على الربحية.

وهذا بالضبط ما سنبدأ اختبار به بعد هذه النتيجة.

الشيء الذي سيجعلني سعيداً جداً من التشغيل الحالي ليس أن يعطينا صفقة شراء.

بل أن يقول شيئاً مثل:

"قرأت 44,640 دقيقة."

"المصادر تطابقت."

"المحركات التالية اشتغلت."

"المحركات التالية غير مهياة، ولم أخترع لها إعدادات."

"وجدت كذا FVG."

"وجدت كذا حالة قلب."

"التدفق المنفذ موجود ومتصالح."

"هذه 100 لحظة مجمدة، وهذا بالضبط ما كان معروفاً وقتها."

هذه ستكون أول مرة نعرف أن المختبر الذي قضينا كل هذا الوقت ببنائه يعمل فعلاً على سوق حقيقي كامل.

بعدها تأتي نقطة مهمة جداً.

لن أقول لك مباشرة:

"يلا نعلم AI."

أولاً سنفتح chart_overlay.csv ونختار عينات من الشارت ونرى فعلياً ماذا اكتشف النظام.

مثلاً سننظر إلى FVG التي وجدها.

هل هي فعلاً مطابقة لتعريف المشروع؟

هل توقيتها صحيح؟

هل تظهر فقط بعد اكتمال الشموع اللازمة؟

وننظر إلى التدفق المنفذ.

وننظر إلى حالات القلب.

وبعد حل سياسة الSwing بطريقة علمية، سنرى الموجات والبنية والسيولة أيضاً.

ثم يأتي الامتحان الحقيقي:

نأخذ الحالات التي اكتشفها النظام، ونقارن المستقبل بعدها مع Baseline.

مثلاً بشكل مبسط جداً:

حالات FVG:

ماذا حدث بعدها؟

مقابل لحظات مشابهة بدون FVG.

ثم:

FVG + حالة قلب معينة.

ثم:

FVG + executed flow معين.

ثم لاحقاً:

FVG + structure + liquidity + flow.

لكن بطريقة مضبوطة حتى لا نجرب مليون تركيبة ونختار أجمل واحدة.

إذا ظهر أن المستقبل بعد هذه الحالات لا يختلف عن Baseline، هذه المعلومة لا تضيف قيمة واضحة.

إذا ظهر فرق في مايو، هذا مثير للاهتمام فقط، وليس إثباتاً بعد.

نأخذ شهوراً أخرى لم نستخدمها لاختيار الفكرة.

إذا بقي الفرق، وقتها المشروع يبدأ يثبت شيئاً حقيقياً.

ثم نبني العقل المتعلم.

ثم بعد ذلك فقط نصل إلى:

اتجاه الصفقة.

رقم الدخول.

رقم الوقف.

الأهداف.

حجم المركز.

نوع الأمر.

ومتى لا نتداول.

فالحلطة الحالية يمكن تلخيصها هكذا:

حتى أمس كنا نسأل:

"هل الآلة مبنية بشكل صحيح؟"

الآن نسأل:

"هل الآلة فعلاً تستطيع رؤية BTC الحقيقي؟"

والمرحلة التالية ستكون:

"هل الذي تراه الآلة يحمل معلومة عن المستقبل؟"

وبعدها:

"هل يمكن تحويل هذه المعلومة إلى مال بعد التكاليف؟"

اترك التشغيل الحالي يكمل. إذا انتهى بنجاح، أرسل لي SUMMARY.json و MACHINE_VALIDATION.json. إذا انتهى بخطأ، أرسل الخطأ كما هو ولا تغير أي إعداد من عندك.

C:\Users\pc\Desktop\usdt\28_BTC_MAY2026_FIELD_RUNNER_PACKAGE_FAST_MODE_PATCHED\BTC_MAY2026_FIELD_PACKAGE>python run_btc_may_2026.py

عم يشتغل لسا

شو لازم يعطيني لحنا يكون ناجح وشو بتتوقع يعطيني ومدى نجاحه ؟

نسقلي اجوبتك ك نص لانو عم يتخربط للكلمات والجمل اوقات

gpt-5.5-instant

الحالة الرسمية: Patch ال FAST mode جاهز. المشروع الرسمي لم يتغير: 183/183، والبيانات الحقيقية لا تزال موجودة عندك أنت فقط. الخطأ السابق تم تحديده وإصلاحه تحديداً، والrunner الآن يفترض FAST mode ولا يحتاج ملف ال1.69GB.

هلق نوقف البيلدر تماماً. دوره انتهى حالياً.

نزل هذا الملف الجديد فقط:

[text]

BTC_MAY2026_FIELD_RUNNER_PACKAGE_FAST_MODE_PATCHED.zip_28

البصمة المرجعية إذا أردت التأكد:

[text]

e43d6c96f1e19c79ef553ed04d7053ca63ab4e952fa6eeef3a7c7aaa82cb7895

فك الضغط في مجلد جديد. لا تفكه فوق الحزمة القديمة حتى ما تختلط الملفات.

داخل المجلد الجديد ستجد:

[text]

data

field_runner

trading_project

run_btc_may_2026.py

الآن من الحزمة القديمة عندك انسخ إلى مجلد data الجديد ملفين فقط:

[text]

minute_facts_BTCUSDT-aggTrades-2026-05.csv

وملف الـ JSON الجانبي الذي نتج معه من المحول.

لا تنسخ ملف الـ 1.69GB. خليه مكانه.

المفروض يصبح تقريباً:

[text]

```
28...PATCHED
|
|— data
|   |— minute_facts_BTCUSDT-aggTrades-2026-05.csv
|   |— <sidecar>.json
|
|— field_runner
|— trading_project
|— run_btc_may_2026.py
```

ثم افتح CMD داخل جذر الحزمة وشغل:

[cmd]

python run_btc_may_2026.py

بس.

ما لازم تعطيه مسارات، ولا Hash، ولا quantile، ولا RAW mode.

شو لازم يصير هذه المرة؟

أولاً سيتأكد:

[text]

Minute Facts

SHA256 = a6a06c...

ثم يفتح الـ sidecar ويتأكد أنه يقول:

[text]

هذه الـ Minute Facts أصلها

...Raw SHA256 = 86d4f3

وأن المحول هو:

[text]

004877...

بعدها سيحصل محلياً على Klines الرسمية الصغيرة من Binance إن كانت الشبكة متاحة.

ثم:

[text]

```
Minute Facts
+
Official Klines
↓
Source Adapter المغلاق
↓
Market Timeline
↓
المشروع الحقيقي
↓
التقرير
: تنبيه مهم: لا تستغرب إذا التقرير النهائي قال عن مجموعة أشياء
```

[text]

NOT_CONFIGURED

خصوصاً:

[text]

Swings

Structure

Liquidity

Order Blocks

Dealing Range

لأننا تعمدنا عدم اختراع swing_quantile.

لكن المفروض أشياء أخرى تعمل فعلياً، منها FVG والتقلب والتدفق المنفذ وبعض الفروع المستقلة.

إذا نجح، سيظهر مجلد:

[text]

outputs\btc_may_2026_reality

لا ترسل لي كل الملفات.

أرسل أولاً محتوى:

[text]

SUMMARY.json

و:

[text]

MACHINE_VALIDATION.json

وإذا كان صغيراً:

[text]

README_RESULT_AR.txt

وقتها لن نرجع للبيلدر فوراً. أنا وأنت أولاً سنقرأ محصول المشروع الحقيقي: ماذا اشتغل؟ ماذا لم يشتغل؟ كم FVG وجد؟ ماذا قال عن Flow؟ وما حجم الأشياء غير المهيأة؟

وبعد ذلك نقرر الخطوة التالية بناءً على النتائج الفعلية، لا بناءً على التصميم النظري.

FIELD RUNNER FAST-MODE PATCHED — READY FOR OWNER RETRY

Date: 2026-09-28 (Asia/Damascus)

Report ID: BTC_MAY2026_FIELD_RUNNER_V1_FAST_MODE_PATCH1

الـ field runner الخارجي. لا تعديل في trading_project ولا Source Adapter المغلاق ولا

converter ولا semantics.

الأمر الذي يُعيد المالك تشغيله (نفسه بالضبط): python run_btc_may_2026.py

0 — حدود البيانات (CRITICAL OWNER-DATA BOUNDARY — مُصَحَّحة هنا)

البيئة البنائية لا تملك أي artifact مالك: لا raw aggTrades (1.69 GiB)، لا minute-facts حقيقية، لا sidecar حقيقية. لا ادعاء تشغيل أي real-owner artifact هنا.

كل regression tests للـ PATCH: SYNTHETIC SMALL FIXTURES ONLY — لا تنزيل بديل، لا طلب نسخ الملف.

البصمات المعطاة (004877 / a6a06c... / ...86d4f3... = metadata مرجعية تُحْمَل كdefaults في الأمر — لا ملفات متاحة.

REAL OWNER RUN يحدث على جهاز المالك فقط، عند إعادة التشغيل.

1 — root cause

الخطأ الحرفي على جهازك:

[text]

AGGTRADES_HASH_MISMATCH

expected=86d4f3d3... ← RAW AGGTRADES SHA256

(!artifact صحيح) actual=a6a06c4c... ← MINUTE FACTS CSV SHA256

في اسمه على aggtrade يحوي النص *.csv كان يَصِف أي detect_inputs في role-aware الغير auto-discovery: السبب في اسمه ⇒ صُيِّف خطأ «aggTrades» يحوي minute_facts_BTCUSD-aggrTrades-2026-05.csv ملفك RAW AGGTRADES. أنه بالمعنى الخاطئ واسم الخطأ mismatch ⇒ artifact minute-facts على expectation raw بوابة الهوية طُبِّقت ⇒ RAW المَعْتَمَد MINUTE_FACTS لم يكن خاطئاً إطلاقاً — وهو بالصبط الـ artifact الـ (AGGTRADES_HASH_MISMATCH) الخاطئ.

الإصلاح (المبادئ المفروضة) — 2

(غامض substring لا) صريحة ومتنافية artifacts أدوار:

RAW_AGGTRADES ← expected 86d4f3d335ae244dcc143569bd1cc382320027c3fe4702e5d3ddf0757e8f1c04

MINUTE_FACTS_CSV ← expected a6a06c4c583e64e7613ddca275533a15062ab13864482cacf30b2f125094601f

MINUTE_FACTS_SIDEAR ← يُلزم: source_sha256 = 86d4f3... + output_csv_sha256 = a6a06c... + converter_sha256 = 0048779bd821dbe6f3cda66517b19e7435fc0b98592ee9aa1980804fe27efb5d

FAST mode (الافتراضي): raw aggTrades إذا القبول فقط إذا SHA256(facts) ==

sidecar.output_csv_sha256 == a6a06c... و sidecar.source_sha256 == 86d4f3... و sidecar.converter_sha256 == 004877... ملف غير موجود hashing لا عبر sidecar، في الـ provenance تُثَبِّت عبر سلسلة raw هوية — 004877...

نفس سلسلة (+) تحقّقاً فعلياً 86d4f3... ⇒ SHA256(raw) يجب أن يكون موجوداً raw: فقط (--mode raw) RAW mode (sidecar) الـ.

يستبعد) منفصل raw نمط ← minute-facts لاكتشاف الـ sidecar-driven ← أولاً explicit paths role-aware: discovery
⇒ داخل دور candidate أكثر من — (klines ويستبعد، aggTrades حتى لو اسمه يحوي، minute-facts كل ما صُيِّف مع عرض المسارات fail-closed).

رسوم artifact-specific (اختفت AGGTRADES_HASH_MISMATCH نهائياً):

MINUTE_FACTS_HASH_MISMATCH / RAW_AGGTRADES_HASH_MISMATCH /

SIDECAR_SOURCE_BINDING_MISMATCH / CONVERTER_BINDING_MISMATCH /

SIDECAR_OUTPUT_BINDING_MISMATCH / MINUTE_FACTS_AMBIGUOUS / RAW_AGGTRADES_AMBIGUOUS /

RAW_AGGTRADES_FILE_REQUIRED.

--allow-hash-mismatch لم يُستخدم كحل في أي اختبار (يبقى override مالك صريح فقط). لا hash guard عُطِّل. لا

semantics تغيّرت. swing_quantile يبقى null — لا engine جديد، لا Reality analysis.

3 — الملفات الخارجية المعدّلة (SHA256 قبل ← بعد)

file قبل بعد

field_runner/runner_btc_may_2026.py

bdf672270f76d70dba5253ddf588e66bf2c333f6b9c2841dd40a3767232207fc

a15672ca1ea68b672a66d1d327068fd4bc3f4b7af0aa10cc4038abf1ca4df74e

field_runner/pipeline.py 27045a57b5000d581b2817516f33226ef2d804f3f8c2d4cec16f92a5b5aedd27

58fe777e960ad77ed88cc1be49de8726182c27df8d10de3badbf771a2b440e5c

field_runner/fixtures.py (TEST-FIXTURE-ONLY)

414dba6cacce8f400b46bdb778df3942e11eb5c17ad2a69a34bf04de07828902

b62d7c1d00b5f53487c27e4445821ef1097342a2a1882ce931a70dbbe751afd3

field_runner/runner_tests/test_runner_integration.py

96baf7dd55c6b5560bca2599c4b16b5ac519e75a325e3c5117f80d47f08c3b38

6450a9e78bcb61b3e41bfe0458001bb55ad80987bfb558e535d5ba90aeaf4217

field_runner/runner_tests/test_artifact_roles.py — (جديد)

0bd27cb4c77f262ab6036dfcf48f724d84b06d49b1eef7bc50197bf629edf2c3

README_OWNER.txt 866c3d14bc8662b75d1e2b85e04e9aed4f91c309873c71b2c4bf0e266fe04724

(مُحدّث: وضع FAST/RAW + رسوم الأدوار)

غير مُلمس: config/field_run_config.json, sample.py, binance_kline_downloader.py, reporting.py (defaults)

كما هي — swing_quantile: null), وكل trading_project.

4 — regression result

python3 -m pytest field_runner/runner_tests ⇒ 31 passed / 0 failed (synthetic fixtures فقط):

FAST mode ⇒ raw CSV بلا، data/minute_facts_BTCUSD-aggrTrades-2026-05.csv + sidecar، حالة المالك حرفياً

سجل المحادثات — zzz.txt

RAW لا يُصنّف «aggTrades» إثبات أن الاسم الحاوي + PASS.
 PASS تشغيل كامل + discovery ⇒ (اكتشاف flags بلا runner_main) أمر المالك العاري
 minute facts + sidecar ⇒ PASS. صحيحان
 minute facts hash خاطئ ⇒ FAIL باسم MINUTE_FACTS_HASH_MISMATCH (وإثبات أن) AGGTRADES_HASH_MISMATCH غير (موجود).
 sidecar.source_sha256 خاطئ ⇒ FAIL SIDECAR_SOURCE_BINDING_MISMATCH.
 sidecar.converter_sha256 خاطئ ⇒ FAIL CONVERTER_BINDING_MISMATCH.
 sidecar.output_csv_sha256 خاطئ ⇒ FAIL SIDECAR_OUTPUT_BINDING_MISMATCH.
 raw mode ⇒ FAIL RAW_AGGTRADES_FILE_REQUIRED. غير موجود في raw ⇒ PASS. FAST غير موجود في raw
 مع عرض المسارين fail-closed ⇒ MINUTE_FACTS_AMBIGUOUS. ملفا minute facts ⇒
 raw + minute facts الصحيح ⇒ PASS! معاً ⇒ كل يُصنّف دوره الصحيح
 RAW_AGGTRADES_HASH_MISMATCH.
 defaults + default="fast". الأمر = بصمات المالك المرجعية الثلاث
 PATCH. كلها سليمة بعد الـ - leakage/fail-closed/determinism/no-model-pnl-signals/AST-guard (من قبل)
 CLOSED + baseline MANIFEST إثبات - 5
 MANIFEST: 183 سطرأ - sha256 = 7796a73fc30fc311902d7ba8e033eb1699a73c5db10e824d02653fbbdd1681587 لم -
 183/183 OK. يتغير - التحقق
 CLOSED UNCHANGED. - كاملة مقارنة ببصمة ما بعد الإغلاق: صفر ملفات متغيرة /trading_project شجرة
 المصحح (جاهز للمالك) ZIP - 6
 deliverables/28_BTC_MAY2026_FIELD_RUNNER_PACKAGE_FAST_MODE_PATCHED.zip

728,322) zip_sha256 = e43d6c96f1e19c79ef553ed04d7053ca63ab4e952fa6eeef3a7c7aaa82cb7895
 (bytes مُتحقق — 205 entries — round-trip 203/203)

البنية كما هي: run_btc_may_2026.bat + run_btc_may_2026.py + README_OWNER.txt + field_runner /مع
 اختبارات الـ (PATCH) + /trading_project كامل (187 ملفاً) + FILE_INVENTORY.sha256 +
 المالك لا يعدّل أي Python: يفك ZIP ← يضع sidecar + minute_facts في \data (اختياري) ← يشغل الأمر نفسه.
 7 — الأمر
 text

python run_btc_may_2026.py
 (أو run_btc_may_2026.bat). الافتراضي = FAST mode: البوابات الثلاث محقّلة ببصماتك المرجعية؛ أي mismatch ⇒
 اسم artifact صحيح في الرسالة.

8 — حدود
 لا تحليل نتائج BTC هنا (البيانات مآلك فقط). لا ادعاء Information Edge/ربحية. لا Model/Strategy/PnL.
 TIE_ORDER_CONTRACT = NOT_PROVEN. RESEARCH-DEBT-020..025 OPEN.
 REAL OWNER RUN = عندك، بهذا الأمر، على الـ ZIP المصحح.
 .STOP

gpt-5.5-instant

الحالة الرسمية: الـ Source Adapter مغلق، والـ Field Runner فقط يحتاج Patch صغير بسبب خلط أدوار الملفات. وأثبت
 عندي من الآن: البيلدر لا يملك أيّاً من بيانات السوق الحقيقية الخاصة بك.

ترتيب الملفات عندنا هو:

[text]

على جهازك أنت فقط:

(1) الملف الخام الكبير
 BTCUSDT-aggTrades-2026-05.csv

الحجم ≈ 1.69 GiB

SHA256 =

86d4f3d335ae244dcc143569bd1cc382320027c3fe4702e5d3ddf0757e8f1c04

(2) ملف Minute Facts

minute_facts_BTCUSDT-aggTrades-2026-05.csv

SHA256 =

a6a06c4c583e64e7613ddca275533a15062ab13864482cacf30b2f125094601f

(3) ملف JSON الجانبي الخاص بالتحويل

موجود عندك أيضاً.

أما عند البيلدر:

[text]

لا يوجد Raw AggTrades الحقيقي.

لا يوجد Minute Facts الحقيقي.

لا يوجد JSON الحقيقي كملف.

هو يعرف فقط المعلومات والبصمات التي نسختها له بالنص. لذلك من الآن أي مهمة أعطيها للبيدler سأتعامل معها بهذه القاعدة:

[text]

BUILDER = CODE ONLY + SYNTHETIC FIXTURES

OWNER PC = REAL MARKET DATA + REAL EXECUTION

ولا نطلب منه مرة ثانية «شغل مايو الحقيقي» أو «اختبر الملف الحقيقي». مستحيل عنده أصلاً.

وبالنسبة للـ Patch الذي أعطيتك إياه: مناسب لهذا الوضع تحديداً. هو لا يحتاج ملفك. يجب أن يصنع Fixture صغيراً بأسماء مشابهة فقط لإعادة إنتاج خطأ التصنيف، مثلاً:

[text]

minute_facts_BTCUSDT-aggTrades-2026-05.csv

لكن محتواه صناعي من 3 أو 10 صفوف.

وبعد ما يصلح الـ runner ويعطيك ZIP جديد، أنت وحدك تشغله على الملفين الحقيقيين.

وأريد أيضاً تبسيط التشغيل أكثر: بما أن عندك الملف الخام والـ Minute Facts معاً، الأفضل ألا يعتمد الـ runner على البحث التلقائي الغامض أصلاً. نخليه يعرف أن الأولوية هي:

[text]

FAST MODE

↓

Minute Facts + Sidecar

↓

لا يقرأ 1.69GB

والملف الخام يبقى عندك كمرجع إثبات فقط.

فقط إذا قلت له صراحة:

[text]

RAW MODE

يقرأ الملف الكبير ويعيد التحويل.

هذا يوفر وقتك ويمنع خلط الملفات.

إذا لم تكن أرسلت بروميت الـ Patch السابق للبيدler بعد، أرسل نفس البروميت السابق وأضف في أوله هذه الكتلة:

[text]

CRITICAL OWNER-DATA BOUNDARY

لا يوجد عندك أي ملف بيانات حقيقي للمالك.

OWNER PC ONLY:

- raw aggTrades 1.69 GiB
- real minute-facts CSV
- real minute-facts sidecar JSON

BUILDER ENVIRONMENT:

- NONE OF THE ABOVE.

ممنوع ادعاء تشغيل أي real-owner artifact.

ممنوع تنزيل بديل عنه.

ممنوع طلب نسخ الملف الكبير.

كل regression tests للـ Patch:

SYNTHETIC SMALL FIXTURES ONLY

البصمات المعطاة في المهمة هي metadata مرجعية فقط، وليست ملفات متاحة لك.

الـ ZIP المصحح سيُشغّل لاحقاً على جهاز المالك، وهناك فقط يحدث REAL OWNER RUN.

هيك صارت الحدود واضحة عندي ولن أبني الخطوات القادمة على افتراض أن البيدler عنده بياناتك.

هلق انا معي المينت فاست وملف الضخم الكيكا عالكمبيوتر عندي مو موجودين ابدا عند البيلدر خلي ببالك هالشي

gpt-5.5-instant

الحالة الرسمية: الـ Field Runner بدأ فعلياً على جهازك، لكنه توقف قبل تشغيل أي محرك لأن حارس الهوية اكتشف أن الملف الذي أعطيته له هو ملف الدقائق، بينما الإعداد يتوقع بصمة ملف الـ aggTrades الخام. هذا توقف صحيح، وليس فساداً في بياناتك. الرسالة نفسها تنشر بالضبط ماذا حصل:

المتوقع:

[text]

86d4f3d335ae244dcc143569bd1cc382320027c3fe4702e5d3ddf0757e8f1c04

وهذه بصمة ملفك الخام:

[text]

BTCUSDT-aggTrades-2026-05.csv

لكن الذي وجدته:

[text]

a6a06c4c583e64e7613ddca275533a15062ab13864482cacf30b2f125094601f

وهذه بالضبط بصمة:

[text]

minute_facts_BTCUSDT-aggTrades-2026-05.csv

يعني الملف صحيح، لكن الـ runner عامل متغير/مسار الـ minute_facts كأنه مسار الـ aggTrades الخام. هذه مشكلة تكامل في واجهة الـ runner، لا مشكلة بالبيانات ولا بالـ Adapter المغلق.

مهم جداً: لا تستخدم:

[text]

allow-hash-mismatch--

رغم أن الرسالة تقترحه. هذا سيسكت حارساً اكتشف مشكلة حقيقية، وليس هذا ما نريده.

وأيضاً لا تعيد معالجة 1.69GB. عندنا minute-facts صحيحة ومصالحتها كاملة، والـ runner نفسه ضمم ليدعم المسار السريع.

المطلوب الآن صغير جداً: البيلدر يصلح الـ runner الخارجي فقط بحيث يميز بوضوح بين هويتين:

[text]

Raw AggTrades SHA256

= 86d4f3...

Minute Facts SHA256

= a6a06c...

ولا يطالب بأن يكون ملف الـ minute-facts مطابقاً لبصمة raw.

أرسل للبيلدر هذا النص:

[text]

FIELD RUNNER — OWNER LOCAL BLOCKER — PATCH ONLY

التشغيل الحقيقي الأول على Windows توقف قبل تشغيل أي engine.

الأمر:

python run_btc_may_2026.py

الخطأ الحرفي:

AGGTRADES_HASH_MISMATCH

expected=86d4f3d335ae244dcc143569bd1cc382320027c3fe4702e5d3ddf0757e8f1c04

actual=a6a06c4c583e64e7613ddca275533a15062ab13864482cacf30b2f125094601f

الحقائق:

RAW AGGTRADES SHA256:

86d4f3d335ae244dcc143569bd1cc382320027c3fe4702e5d3ddf0757e8f1c04

MINUTE FACTS CSV SHA256:

a6a06c4c583e64e7613ddca275533a15062ab13864482cacf30b2f125094601f

المالك وضع sidecar + minute-facts CSV في data/ كما طلب تقريرك.

إذن actual=a6a06... ليس ملفاً خاطئاً:
إنه بالضبط minute-facts artifact المعتمد.

المشكلة:

الـ runner يطبق expectation identity/hash raw AggTrades على minute-facts artifact، أو auto-discovery
يربط artifact role خاطئاً.

PATCH ONLY للـ runner field الخارجي.

ممنوع:

- تعديل .trading_project
- تعديل .Source Adapter CLOSED
- تعديل .converter
- استخدام --allow-hash-mismatch كحل.
- تعطيل أي hash guard
- مطالبة المالك بإعادة تحويل 1.8GB.
- تنزيل aggTrades بديل.
- أي تغيير في semantics.

المطلوب:

1. افصل artifact roles صراحةً:

RAW_AGGTRADES

expected hash:

86d4f3d335ae244dcc143569bd1cc382320027c3fe4702e5d3ddf0757e8f1c04

MINUTE_FACTS_CSV

expected hash:

a6a06c4c583e64e7613ddca275533a15062ab13864482cacf30b2f125094601f

MINUTE_FACTS_SIDECAR

يجب أن يربط:

source_sha256 = raw aggTrades hash

output_csv_sha256 = minute facts hash

converter_sha256 =

0048779bd821dbe6f3cda66517b19e7435fc0b98592ee9aa1980804fe27efb5d

2. في FAST mode:

لا يحتاج raw aggTrades file أن يكون موجوداً.

يُقبل:

minute facts CSV

+

minute facts sidecar

فقط إذا:

SHA256(minute facts CSV)

==

sidecar.output_csv_sha256

==

a6a06...

sidecar.source_sha256

```
==
86d4f3...
```

sidecar.converter_sha256

```
==
004877...
```

أي أن raw identity ثبت عبر سلسلة provenance المسجلة في sidecar، لا عبر hashing ملف raw غير موجود.

3. في RAW mode فقط:

إذا أعطى المالك raw aggTrades file، عندها تحقق فعلياً من SHA256(raw)

```
==
86d4f3...
```

ثم converter

→ minute facts

4. أصلح auto-discovery:

لا يجوز اختيار minute_facts*.csv كـ raw aggTrades بسبب احتواء الاسم على aggTrades.

تعريف discovery يجب أن يكون role-aware، لا substring ambiguous.

الأفضل:

- explicit paths أولاً.

- sidecar-driven minute-facts discovery.

- raw discovery pattern - منفصل.

- ambiguity بين أكثر من fail-closed ⇒ candidate مع عرض المسارات.

5. الرسائل:

لا تسم mismatch للـ minute-facts:

.AGGTRADES_HASH_MISMATCH

استخدم artifact-specific errors:

RAW_AGGTRADES_HASH_MISMATCH

MINUTE_FACTS_HASH_MISMATCH

SIDECAR_SOURCE_BINDING_MISMATCH

CONVERTER_BINDING_MISMATCH

6. أضف regression test يعيد حالة المالك حرفياً:

data/

minute_facts_BTCUSDT-aggTrades-2026-05.csv

+ sidecar

ولا raw CSV.

يجب أن FAST mode يمر.

ويجب إثبات أن:

"minute-facts filename containing "aggTrades"

لا يُصنف RAW.

7. أضف هجمات:

- minute facts صحيحة + sidecar صحيح ⇒ PASS.

- minute facts hash خاطئ ⇒ FAIL.

- sidecar source raw hash خاطئ ⇒ FAIL.

- converter hash خاطئ ⇒ FAIL.

- raw FAST mode غير موجود في ⇒ PASS.

- raw RAW mode غير موجود في ⇒ FAIL.

- minute facts ⇒ ambiguity fail-closed.

- موجودان ⇒ كل واحد يصنف بدوره الصحيح minute facts و raw ملف.

سجل المعادلات — zzz.txt